



SISTEMA INTELIGENTE DE ASISTENCIA AL DESARROLLO ESTRUCTURADO Y SEGUIMIENTO DE PROYECTOS DE SOFTWARE

ADRIA CEBRIAN RUIZ

Director/a

LUIS JONÁS CUERVO DERQUI (COLBAI ADVISORS SL)

Ponente: DAVID GARCIA SORIANO (Departamento de Ciencias de la Computación)

Titulación

Grado en Ingeniería Informática (Computación)

Memoria del trabajo de fin de grado

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

30/06/2026

Abstract

The present project consists of the design and application of an intelligent assistance system, with the objective of automating the various stages in the development of software projects —requirements gathering, Spec-Driven Development, and client follow-up— through the orchestration of generative Artificial Intelligence tools. Placing emphasis on the interaction with the human developer, following a philosophy of *Human in the loop*, and the generation of documentation to guide the process. This methodology aims to reduce the quality loss typically caused by the use of AI in the development of these projects. The experimental evaluation conducted suggests improvements in code quality and maintainability in greenfield scenarios, with mixed results in refactoring tasks when using the proposed system. This final degree project is applied and developed for the working environment at the software consulting company *Colb.AI*.

Keywords: Spec-Driven Development (SDD), Large Language Models (LLM), Coding agents, Prompt engineering, Human-in-the-Loop (HITL), Software development

Resumen

El presente proyecto consiste en el diseño y aplicación de un sistema para la automatización de las diversas etapas en el desarrollo de proyectos de *software* —toma de requerimientos, *Spec-Driven Development* y seguimiento del cliente— utilizando principalmente la orquestación de herramientas de Inteligencia Artificial generativa. Haciendo un especial énfasis en la interacción de estos sistemas de automatización con el desarrollador, siguiendo una filosofía de *Human in the loop*, y la generación de documentación para guiar el proceso. Esta metodología busca reducir la pérdida de calidad típicamente resultante del uso de IA en el desarrollo de los proyectos. La evaluación experimental realizada sugiere mejoras en la calidad y mantenibilidad del código en proyectos comenzados de cero, con resultados variados para la edición de un proyecto ya existente al utilizar el sistema propuesto. Este trabajo de final de grado se aplica y desarrolla para el entorno laboral de la consultora de *software Colb.AI*.

Palabras clave: Spec-Driven Development (SDD), Large Language Models (LLM), Agentes de código, Ingeniería de *prompts*, Human-in-the-Loop (HITL), Desarrollo de software

Resum

El present projecte consisteix en el disseny i aplicació d'un sistema intel·ligent per a l'automatització de les etapes en el desenvolupament de projectes de *software* —recollida de requeriments, *Spec-Driven Development* i seguiment del client— aplicant principalment l'orquestració d'eines d'Intel·ligència Artificial generativa. Es fa un especial èmfasi en la interacció d'aquests sistemes amb el desenvolupador humà, seguint una filosofia de *Human in the loop*, i la generació de documentació per a guiar el procés. Aquesta metodologia tracta de reduir la pèrdua de qualitat típicament conseqüència de l'ús de la IA en el desenvolupament d'aquests projectes. L'avaluació experimental realitzada suggereix millores en la qualitat i mantenibilitat del codi en projectes nous, amb resultats variats per a la modificació de projectes ja existents, en utilitzar el sistema proposat. Aquest treball de final de grau s'aplica i desenvolupa per a l'entorn laboral de la consultoria de *software Colb.AI*.

Paraules clau: Spec-Driven Development (SDD), Large Language Models (LLM), Agents de codi, Enginyeria de *prompts*, Human-in-the-Loop (HITL), Desenvolupament de *software*

Índice

1. Introducción y contexto	12
1.1. Definición del Problema	12
1.1.1. Toma de requerimientos y planificación de la arquitectura	13
1.1.2. Desarrollo de código	13
1.1.3. Seguimiento del cliente	13
1.2. Conceptos Relevantes Previos	14
1.2.1. Spec-Driven-Development (SDD)	14
1.2.2. Toma de requerimientos	14
1.2.3. Asistentes de Inteligencia Artificial y Agentes Basados en LLMs	14
1.2.4. Transcripción automática y diarización	15
1.3. Stakeholders	15
1.3.1. Desarrolladores	15
1.3.2. Consultores	15
1.3.3. Cliente y sus usuarios	16
1.4. Estado del arte y soluciones existentes	16
1.4.1. Spec Kit	16
1.4.2. AgentOS	16
1.4.3. Asistentes Comerciales	16
1.4.4. Herramientas de transcripción de reuniones	16
1.4.5. Herramientas de documentación de reuniones	17
1.4.6. Claude Cowork	17
1.4.7. Asistentes CLI extensibles	17
1.4.8. superpowers plugin	17
1.4.9. Enfoques académicos: planificación previa y agentes múltiples	17
1.5. Análisis comparativo: ¿Está justificada una solución a medida?	18
1.6. Propuesta	20
1.7. Flujo de trabajo completo	20
2. Alcance del Proyecto	22
2.1. Objetivo General	22
2.2. Requerimientos	23
2.2.1. Requerimientos Funcionales	23
2.2.2. Requerimientos No Funcionales	23
2.3. Métricas de Evaluación del Código	24
2.3.1. Complejidad Ciclomática	24
2.3.2. Índice de Mantenibilidad	24
2.4. Obstáculos y Riesgos	25
2.5. Integración de conocimientos	26
2.5.1. Competencias técnicas del proyecto	26
2.5.2. Asignaturas	27
2.6. Revisión del alcance inicial	28
3. Metodología	29
3.1. Metodología de Trabajo	29
3.2. Herramientas de Seguimiento y Control	29

3.3. Uso de IA generativa en el desarrollo del proyecto	30
4. Gestión del Proyecto	31
4.1. Planificación	31
4.1.1. GP - Gestión del proyecto	31
4.1.2. TP - Trabajo Previo	32
4.1.3. DA - Desarrollo Ágil (Sprints Iterativos)	32
4.1.4. PV - Pruebas y Validación	33
4.1.5. Recursos	35
4.1.6. Tabla resumen de tareas	36
4.1.7. Diagrama de Gantt	38
4.1.8. Revisión de la planificación inicial	40
4.2. Gestión de riesgos: Planes alternativos y obstáculos	41
4.2.1. Riesgo 1: Cambios repentinos en los requerimientos por parte de la empresa .	41
4.2.2. Riesgo 2: Bloqueo tecnológico (Vendor Lock-in) por cambio de precios o políti- cas en las APIs de los LLMs	41
4.2.3. Riesgo 3: Documentación interna insuficiente para la generación autónoma de código estructurado	42
4.2.4. Riesgo 4: Limitaciones de control sobre el flujo de orquestación en el ecosis- tema IDE	42
4.2.5. Riesgo 5: Desbordamiento de la ventana de contexto en proyectos grandes . .	43
4.2.6. Riesgo 6: Exceso de fricción en el bucle de desarrollo por auditorías demasiado estrictas	43
4.3. Gestión económica	44
4.3.1. Coste del personal	44
4.3.2. Coste por actividad	44
4.3.3. Costes genéricos	45
4.3.4. Contingencias e imprevistos	47
4.3.5. Coste total	48
4.3.6. Revisión de los costes iniciales	48
4.3.7. Control de gestión	49
4.4. Informe de sostenibilidad	51
4.4.1. Autoevaluación de la competencia	51
4.4.2. Dimensión económica	51
4.4.3. Dimensión social	52
4.4.4. Dimensión ambiental	52
4.5. Leyes y regulaciones	53
5. Módulos de Transcripción y Documentación	54
5.1. Toma de Requerimientos	54
5.1.1. Módulo de transcripción	54
5.1.2. Módulo de procesamiento de requerimientos	56
5.2. Seguimiento del Cliente	60
5.2.1. Generación de documentación con Claude Cowork	60
5.2.2. Alternativa de código abierto	61
6. Spec Driven Development: Implementación	62
6.1. Asistentes de Programación	62

6.1.1.	ClaudeCode	62
6.1.2.	OpenCode	62
6.1.3.	Justificación de la doble integración	62
6.1.4.	Arquitectura interna: el patrón ReAct y su implementación en TypeScript . .	63
6.2.	Mecanismos de extensibilidad	65
6.2.1.	Conceptos clave	65
6.2.2.	El ciclo de vida completo de una sesión en OpenCode	66
6.2.3.	Similitud con ClaudeCode	68
6.2.4.	Hooks de ciclo de vida	69
6.2.5.	Skills y agentes en colbPowers	69
6.3.	Herramientas de Apoyo al Flujo SDD	70
6.3.1.	code-review-graph	70
6.3.2.	Definición del flujo de trabajo, colbPowers	71
6.3.3.	La plantilla de repositorio	74
7.	Experimentos y evaluación	75
7.1.	Diseño experimental	75
7.2.	Caso 1: Aplicación Flutter	76
7.3.	Caso 2: Visualizador de clasificadores en Python	77
7.4.	Discusión de los resultados	79
7.4.1.	Análisis de consumo	79
7.4.2.	Métricas de calidad	80
8.	Conclusiones	86
8.1.	Conclusiones generales	86
8.2.	Trabajo futuro	90
8.2.1.	Plugin colbPowers y flujo SDD	90
8.2.2.	Módulos de transcripción y documentación	91
	Referencias	92
A.	Tablas completas análisis estadístico	96
B.	Gráficas adicionales del experimento	98

Índice de figuras

1.	Flujo de trabajo completo del sistema: desde la primera reunión con el cliente hasta el seguimiento iterativo del proyecto.	21
2.	Diagrama de Gantt, creado con el software <i>GanttProject</i>	39
3.	Diagrama de flujo de el proceso de transcripción.	56
4.	Skill de toma de requerimientos en Claude Cowork.	57
5.	Diagrama de flujo de el proceso generación de plan en la toma de requerimientos. . .	59
6.	Skill de seguimiento del cliente en Claude Cowork.	60
7.	Flujo de la implementación propia del módulo de seguimiento del cliente: generación del resumen inicial y bucle iterativo de refinamiento con <i>Human in the Loop</i>	61
8.	Ciclo iterativo ReAct: el agente alterna entre Razonamiento, Acción y Observación hasta que no hay más herramientas que invocar.	63
9.	Ciclo de vida completo de una sesión en OpenCode: desde la inicialización perezosa hasta la entrega de la respuesta, con el ciclo ReAct en el núcleo del bucle conversacional.	66
10.	Gráfico estructural del código generado por code-review-graph sobre el repositorio del proyecto (izquierda) y leyenda de tipos de nodo y arista (derecha). Generado sobre un repositorio ya existente de <i>Colb.AI</i>	71
11.	Diagrama de flujo de colbPowers.	73
12.	Tokens totales por experimento y condición, separados por asistente. La comparación entre columnas de distinto asistente no es directa por diferencias de tokenizador y caché.	79
13.	Test t de Welch — colbPowers vs. baseline, OpenCode, Caso 1.	79
14.	Test t de Welch — colbPowers vs. baseline, ClaudeCode, Caso 1.	80
15.	Métricas de calidad del código — Caso 2 (ClassificationVisualization, radon/Python). Medias por asistente y condición.	81
16.	Efecto de colbPowers sobre la calidad del código en el Caso 2 (delta = colbPowers – baseline; verde: mejora, rojo: empeora).	81
17.	colbPowers vs. baseline, OpenCode, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).	81
18.	colbPowers vs. baseline, ClaudeCode, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).	82
19.	OpenCode vs. ClaudeCode, baseline, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).	82
20.	OpenCode vs. ClaudeCode, colbPowers, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).	83
21.	Métricas de calidad del código — Caso 1 (Repo Existente, DCM/Flutter). Medias por asistente y condición.	83
22.	Efecto de colbPowers sobre la calidad del código en el Caso 1 (delta = colbPowers – baseline; verde: mejora, rojo: empeora).	84
23.	OpenCode vs. ClaudeCode, baseline, Caso 1. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo.	84
24.	OpenCode vs. ClaudeCode, colbPowers, Caso 1. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo.	85
25.	Análisis estadístico: colbPowers vs. baseline. Un $p < 0,05$ permite rechazar la hipótesis nula; un p alto no descarta el efecto, solo que el test no lo ha detectado. La g de Hedges cuantifica el tamaño del efecto.	96

26.	Análisis estadístico: OpenCode vs. ClaudeCode. Un $p < 0,05$ permite rechazar la hipótesis nula; un p alto no descarta el efecto, solo que el test no lo ha detectado. La g de Hedges cuantifica el tamaño del efecto.	97
27.	Desglose de tokens por tipo (input, output, caché leída) para cada experimento y condición.	98
28.	Tool calls totales del orquestador y número de subagentes por sesión, por condición y experimento.	99
29.	Llamadas al plugin code-review-graph por condición y experimento.	99
30.	Ratios de eficiencia: porcentaje de tokens servidos desde caché y porcentaje de tokens consumidos por el orquestador.	100
31.	Dispersión: tokens totales vs. métricas de calidad del código en ambos experimentos. Fila superior: Repo Existente (DCM); fila inferior: ClassificationVisualization (radon). Cada punto representa una sesión individual.	101
32.	Distribución de tool calls del orquestador por tipo (top 10 herramientas, sesiones con colbPowers).	102

Índice de tablas

1.	Comparativa de soluciones existentes.	19
2.	Comparativa entre el alcance comprometido en el GEP y el alcance final ejecutado. .	28
3.	Resumen de las tareas del proyecto con sus dependencias y recursos asignados. . . .	37
4.	Distribución estimada de horas del autor por rol a lo largo del proyecto.	38
5.	Comparativa entre la planificación del GEP y la planificación final ejecutada. . . .	40
6.	Costes de personal por hora según el rol asumido. Basada en información extraída de [26].	44
7.	Cálculo de la amortización de los recursos de <i>Hardware</i>	46
8.	Presupuesto total estimado del proyecto.	48
9.	Conceptos clave del sistema de extensibilidad de los asistentes CLI.	65
10.	Comparativa arquitectónica entre ClaudeCode y OpenCode.	69
11.	Condiciones experimentales.	75
12.	Caso 1 — OpenCode con DeepSeek V4-Flash. Caché = tokens leídos de caché. Métricas de calidad calculadas con DCM.	77
13.	Caso 1 — ClaudeCode con Haiku 4.5. Caché = tokens leídos de caché. Métricas de calidad calculadas con DCM.	77
14.	Caso 2 — OpenCode con DeepSeek V4-Flash. Caché = tokens leídos de caché. . . .	78
15.	Caso 2 — ClaudeCode con Haiku 4.5. Caché = tokens leídos de caché.	78

Lista de siglas

API	Application Programming Interface
CC	Complejidad Ciclomática
CLI	Command-Line Interface
CRG	Code-Review-Graph
DCM	Dart Code Metrics
HITL	Human-In-The-Loop
IDE	Integrated Development Environment
LLM	Large Language Model
LoC	Lines of Code
MCP	Model Context Protocol
MI	Maintainability Index
MVP	Minimum Viable Product
ReAct	Reasoning + Acting
SAS	Shared Access Signature
SDK	Software Development Kit
SDD	Spec-Driven Development
WSL	Windows Subsystem for Linux

1. Introducción y contexto

Este trabajo de final de grado consiste en la creación de un sistema de automatización para el desarrollo y diseño de proyectos de software, utilizando como pilar principal herramientas basadas en *LLMs*. El proyecto se enmarca para su uso dentro de una empresa real, *Colb.AI*, dedicada principalmente a la consultoría de proyectos de software.

Es innegable que las herramientas de inteligencia artificial, principalmente los *Large Language Models*, están cambiando la forma en que se desarrollan los proyectos de software para la mayor parte de las empresas del sector [29]. Estas tecnologías presentan la oportunidad de automatizar muchas de las tareas que tradicionalmente habían sido llevadas a cabo exclusivamente por los desarrolladores humanos, pero su adopción sin la supervisión humana adecuada conlleva riesgos que pueden comprometer la calidad y sostenibilidad del software generado, sobre todo a largo plazo.

Se presentan a continuación tres módulos que se han automatizado en el desarrollo de este TFG:

1. **Toma de requerimientos y diseño de arquitectura:** Necesidades del cliente a partir de la transcripción de reuniones, exploración de *frameworks* posibles a utilizar, y diseño de la arquitectura del proyecto.
2. **Desarrollo del código:** Asistencia de programación al desarrollador, generación de código a partir de las especificaciones. Evitando la generación de código de baja calidad, principalmente mediante la generación de documentación y especificaciones previas a la generación de código.
3. **Seguimiento del cliente:** Generación de informes de seguimiento para el cliente, a partir de la transcripción de las reuniones periódicas con el mismo.

1.1. Definición del Problema

El reto central de la automatización de estas tareas sin perder calidad, es conseguir un equilibrio entre la agilidad que ofrecen los asistentes de inteligencia artificial y la involucración de un desarrollador que garantice que cada paso cubre unos estándares de estructura y calidad. Este equilibrio es difícil de mantener en la práctica, y su dificultad puede explicarse a través de un principio en el ámbito de la psicología cognitiva denominada como *The Law of Least Effort*, o en otras palabras, la ley del mínimo esfuerzo. Esta teoría sostiene que los humanos y otros organismos tienden a priorizar la opción que presenta la menor resistencia para alcanzar sus objetivos, ya sea en términos de esfuerzo físico o cognitivo[32]. Aunque pueda parecer paradójico —puesto que a las personas también nos gustan los retos y los puzles [27]—, este fenómeno presenta una posible explicación al reto principal de la automatización, y es que el desarrollador tiene la tentación de delegar todo el trabajo en el asistente de IA sin involucrarse lo suficiente, comprometiendo así los estándares de calidad y estructura necesarios para elaborar un proyecto profesional y sostenible a largo plazo.

A continuación, se hace una definición más específica de los problemas y puntos a cubrir detectados en cada apartado del proyecto.

1.1.1. Toma de requerimientos y planificación de la arquitectura

La toma de requerimientos es una fase especialmente crítica en el desarrollo de software, puesto que sienta las bases del proyecto: su alcance, las tecnologías a utilizar y la arquitectura del sistema. Si esta fase no se lleva a cabo con el adecuado rigor, puede suponer todo tipo de imprevistos en la etapa de desarrollo del proyecto, causando retrasos en la entrega del proyecto, incrementos en el presupuesto y, en ocasiones, dejando inservibles módulos ya implementados [55].

Por lo tanto, es crucial que este apartado se lleve a cabo con la máxima calidad y supervisión humana posible, por lo que la automatización de este módulo debe tener un componente importante de interacción con el diseñador.

1.1.2. Desarrollo de código

Actualmente, los asistentes de código más populares (GitHub Copilot, Cursor, Claude Code, etc.) operan bajo un paradigma *Code-Driven*: el desarrollador describe vagamente lo que quiere y el modelo genera código de forma inmediata, sin exigir ninguna especificación previa. Este enfoque tiene tres consecuencias negativas directas [58]:

- **Generación de *slop code*:** El código producido resuelve el problema inmediato, pero carece de estructura coherente, no sigue los estándares del proyecto y acumula deuda técnica [14]. En [46] se evaluó empíricamente este fenómeno generando más de 1.600 programas con GitHub Copilot, y se encontró que aproximadamente el 44 % contenía vulnerabilidades de seguridad, poniendo de manifiesto los riesgos de delegar la generación de código a agentes de IA sin una estructura de supervisión adecuada. Al no haber una especificación de referencia, el modelo no puede saber qué es correcto más allá de lo que el usuario describe en ese momento. El resultado es un código difícil de mantener, escalar o auditar.
- **Falta de trazabilidad entre requisitos y código:** Sin un documento de especificación que actúe como fuente de verdad, resulta imposible saber si el código generado cumple realmente con los requisitos del negocio, si ha habido desviaciones respecto al diseño original, o quién tomó qué decisión y cuándo.
- **Barrera de incorporación para nuevos desarrolladores:** La ausencia de documentación estructurada convierte la incorporación de un nuevo miembro al equipo en un proceso lento y costoso. Sin especificaciones claras que expliquen el porqué de cada decisión de diseño, el nuevo desarrollador debe deducir la arquitectura y las reglas del proyecto leyendo directamente el código, lo que ralentiza su productividad y aumenta el riesgo de que introduzca cambios inconsistentes con la visión original del sistema.

Aunque existen herramientas que sí exigen especificación previa como *Spec Kit* o *AgentOS*, estas no se adaptaban adecuadamente al flujo de trabajo de *Colb.AI*, su modelo de documentación era demasiado rígido y generaba una fricción excesiva incluso para cambios menores.

1.1.3. Seguimiento del cliente

Este apartado se considera considerablemente relevante en el ámbito de la consultoría, debido a que la relación y el involucramiento con el cliente son un componente esencial para que un proyecto de software progrese adecuadamente. En la empresa *Colb.AI* se acostumbra a escribir un documento *Word* de seguimiento para el cliente después de cada reunión de seguimiento, para dejarle al cliente

por escrito los temas tratados en la reunión, con las tareas cubiertas, las pendientes por cubrir, y el estado actual de la implementación.

1.2. Conceptos Relevantes Previos

Los principales conceptos y definiciones relevantes que envuelven los ámbitos tratados en este TFG son los siguientes:

1.2.1. Spec-Driven-Development (SDD)

El *Spec-Driven Development* (SDD) es una filosofía donde la documentación detallada de los requisitos y el comportamiento esperado del sistema (la especificación) dirige la creación del software. A diferencia del enfoque tradicional, donde estos documentos descriptivos solían quedar subordinados o desactualizarse frente al código, el SDD invierte esta relación. El documento estructurado en lenguaje natural actúa como la base de la verdad sobre la que se construye el código. [43] [20]

En otras palabras, hay que tener definido previamente exactamente qué queremos que haga cada funcionalidad de código, y cómo queremos que lo haga. Cuando se quiera hacer cambios en la implementación, se han de aplicar primero en los documentos de especificación y luego en el código del proyecto.

1.2.2. Toma de requerimientos

La toma de requerimientos (en inglés *Requirements Gathering*) es una fase fundamental dentro de la ingeniería de software. En ella se investigan, descubren y extraen las necesidades de los clientes, usuarios y otras partes interesadas del sistema. A través de conversaciones con un cliente, que en muchas ocasiones no está muy seguro de qué quiere exactamente, el analista debe generar diseños y especificaciones funcionales específicas con comportamientos detallados [28] [52].

1.2.3. Asistentes de Inteligencia Artificial y Agentes Basados en LLMs

En el ámbito del desarrollo de software, los Modelos de Lenguaje de Gran Tamaño (LLMs) son la base tecnológica tanto de los asistentes de código como de los sistemas de procesamiento de documentos. Un asistente basado en LLM actúa como un colaborador interactivo que puede ayudar al desarrollador a generar código, pero también a analizar transcripciones de reuniones, estructurar requerimientos o redactar documentación de seguimiento. Para comprender cómo estas herramientas se estructuran y evolucionan en este proyecto, es necesario definir los siguientes conceptos clave:

- **Instrucciones (*Prompts*):** Son las entradas en lenguaje natural que el usuario utiliza para comunicarse con el modelo. Un *prompt* define la tarea a realizar y proporciona el contexto y las restricciones necesarias.
- **Agentes de Código:** Un agente es un asistente al que se le asigna un rol mediante un *prompt* y se le equipa con distintas *Agent Skills* y herramientas varias. Uno de sus usos mas útiles es que un agente orquestador lo pueda invocar para que tenga un contexto limpio, y así no esté sesgado por la conversación que se ha tenido previamente. Su propósito es llevar a cabo una

tarea específica. Además, es posible encadenar varios agentes especializados para construir un sistema coordinado. [38] [59]

- **Habilidades del Agente (*Agent Skills*):** Son documentos de instrucciones procedimentales en formato *Markdown* que el agente consulta bajo demanda cuando necesita ejecutar un tipo de tarea específico. Al activar una *Skill*, el agente lee primero su contenido, que define *cómo* debe razonar y actuar ante esa clase de tarea, y puede además invocar *scripts* o plantillas incluidos en la propia *Skill* durante su ejecución. [1] [61]

1.2.4. Transcripción automática y diarización

La transcripción automática es el proceso de convertir el audio de una conversación en texto estructurado mediante modelos de reconocimiento de voz. Un concepto clave asociado es la *diarización* (*speaker diarization*): la capacidad de identificar y etiquetar qué fragmento de texto fue pronunciado por cada hablante. En el contexto de la toma de requerimientos, esta atribución resulta especialmente relevante, ya que permite distinguir qué peticiones o restricciones provienen del cliente y cuáles son comentarios del equipo técnico, mejorando así la calidad de la especificación resultante. [49]

1.3. Stakeholders

Los principales stakeholders implicados en esta situación son los siguientes:

1.3.1. Desarrolladores

Los primeros beneficiarios de la implantación de este sistema son los desarrolladores y diseñadores de software. Al implementar metodologías de código orientadas a la creación de código estructurado y documentado, el mantenimiento del código se haría mucho más sencillo, así como el desarrollo y entendimiento de este. Modularizando las distintas *features* e implementaciones de código, facilitaríamos el entendimiento de la estructura general del producto, lo cual resultaría altamente beneficioso para su futura escalabilidad y mantenimiento. Además, el módulo de toma de requerimientos les proporciona las especificaciones ya estructuradas antes de comenzar el desarrollo, reduciendo la ambigüedad y el tiempo invertido en interpretar notas informales de reuniones.

1.3.2. Consultores

Los consultores son los miembros del equipo responsables de gestionar la relación con el cliente y coordinar el avance del proyecto. Son los principales beneficiarios de los módulos de toma de requerimientos y seguimiento del cliente: actualmente invierten un tiempo considerable en estructurar manualmente las necesidades expresadas por el cliente en conversaciones informales y en redactar la documentación de seguimiento posterior a cada reunión. La automatización de estas tareas les permite centrarse en el trabajo de mayor valor añadido: el análisis, la toma de decisiones y la relación directa con el cliente.

1.3.3. Cliente y sus usuarios

Tener un código bien estructurado y entendido en profundidad por sus creadores significa un producto de mayor calidad, más fácil de mantener y, en muchos casos, más seguro y eficiente. Además, el cliente se beneficia directamente del módulo de seguimiento: recibe documentación estructurada y consistente tras cada reunión, con los acuerdos alcanzados y los próximos pasos claramente definidos para ambas partes, sin la latencia habitual en la producción manual de este tipo de documentación.

1.4. Estado del arte y soluciones existentes

Para contextualizar el desarrollo de nuestro sistema, es fundamental analizar las herramientas actuales que buscan resolver problemas similares en el ámbito de la ingeniería de software asistida por IA.

1.4.1. Spec Kit

Spec Kit es una herramienta diseñada específicamente para facilitar el desarrollo basado en especificaciones. Su enfoque principal radica en la generación de documentos de diseño y arquitectura como paso previo y obligatorio a la implementación de código, estructurando así el ciclo de vida del desarrollo. [20]

1.4.2. AgentOS

AgentOS propone un marco de trabajo (*framework*) integral para la creación y orquestación de agentes autónomos. Proporciona una estructura robusta que permite a los desarrolladores construir sistemas basados en inteligencia artificial capaces de ejecutar tareas complejas y encadenadas de forma independiente. [8] [61]

1.4.3. Asistentes Comerciales

En el mercado actual destacan diversas herramientas comerciales de asistencia a la programación impulsadas por IA, tales como GitHub Copilot, Cursor o Devin. Estas soluciones son altamente eficientes para la generación de código en tiempo real y la resolución de dudas técnicas, operando bajo un paradigma principalmente *Code-Driven* (orientado a la escritura directa de código). [21] [15] [11] [47]

1.4.4. Herramientas de transcripción de reuniones

La obtención de una transcripción como punto de partida es un requisito compartido por los módulos de toma de requerimientos y seguimiento del cliente. Las principales soluciones existentes se dividen en dos categorías según el tipo de reunión.

Para reuniones telemáticas, las plataformas de videoconferencia más extendidas (Microsoft Teams, Google Meet, Zoom) ofrecen transcripción automática integrada con diarización nativa, ya que cada participante habla desde su propio dispositivo. Para reuniones presenciales, las alternativas principales son Whisper [49], el modelo de reconocimiento de voz de OpenAI (ejecución local, sin coste de API, pero sin diarización nativa), y **Azure Speech Service** (servicio cloud con

diarización de hablantes). Se optó por Azure Speech Service al estar incluido en la suscripción de Azure de *Colb.AI* y por ofrecer resultados satisfactorios en las pruebas realizadas.

1.4.5. Herramientas de documentación de reuniones

Existen herramientas comerciales orientadas a la generación automática de resúmenes y documentación post-reunión, como Otter.ai [44], Fireflies.ai [19] o Microsoft Copilot for Teams. Estas soluciones se descartaron por dos razones complementarias: en primer lugar, generan resúmenes genéricos, y en segundo lugar, ya se disponía de Claude Cowork con skills, capaz de generar documentación de seguimiento con la misma eficacia a través de un skill dedicado. Por otro lado, la implementación de un módulo Python propio para este fin no supone una complejidad técnica elevada, lo que refuerza la decisión de desarrollar una solución a medida en lugar de depender de una herramienta externa.

1.4.6. Claude Cowork

Claude Cowork [4] es una herramienta de Anthropic orientada a la automatización de tareas en entorno de escritorio. Su modelo de uso se basa en skills que pueden invocar herramientas nativas de acceso al sistema de archivos, búsqueda en internet y ejecución de scripts, lo que la convierte en un entorno idóneo para implementar flujos de trabajo ligados al ciclo de vida del desarrollo de software.

1.4.7. Asistentes CLI extensibles

Las herramientas de asistencia a la programación de tipo CLI (*Command-Line Interface*) representan una categoría diferenciada de los asistentes integrados en el IDE. Operan directamente desde la terminal con acceso nativo al sistema de archivos y exponen mecanismos de extensibilidad nativos —plugins, hooks de ciclo de vida y el protocolo MCP (*Model Context Protocol* [3])— que permiten definir flujos de trabajo de programación estructurados de forma composable. Las dos principales herramientas de este tipo son ClaudeCode [2] (Anthropic) y OpenCode [42] (open source, compatible con modelos locales y remotos).

1.4.8. superpowers plugin

superpowers [53] es un plugin open source para ClaudeCode que implementa el patrón orquestador + subagentes mediante skills. Proporciona una arquitectura base para flujos de trabajo *Spec-First*, en la que un agente orquestador coordina subagentes especializados a través de skills reutilizables. Sin embargo, carece de un mecanismo de constitución ni de features compartidas entre sesiones o desarrolladores distintos, lo que limita su capacidad para garantizar coherencia arquitectónica a lo largo del ciclo de vida de un proyecto.

1.4.9. Enfoques académicos: planificación previa y agentes múltiples

Paralelamente al desarrollo de herramientas comerciales, la comunidad académica ha explorado enfoques estructurados para mejorar la calidad del código generado por LLMs, aportando evidencia empírica sobre las limitaciones del paradigma *Code-Driven* y sobre la viabilidad de enfoques más estructurados.

Planificación previa a la generación de código. Jiang et al. [30] proponen un enfoque de dos fases —planificar y luego generar— en el que el LLM produce primero un plan estructurado y después genera el código guiado por él. Los resultados muestran mejoras de hasta un 25,4 % frente a la generación directa y un 11,9 % frente a *chain-of-thought*, con evaluadores humanos confirmando ganancias en corrección, legibilidad y robustez del código producido. Este trabajo ofrece validación empírica directa al principio central del SDD: imponer una fase de especificación previa a la implementación mejora sistemáticamente la calidad del resultado.

Sistemas multi-agente para el ciclo de vida del software. MetaGPT [25] codifica procedimientos operativos estándar (SOPs) en secuencias de *prompts* y asigna roles diferenciados a agentes LLM organizados en *pipeline*, reduciendo las alucinaciones en cascada que surgen al encadenar modelos de lenguaje de forma directa. ChatDev [48] orquesta agentes especializados mediante una cadena de comunicación que guía el desarrollo a través de fases (diseño, codificación y pruebas), demostrando que la comunicación estructurada entre agentes es eficaz a lo largo de todo el ciclo de vida del software. Ambos trabajos comparten con *colbPowers* la idea de que la separación de responsabilidades entre agentes especializados y la imposición de un protocolo estructurado producen código de mayor calidad que la generación directa.

Evaluación de agentes en escenarios reales. Jimenez et al. [31] introdujeron SWE-bench, un *benchmark* de 2.294 problemas reales extraídos de repositorios GitHub, y observaron que el modelo más capaz en el momento de la publicación solo resolvía el 1,96 % de los casos de forma autónoma. Este resultado ilustra la brecha entre la generación de código en aislamiento y la resolución de tareas de ingeniería reales, y contextualiza por qué la supervisión humana estructurada —*Human in the Loop*— sigue siendo indispensable en los flujos de desarrollo asistidos por IA.

1.5. Análisis comparativo: ¿Está justificada una solución a medida?

Si bien las alternativas mencionadas presentan avances significativos, su aplicación directa en nuestro entorno de trabajo plantea diversos desafíos operativos y de arquitectura.

En primer lugar, los asistentes puramente comerciales fomentan un ciclo de desarrollo directo e impulsivo. Al no exigir una especificación estructurada previa, agravan el problema del *slop code* (código descuidado o de baja calidad estructural) mencionado en la introducción. Por otro lado, herramientas enfocadas en la especificación como Spec Kit generan una fricción operativa excesiva, produciendo una cantidad desproporcionada de documentos incluso para implementar *features* muy sencillas, lo que ralentiza el desarrollo ágil. Finalmente, marcos de trabajo como AgentOS están diseñados bajo un enfoque *greenfield* (proyectos creados desde cero), resultando sumamente complejos de integrar en repositorios ya existentes.

En la Tabla 1 se presenta un resumen de las características y limitaciones que estas soluciones presentan:

Herramienta	Enfoque Principal	Inconveniente Principal	Integración en Código Existente
Spec Kit	Basado en especificaciones	Exceso de documentación, alta fricción operativa	Moderada
AgentOS	Orquestación de agentes	Diseñado principalmente para proyectos <i>greenfield</i>	Muy compleja (<i>Legacy</i>)
Copilot/Cursor	<i>Code-Driven</i>	Fomenta el <i>slop code</i> , falta de planificación	Alta (pero sin contexto global)
ClaudeCode / OpenCode	CLI extensible mediante plugins y MCP	Requiere plugin/configuración para imponer flujo <i>Spec-First</i>	Alta
superpowers	Orquestador + subagentes via skills	Sin constitución ni <i>features</i> globales entre sesiones	Alta

Tabla 1: Comparativa de soluciones existentes.

De las herramientas analizadas, *superpowers* fue evaluado como candidato directo al problema planteado. Resuelve la orquestación de agentes, pero no garantiza coherencia arquitectónica entre sesiones ni entre desarrolladores distintos, ya que carece de un mecanismo de contexto persistente y compartido. Se decidió adoptarlo como base y extenderlo: el resultado es **colbPowers**, que añade la capa de `constitution.md` (reglas y estándares del proyecto) y `features.md`, y ha requerido tanto la modificación de skills ya existentes como la creación de nuevas skills —*defining-constitution* y *defining-features*— para que estos documentos se integren adecuadamente en el flujo del agente y sean consultados de forma sistemática en cada fase del ciclo SDD.

A la vista de estas limitaciones, la creación de una solución propia resulta razonable. Desarrollar nuestro propio sistema nos proporciona la flexibilidad indispensable para integrarlo de forma nativa con nuestro *pipeline* de trabajo.

1.6. Propuesta

Concluido que las herramientas preexistentes no se adaptan plenamente a nuestros requerimientos de agilidad e integración, nuestra propuesta toma inspiración de sus puntos fuertes para construir un sistema a medida.

Aprovechando la arquitectura de *superpowers*, el proyecto propone el desarrollo de **colbPowers**: un plugin para ClaudeCode y OpenCode que extiende el patrón orquestador/subagentes añadiendo una capa de contexto persistente basada en `constitution.md` y `features.md`, y creando y modificando las skills necesarias para que el agente incorpore estos documentos de forma sistemática en su flujo de trabajo. El ciclo de vida del software girará en torno a tres pilares complementarios:

1. **Toma de requerimientos:** transformar transcripciones de reuniones en planes de implementación estructurados mediante búsqueda semántica web.
2. **Spec-Driven Development:** usar *colbPowers* para garantizar que cada implementación esté precedida de una especificación validada, coordinada por un agente orquestador que invoca subagentes especializados (implementador, revisor de código, revisor de compatibilidad con specs/constitución/features).
3. **Seguimiento del cliente:** automatizar la generación de documentación post-reunión con el formato específico de *Colb.AI*.

Originalmente se contempló construir este sistema desde cero. Durante la fase de investigación previa (TP1) se descubrió que *superpowers* ya ofrecía la arquitectura base necesaria, lo que permitió redirigir el esfuerzo hacia la contribución diferencial: la capa de constitución y features —junto con las skills que hubo que crear y modificar para incorporar estos documentos al flujo del agente— y los tres pilares del ciclo de vida del software.

Cabe destacar que este trabajo gira principalmente en torno al **aprovechamiento** de herramientas ya existentes. El esfuerzo se ha centrado en la integración de dichas herramientas en un flujo de trabajo cohesivo, en la modificación de aquellas que lo requerían para adaptarse al flujo propuesto, y en la creación de la infraestructura necesaria para que operen como un sistema completo.

1.7. Flujo de trabajo completo

La Figura 1 muestra el flujo de trabajo del sistema de principio a fin. El proceso arranca con la **toma de requerimientos**: la primera reunión con el cliente se transcribe de forma diferente según su modalidad —en reuniones telemáticas se aprovechan las transcripciones automáticas de plataformas como Teams o Meet, mientras que en reuniones presenciales se graba el audio y se procesa con Azure Speech Service [36] con diarización de hablantes— y el texto resultante se envía al módulo de procesamiento, que puede ejecutarse mediante la *skill* de Claude Cowork o mediante una implementación propia en Python con LangChain y Tavily [54]. En ambos casos, la salida es un plan de implementación estructurado que el usuario revisa y aprueba antes de continuar.

A partir de ese plan aprobado, el flujo entra en la fase de **desarrollo con colbPowers**: el agente orquestador guía al equipo a través del ciclo SDD completo —definición del contexto del proyecto, especificación de la funcionalidad, implementación y revisión—, con puntos de validación humana en cada etapa que garantizan el principio de *Human in the Loop* [58].

Una vez entregado el desarrollo, el sistema no termina: comienza la fase de **seguimiento del cliente**. Al igual que en la toma de requerimientos, la transcripción de la reunión de seguimiento

se obtiene de forma automática si es telemática, o mediante Azure Speech si es presencial. A partir de ella, Claude Cowork o la alternativa propia generan un documento Word con el resumen, las decisiones tomadas y los próximos pasos. Si en esa reunión se identifican nuevas funcionalidades, el flujo regresa directamente a colbPowers para iniciar un nuevo ciclo de desarrollo — convirtiendo el sistema en un **bucle continuo** que acompaña al proyecto durante toda su vida útil.

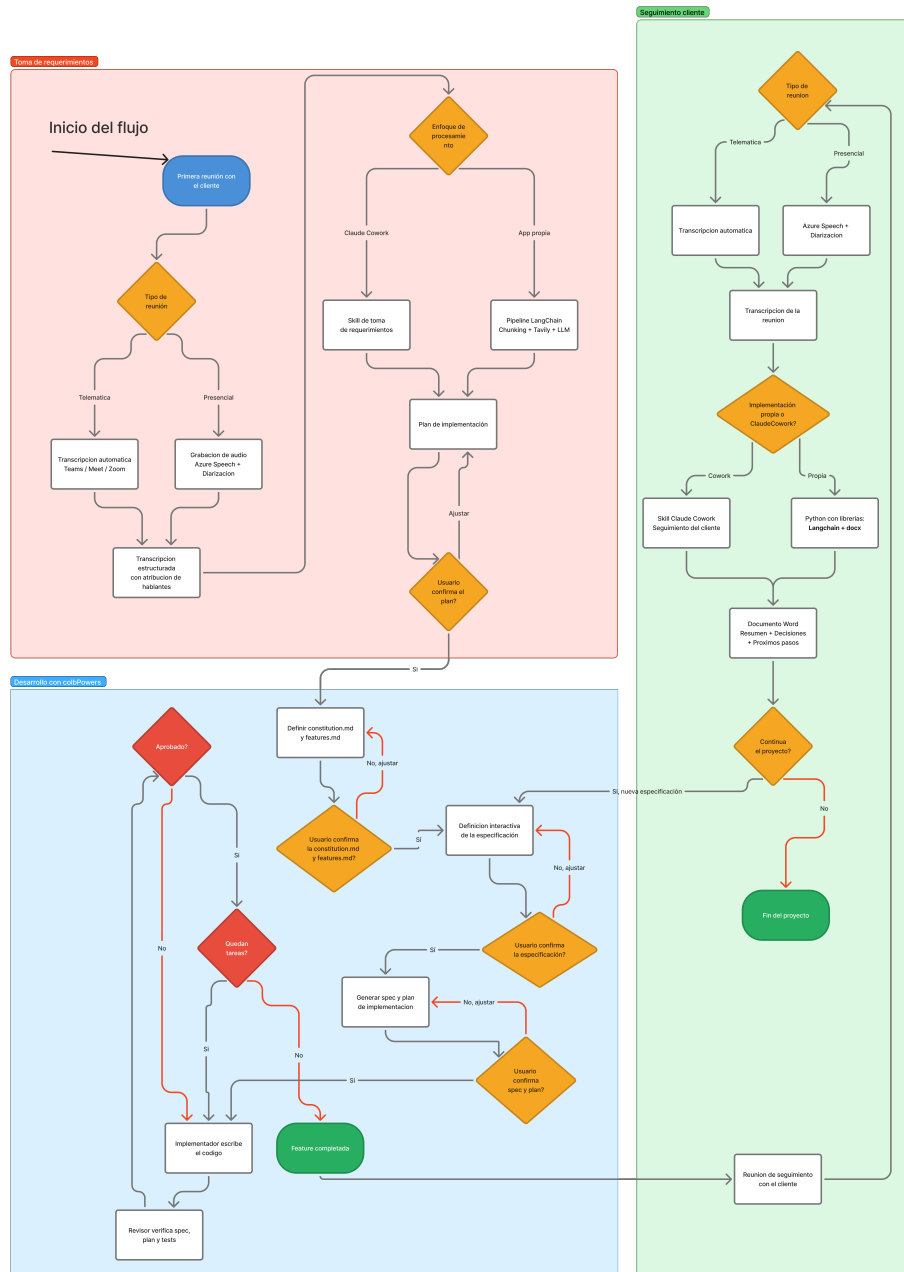


Figura 1: Flujo de trabajo completo del sistema: desde la primera reunión con el cliente hasta el seguimiento iterativo del proyecto.

2. Alcance del Proyecto

En esta sección del trabajo se declaran los objetivos del proyecto, cuáles son los requerimientos para dar este objetivo por cumplido, las métricas que se utilizan para evaluar el sistema, y los posibles obstáculos y riesgos que pueden aparecer en el desarrollo del proyecto (Tal y como pide la metodología de *GEP*). Adicionalmente, se ha añadido un último subapartado destacando la evolución que ha tenido este proyecto respecto la planificación inicial de *GEP*.

2.1. Objetivo General

El objetivo principal de este proyecto es diseñar e implementar un **sistema integral de asistencia al desarrollo de software**, articulado en tres pilares originales:

1. **Plugin colbPowers** para los asistentes CLI ClaudeCode y OpenCode: extiende *superpowers* añadiendo `constitution.md` y `features.md` como contexto persistente compartido entre sesiones y desarrolladores, e implementa el flujo *Spec-First* mediante un agente orquestador y subagentes especializados. La integración con OpenCode requirió el análisis en profundidad de su arquitectura interna —disponible como software libre— y el desarrollo manual de *hooks* nativos en JavaScript. Este trabajo fue necesario para lograr la compatibilidad de OpenCode con herramientas como *code-review-graph* 6.3.1, que implementan su integración mediante hooks nativos de ClaudeCode: dado que los sistemas de hooks de ambos asistentes difieren arquitectónicamente, fue necesario implementar los adaptadores equivalentes en JavaScript para que OpenCode pudiera invocar estos servidores MCP con el mismo comportamiento.
2. **Módulo de toma de requerimientos**: transforma transcripciones de reuniones con el cliente en planes de implementación estructurados. Incluye un componente de transcripción con Azure Speech Service (con diarización de hablantes y gestión mediante gestión de estados de la transcripción y hashes SHA-256). Luego se utiliza una skill de Claude Cowork para diseñar un plan de la toma de requerimientos, y alternativamente se implementa un pipeline propio —para evitar el *vendor lock-in*— de procesamiento basado en LangChain y Tavily para búsqueda semántica y síntesis arquitectónica, con la misma finalidad que skill mencionada de Claude Cowork.
3. **Módulo de seguimiento del cliente**: automatiza la generación de documentación post-reunión en el formato específico de *Colb.AI*, disponible como skill de Claude Cowork y como implementación propia de código abierto con *python-docx* y *langchain* para el agente que procesa la transcripción.

La validación de los resultados del sistema se realiza a través de ciertas métricas de análisis de calidad del código, conocidas y utilizadas en el paradigma del desarrollo de software. Además de, por supuesto, los revisores humanos, que han de analizar en todo momento el código generado y los resultados producidos, haciendo a este un sistema con lo que se conoce *Human In The Loop* [58], cosa que significa que en todo momento hay supervisión e intervención humana.

Para alcanzar el objetivo general, se establecen los siguientes sub-objetivos funcionales y arquitectónicos:

- **Separación efectiva de responsabilidades en el ciclo de desarrollo**: Conseguir que cada fase del proceso (diseño, implementación y revisión) sea ejecutada con criterios y restricciones propios, garantizando que ninguna etapa sea omitida. Esta separación se materializa mediante

skills que invocan subagentes especializados: un agente implementador, un agente revisor de código y un agente revisor de compatibilidad con las especificaciones, la constitución y las features del proyecto.

- **Coherencia arquitectónica mantenida a lo largo de todo el proyecto:** Lograr que los agentes dispongan en todo momento de un contexto actualizado con las reglas del proyecto, su *roadmap* y el estado de las tareas, eliminando las inconsistencias causadas por la pérdida de memoria entre sesiones. Esta coherencia se implementa concretamente mediante `constitution.md` y `features.md` como fuente de verdad compartida entre sesiones y desarrolladores.
- **Trazabilidad completa entre especificación y código entregado:** Garantizar que cada funcionalidad implementada pueda rastrearse desde su especificación original hasta el código final revisado. Esta trazabilidad se garantiza porque todo cambio debe ser coherente con `constitution.md` y `features.md` antes de ser implementado, ofreciendo evidencia verificable de que lo entregado es coherente con lo planificado y cumple los estándares de calidad definidos.

2.2. Requerimientos

2.2.1. Requerimientos Funcionales

- El sistema debe permitir la ingesta y gestión de reglas fundamentales del proyecto (pila tecnológica, estándares de código) y mantener un registro actualizado de las características a desarrollar.
- El sistema debe ser capaz de interpretar funcionalidades de alto nivel y desglosarlas en un plan de tareas granulares y estructuradas.
- El sistema debe incluir un mecanismo de auditoría autónoma que contraste el código fuente generado con las especificaciones documentadas, para ver si existe algún conflicto con las documentación y el código generado.
- El sistema debe incluir un módulo de procesamiento de transcripciones capaz de generar planes de implementación estructurados a partir de reuniones con el cliente, explorando automáticamente los *frameworks* y tecnologías relevantes mediante búsqueda semántica en la web.
- El sistema debe incluir un módulo de generación de documentación de seguimiento post-reunión con el formato específico de *Colb.AI*, a partir de la transcripción de reuniones periódicas con el cliente.

2.2.2. Requerimientos No Funcionales

- **Transparencia y Trazabilidad:** Todo el estado, la memoria y las normativas manejadas por los agentes deben persistir en formatos de texto en un sistema de control de versiones.
- **Integración con asistentes CLI:** El plugin debe ser compatible con ClaudeCode y OpenCode, utilizando sus mecanismos nativos de extensibilidad (MCP, hooks, skills) para actuar directamente sobre los archivos locales del proyecto.
- **Modularidad:** El diseño de los roles (*prompts* y *skills*) debe ser lo suficientemente modular para permitir futuras expansiones.

2.3. Métricas de Evaluación del Código

Para validar empíricamente la calidad del código generado por el sistema, se emplean métricas de análisis estático ampliamente adoptadas en la ingeniería del software [52]. Estas métricas se calculan con herramientas de análisis estático según el lenguaje del proyecto: **radon** [33] para proyectos Python y **DCM — Dart Code Metrics** [16] para proyectos Flutter/Dart. Es importante destacar que las métricas no forman parte del flujo automatizado de los agentes: constituyen una herramienta de evaluación independiente que el desarrollador utiliza para analizar manualmente la calidad del código generado y contrastar los resultados entre distintas ejecuciones del sistema.

Las métricas empleadas se definen formalmente a continuación.

2.3.1. Complejidad Ciclomática

La **Complejidad Ciclomática** [34] de un módulo o función se define a partir de su grafo de flujo de control $G = (V, E)$, donde V es el conjunto de nodos (bloques básicos de instrucciones) y E es el conjunto de aristas (transferencias de control). La complejidad ciclomática $CC(G)$ se calcula como:

$$CC(G) = |E| - |V| + 2p \quad (1)$$

donde $|E|$ es el número de aristas del grafo, $|V|$ es el número de nodos, y p es el número de componentes conexas (típicamente $p = 1$ para una función individual). Esta métrica cuantifica el número de caminos de ejecución linealmente independientes dentro del módulo. Un valor elevado de $CC(G)$ indica una lógica excesivamente ramificada, lo que dificulta las pruebas y el mantenimiento del código. Como referencia orientativa, se considera que valores $CC(G) \leq 10$ corresponden a módulos simples y bien estructurados [34].

2.3.2. Índice de Mantenibilidad

El **Índice de Mantenibilidad** (*Maintainability Index*) [40, 37] es una métrica compuesta definida a partir de tres medidas base. Sea η_1 el número de operadores distintos, η_2 el número de operandos distintos, N_1 el número total de operadores y N_2 el número total de operandos de un módulo. El *volumen de Halstead* [23] se define como:

$$V_H = (N_1 + N_2) \log_2(\eta_1 + \eta_2) \quad (2)$$

Sean además CC la complejidad ciclomática del módulo (Ecuación 1) y LoC el número de líneas de código fuente. El Índice de Mantenibilidad se expresa como:

$$MI = \max\left(0, \frac{171 - 5,2 \ln(V_H) - 0,23 CC - 16,2 \ln(LoC)}{171} \times 100\right) \quad (3)$$

El resultado $MI \in [0, 100]$ proporciona un índice global de facilidad de mantenimiento del módulo analizado. Valores bajos (típicamente $MI < 20$ [37]) indican código difícil de comprender y modificar.

2.4. Obstáculos y Riesgos

Todo proyecto conlleva dificultades que deben analizarse anticipadamente para minimizar su impacto. En la sección 4.2 se especifican los riesgos y obstáculos del proyecto con detalle, incluyendo sus planes alternativos y las tareas afectadas. A modo de resumen, se enumeran las principales dificultades identificadas:

- **Cambios repentinos en los requerimientos:** La empresa o el director del proyecto puede solicitar cambios de alcance, de prioridades o una reorientación del sistema durante el desarrollo, lo que podría invalidar trabajo ya implementado en un *Sprint* avanzado. El impacto es mayor si el cambio se produce en fases tardías como DA3 o DA4, donde la integración entre componentes ya es significativa. La mitigación pasa por la propia metodología ágil adoptada (SCRUM): el *Sprint Backlog* se revisa en cada iteración, lo que permite absorber modificaciones de forma incremental y limitar el retrabajo al sprint en curso, sin comprometer el resto de la planificación.
- **Bloqueo tecnológico (*Vendor Lock-in*) en APIs de LLMs:** El sistema depende de APIs de modelos de lenguaje proporcionadas por terceros (Anthropic, OpenAI u otros). Cambios repentinos en las condiciones de uso, incrementos significativos de precio, degradación del servicio o restricciones de acceso podrían bloquear el avance del proyecto sin margen de reacción. La mitigación pasa por diseñar el módulo de conexión con los LLMs como un componente intercambiable (*wrapper* o adaptador), de modo que sea viable sustituir el proveedor —o migrar hacia modelos *Open Source* ejecutados en local— sin necesidad de reescribir la lógica central del sistema.
- **Limitaciones de control en el ecosistema IDE:** La decisión de construir el sistema sobre agentes nativos de entornos de desarrollo integrados (como GitHub Copilot, Claude Code o Cursor) implica aceptar las restricciones que estas plataformas imponen sobre el flujo de orquestación. A diferencia de una solución construida desde cero con herramientas de más bajo nivel —como una combinación de *LangChain* con llamadas directas a las APIs de los LLMs—, los asistentes integrados en el IDE no permiten un control tan granular sobre el encadenamiento de agentes, la gestión de memoria intermedia o la lógica de decisión entre pasos. Esto puede limitar la capacidad de implementar flujos de trabajo complejos o de depurar el comportamiento del sistema cuando este no actúa como se espera. La mitigación pasa por diseñar los componentes del sistema de forma modular, de modo que, si las restricciones del IDE resultan insalvables, sea viable migrar la lógica de orquestación a una solución de más bajo nivel sin reescribir el sistema completo.
- **Gestión de la Ventana de Contexto:** Los LLMs disponen de una capacidad limitada de contexto activo (*context window*). A medida que el proyecto crezca —más archivos de especificación, más reglas arquitectónicas, más historial de tareas— mantener todo ese conocimiento disponible para el agente sin superar el límite de *tokens* se convierte en un reto técnico no trivial. Si no se gestiona correctamente, el agente puede operar con información incompleta y generar código incoherente con el estado real del proyecto. La mitigación requiere diseñar estrategias de recuperación selectiva de información (*Information Retrieval*), priorizando el contexto más relevante para cada tarea concreta.
- **Fricción en el Bucle de Desarrollo:** Un sistema de auditoría basado en especificaciones corre el riesgo de volverse demasiado restrictivo: si cualquier desviación menor del código respecto a la especificación genera un bloqueo, el desarrollador percibirá el sistema como un obstáculo y no como un asistente. Este desequilibrio podría llevar al abandono de la herramien-

ta o a la búsqueda de formas de saltarse las validaciones, anulando su propósito. La mitigación implica definir niveles de severidad en las auditorías (errores críticos vs. advertencias menores) y calibrar el umbral de tolerancia del sistema en las fases de prueba con usuarios reales.

- **Dependencia de la calidad de las especificaciones:** La eficacia del sistema está directamente condicionada por la calidad de la documentación que el desarrollador proporciona como entrada. Si las especificaciones son ambiguas, incompletas o contradictorias, los agentes generarán código igualmente deficiente, trasladando el problema en lugar de resolverlo. Este riesgo es especialmente relevante en equipos con poca cultura de documentación previa. La mitigación pasa por diseñar el agente diseñador con capacidad de detectar ambigüedades y solicitar aclaraciones antes de proceder, manteniendo siempre al desarrollador humano como árbitro final (*Human in the Loop*).

2.5. Integración de conocimientos

Como demanda la asignatura de *GEP*, en esta sección se realiza un análisis de la integración de conocimientos adquiridos a lo largo del grado y su aplicación en el desarrollo de proyecto.

2.5.1. Competencias técnicas del proyecto

En esta sección se va a asociar las competencias técnicas asignadas al proyecto y los conocimientos aplicados en el desarrollo de este.

- **CCO1.3: Definir, evaluar y seleccionar plataformas de desarrollo y producción hardware y software para el desarrollo de aplicaciones y servicios informáticos de diversa complejidad. [Un poco]** El proyecto evalúa y selecciona herramientas para cada módulo del sistema: ClaudeCode y OpenCode como asistentes CLI frente a extensiones de IDE o frameworks de bajo nivel, Azure Speech Service frente a Whisper para la transcripción con diarización, y Tavily frente a APIs de búsqueda convencionales para la búsqueda semántica orientada a agentes. Cada decisión está justificada por criterios técnicos y de integración con el ecosistema de *Colb.AI*.
- **CCO2.1: Demostrar conocimiento de los fundamentos, paradigmas y técnicas de los sistemas inteligentes, y analizar, diseñar y construir sistemas, servicios y aplicaciones informáticas que utilicen estas técnicas. [En profundidad]** Este proyecto diseña e implementa un sistema multi-agente basado en el patrón ReAct (Reasoning + Acting), el paradigma estándar para la construcción de agentes sobre LLMs. Se diseña una arquitectura con orquestador y subagentes aislados, se implementa la gestión del ciclo de vida de sesión mediante hooks, y se aplica ingeniería de prompts como mecanismo de control del comportamiento.
- **CCO2.2: Capacidad para adquirir, obtener, formalizar y representar el conocimiento humano de forma computable para la resolución de problemas mediante un sistema informático. [Bastante]** El módulo de toma de requisitos convierte conversaciones informales con clientes en especificaciones técnicas computables. El pipeline adquiere el conocimiento (grabación y transcripción), extrae el contenido relevante (chunking y filtrado configurable), y lo formaliza en documentos estructurados (constitution.md, features.md, design.md) que los agentes del sistema pueden leer y sobre los que pueden razonar. La diarización de hablantes permite distinguir qué restricciones provienen del cliente y cuáles del equipo técnico.

2.5.2. Asignaturas

En este apartado se presentan las asignaturas de las cuales se han podido utilizar conocimientos que aplicar al proyecto.

- **Compresión de datos e imágenes (CDI):** Para la implementación propia del módulo de transcripción, se utiliza ffmpeg con técnicas de compresión vistas en CDI, para que el proceso de transcripción tarde menos sin perder demasiada calidad. Principalmente, se aplica un *Downsampling* —reducción en el muestreo del audio— y luego se convierte a MP3 mono, formato que aprovecha la compresión con pérdida basada en MDCT (*Modified Discrete Cosine Transform*) y modelado psicoacústico para reducir significativamente el tamaño del archivo.
- **Búsqueda y análisis de información masiva (CAIM):** Del apartado de *Local Sensitive Hashing* de la asignatura, se extrajo la idea de almacenar los audios transcritos con sus respectivos hashes, aunque en la implementación no se utiliza ningún método de *LSH*, se utiliza un hash criptográfico SHA-256 para identificación exacta del contenido. Los apartados de *Web, Information Retrieval* fueron de utilidad para entender y implementar servicios como *Tavily* para la búsqueda web, así como el temario de *Networks* para la formación del grafo de *code-review-graph*, que facilita a los asistentes de programación la búsqueda y exploración del código.
- **Introducción a la ingeniería del software (IES):** La base proporcionada en esta asignatura sobre requisitos de software, diseño de sistemas y creación de diagramas de flujo, fueron extremadamente útiles tanto para planificar el sistema implementado en este proyecto como para planificar el desarrollo del proyecto en sí.

2.6. Revisión del alcance inicial

La planificación inicial del proyecto (GEP) establecía como objetivo principal la construcción desde cero de un ecosistema de tres agentes especializados (Architect, Developer, Reviewer) integrados en el entorno de desarrollo. Durante la fase de Trabajo Previo (TP1), la investigación del ecosistema de asistentes CLI reveló que *superpowers* ya ofrecía la arquitectura orquestador/subagentes necesaria, haciendo redundante construir ese núcleo desde cero.

Alcance GEP	Alcance final
Construir agentes Architect, Developer y Reviewer desde cero	Desarrollar colbPowers como extensión de <i>superpowers</i>
Integración en el IDE	Integración con asistentes CLI (ClaudeCode, OpenCode)
Un único pilar: SDD	Tres pilares: Toma de requerimientos, SDD, Seguimiento del cliente
Sin mecanismo de contexto persistente	<code>constitution.md</code> y <code>features.md</code> como fuente de verdad compartida

Tabla 2: Comparativa entre el alcance comprometido en el GEP y el alcance final ejecutado.

La justificación del cambio es doble. Por un lado, la adopción de *superpowers* como base eliminó la necesidad de construir la capa de orquestación desde cero, redirigiendo el esfuerzo hacia la contribución diferencial: la capa de `constitution.md` y `features.md`, así como la creación y modificación de skills para incorporar adecuadamente estos documentos al flujo del agente. Por otro lado, la reducción de esfuerzo en el núcleo SDD liberó capacidad para incorporar los módulos de toma de requerimientos y seguimiento del cliente, ampliando el alcance del proyecto para cubrir el ciclo de vida completo del software.

3. Metodología

3.1. Metodología de Trabajo

Para el desarrollo de este Trabajo de Final de Grado, se adopta una **metodología ágil**, específicamente en el marco de *SCRUM*, orientada a la entrega continua de valor y a la adaptación rápida frente a posibles cambios.

La idea es que el flujo de trabajo sea mediante ciclos iterativos breves, para ir validando e iterando sobre el producto, haciendo cambios sobre esta basándonos en las valoraciones del ciclo anterior. [18] [51]

Una pieza fundamental de esta metodología son las reuniones diarias. Estas sesiones breves se llevan a cabo de forma telemática mediante **Microsoft Teams** junto al equipo de *Colb.Ai*. En ellas se expone el progreso realizado el día anterior y se itera sobre las ideas del proceso actual. Esto asegura que el desarrollo se mantenga alineado con los objetivos de la empresa y los estándares de calidad esperados.

Un ejemplo concreto de esta metodología en acción fue el cambio de alcance producido al finalizar la fase TP1. Los hallazgos sobre la extensibilidad de los asistentes CLI y la existencia de *superpowers* fueron presentados al director del proyecto al cierre del sprint, permitiendo reorientar el Sprint Backlog en consecuencia sin comprometer la planificación global ni el presupuesto estimado.

3.2. Herramientas de Seguimiento y Control

Para garantizar el correcto seguimiento del progreso del proyecto, se emplean las siguientes herramientas y dinámicas:

- **Gestión de Tareas (Odoo):** La planificación ágil y el seguimiento de las tareas del proyecto se gestionan íntegramente a través de **Odoo**, la herramienta utilizada internamente por la empresa.
- **Control de Versiones (Git y GitHub):** El código fuente, la documentación y la evolución de los *prompts* están controlados utilizando *Git*. El repositorio remoto está alojado en los servidores de **GitHub**. Esta combinación facilita controlar y evaluar los avances que se hacen sobre el proyecto.
- **Reuniones de Seguimiento Académico:** Además de las interacciones diarias con la empresa, se establecen reuniones periódicas con el director académico del TFG. La frecuencia exacta de estos encuentros se va definiendo y ajustando de mutuo acuerdo a lo largo del desarrollo, con el objetivo de validar el rigor del proyecto, y revisar los entregables de documentación.

3.3. Uso de IA generativa en el desarrollo del proyecto

Como se ha mencionado al comienzo de este documento, la IA generativa está cambiando cómo trabajamos, la mayoría de estudiantes la utilizan —aunque no todos de forma responsable—, para asistir en el desarrollo de sus proyectos. Este proyecto también la emplea, por lo que en aras a la transparencia, esta sección declara los usos que se le han dado a herramientas de inteligencia artificial generativa para el desarrollo de este TFG.

Asistencia en la programación. Los módulos de software del proyecto —el pipeline de toma de requerimientos, el módulo de seguimiento del cliente y la configuración del plugin colbPowers— se desarrollaron con asistencia de ClaudeCode y OpenCode como herramienta de programación. El asistente se utilizó para acelerar la implementación de los distintos módulos, por supuesto bajo una supervisión responsable por parte del desarrollador.

Desarrollo de la memoria. Se ha utilizado IA generativa para la asistencia en la redacción de esta memoria, principalmente para borradores iniciales, además de para reestructurar y reescribir algunas secciones. Todo el contenido generado se ha revisado, y el autor se hace responsable de la veracidad y exactitud de la información presentada. La IA se ha empleado como herramienta de apoyo, no como fuente primaria de contenido.

Exploración de ideas. Durante las fases de diseño se ha empleado IA generativa para explorar alternativas arquitectónicas, evaluar trade-offs entre opciones tecnológicas y evaluar el razonamiento tomar decisiones. Este proceso ha sido orientativo y exploratorio, y las decisiones finales se han tomado basándose en la experiencia del desarrollador y en la revisión de fuentes verídicas.

4. Gestión del Proyecto

4.1. Planificación

Para garantizar el éxito en el desarrollo de colbPowers y los módulos complementarios de toma de requerimientos y seguimiento del cliente, se ha elaborado una planificación temporal detallada. El proyecto comenzó el 12 de febrero de 2026 (contando la documentación de esta asignatura) y tiene prevista su finalización y defensa entre el **23 de junio** hasta el **1 de julio** de 2026. Se estima que la dedicación total para la realización de este Trabajo de Fin de Grado es de aproximadamente 540 horas. La dedicación media es de unas 5 horas diarias durante los días laborables, y en ocasiones, un extra en las etapas en que se deba escribir bastante documentación.

4.1.1. GP - Gestión del proyecto

Este bloque engloba las tareas organizativas y documentales. Estas tareas son concurrentes y se extienden a lo largo de toda la vida del proyecto.

- **GP1 - Contexto y Alcance (20h):** Definición de los objetivos principales, motivación del proyecto y delimitación de las funcionalidades del sistema basado en agentes.
- **GP2 - Planificación temporal (15h):** Elaboración de la estructura de tareas y dependencias para organizar el desarrollo de forma ágil mediante iteraciones o *Sprints*.
- **GP3 - Presupuesto y Sostenibilidad (15h):** Análisis de los costes de recursos materiales, de software y humanos, junto con la evaluación del impacto ambiental y social.
- **GP4 - Documento final (7h):** Integración y entrega final del plan de proyecto con los apartados anteriores (GP1, GP2 y GP3) revisados y corregidos.
- **GP5 - Reuniones de Seguimiento (20h):** Encuentros periódicos con el director, tutor y equipo de la empresa para validar el progreso, resolver dudas y reajustar prioridades.
- **GP6 - Documentación (70h):** Redacción continua de la memoria técnica del TFG, explicación de la arquitectura y justificación de las decisiones de diseño tomadas.
- **GP7 - Defensa del proyecto (20h):** Preparación de la presentación, creación de diapositivas y ensayo de la exposición final ante el tribunal evaluador.

4.1.2. TP - Trabajo Previo

Fase de preparación previa antes de comenzar el desarrollo del proyecto.

- **TP1 - Estudio estado del arte (30h):** Investigar el ecosistema de asistentes CLI extensibles (ClaudeCode, OpenCode) y plugins existentes, especialmente *superpowers*. Este estudio reveló que la arquitectura base ya existía y que su adopción era viable dentro del alcance del TFG. Este descubrimiento, unido a la constatación de que el esfuerzo de implementación del núcleo SDD era menor del previsto, permitió expandir el alcance del proyecto para incluir dos pilares adicionales: el módulo de toma de requerimientos y el módulo de seguimiento del cliente.
- **TP2 - Planificación de estructura y flujo (20h):** En esta etapa se planifica cuál es la estructura básica del proyecto y cómo es el flujo de utilización del producto final.

4.1.3. DA - Desarrollo Ágil (Sprints Iterativos)

El desarrollo de este ecosistema se rige por una metodología ágil adaptada, tomando como referencia principal el marco de trabajo SCRUM. Esta metodología es ideal para un proyecto con la incertidumbre y el dinamismo que supone el entorno, donde nuevas tecnologías y sistemas surgen frecuentemente.

El proyecto se estructura en iteraciones o Sprints de 2 a 3 semanas de duración, centrados en la entrega continua de incrementos funcionales (Product Increments). Esto permite evaluar empíricamente el sistema y aplicar mejoras continuas.

Apunte: debido a la naturaleza dinámica del proyecto, y la flexibilidad que nos proporciona la metodología ágil, la descripción y estimación de horas de las tareas podrían llegar a variar a lo largo del trabajo.

La distribución de horas entre sprints no es uniforme, sino que responde a la complejidad técnica acumulada de cada iteración. Los primeros sprints (DA1 y DA2, 50h cada uno) son más ligeros porque su alcance está acotado: DA1 establece la base funcional de los agentes sobre tecnologías ya estudiadas en la fase TP, y DA2 añade un conjunto inicial de métricas de análisis con scripts relativamente independientes. En cambio, DA3 y DA4 (80h cada uno) requieren significativamente más esfuerzo: DA3 introduce la integración con protocolos MCP y el desarrollo de *AgentSkills*, que exigen una depuración iterativa compleja al operar directamente sobre el entorno local; y DA4 afronta la implementación de técnicas de *Information Retrieval* y la integración de herramientas externas avanzadas, además de incorporar el refinamiento global del sistema acumulado en los sprints anteriores.

- **DA1 - Sprint 1: MVP (50h):** Adoptar *superpowers* como base de colbPowers e instalar el plugin en ClaudeCode y OpenCode, definir la estructura de `constitution.md` y `features.md` como contexto persistente compartido entre sesiones y desarrolladores, y adaptar la skill *using-superpowers* como *using-colbPowers* para incorporar estos documentos al flujo del agente.
 - **DA1.1** Configuración del plugin e integración con ClaudeCode y OpenCode (15h).
 - **DA1.2** Definición de la estructura de `constitution.md` y `features.md` (20h).
 - **DA1.3** Adaptación de la skill *using-superpowers* como *using-colbPowers* e integración del hook `SessionStart` para la inyección automática de las reglas de orquestación al inicio de cada sesión (15h).

- **DA2 - Sprint 2: Skills SDD y métricas de evaluación (50h):** Crear las skills *defining-constitution* y *defining-features*, y adaptar las skills heredadas de *superpowers* para que los subagentes incorporen `constitution.md`, `features.md` y los documentos de especificación en cada fase del ciclo SDD. Integrar las herramientas de análisis estático como mecanismo de evaluación independiente.
 - **DA2.1** Creación de la skill *defining-constitution* y adaptación de la skill de implementación de *superpowers* para que el subagente incorpore `constitution.md` y los documentos de especificación como contexto (15h).
 - **DA2.2** Creación de la skill *defining-features* y adaptación de las skills revisoras de *superpowers* para incorporar `features.md` y los documentos de especificación en la revisión de código y de coherencia con la constitución del proyecto (20h).
 - **DA2.3** Desarrollo de scripts de análisis estático como herramienta de evaluación manual de la calidad del código generado: **radon** para Python y **DCM** (Dart Code Metrics) para Flutter/Dart (15h).
- **DA3 - Sprint 3: Habilidades e Integración MCP (80h):** Ampliar las capacidades de colbPowers mediante MCPs adicionales y refinar el flujo completo basándose en el *feedback* de los sprints anteriores.
 - **DA3.1** Desarrollo de *skills* adicionales para colbPowers correspondientes a las fases del flujo SDD: *brainstorming*, implementación y revisión de código contra especificaciones (30h).
 - **DA3.2** Integración de *code-review-graph* como servidor MCP y desarrollo de los adaptadores JavaScript necesarios para que OpenCode pudiera invocarlo con el mismo comportamiento que ClaudeCode (25h).
 - **DA3.3** Análisis de errores de los sprints anteriores, corrección de prompts y mejoras de estabilidad del plugin (25h).
- **DA4 - Sprint 4: Módulos complementarios y refinamiento final (80h):** Implementar el módulo de toma de requerimientos y el módulo de seguimiento del cliente, y refinar el sistema completo.
 - **DA4.1** Módulo de transcripción: integración con Azure Speech Service con diarización, implementación de la gestión idempotente mediante máquina de estados en Azure Blob Storage y hashes SHA-256, y compresión determinista del audio con `ffmpeg` (35h).
 - **DA4.2** Módulo de procesamiento de requerimientos: implementación del pipeline con LangChain, investigación y pruebas de integración de Tavily como motor de búsqueda semántica, y validación del flujo completo de generación del plan arquitectónico (25h).
 - **DA4.3** Módulo de seguimiento del cliente y refinamiento global del sistema (20h).

4.1.4. PV - Pruebas y Validación

Fase dedicada a la validación empírica y técnica del sistema, evaluando tanto el código como la interacción del usuario.

- **PV1 - Pruebas unitarias de las Agent Skills (20h):** Verificación técnica de los scripts de `radon` y las configuraciones del plugin `colbPowers` para asegurar que interactúan correctamente con el sistema de archivos y los asistentes CLI.

- **PV2 - Pruebas del bucle completo (Human in the Loop) (40h):** Simulaciones de desarrollo de funcionalidades (*features*) reales para auditar la colaboración entre el usuario y los agentes. Incluye la ejecución de los experimentos controlados (baseline vs. colbPowers), la recogida de métricas de calidad del código mediante análisis estático, y el análisis estadístico de los resultados (20h).

4.1.5. Recursos

A continuación se listan los recursos Materiales, Humanos, y de software de los que depende el proyecto.

Recursos Materiales Para el desarrollo óptimo de este proyecto, se requiere la siguiente infraestructura física:

- **Estación de trabajo:** Ordenador personal (PC o portátil) con capacidad suficiente para ejecutar entornos de desarrollo virtualizados (mínimo recomendado 16GB de RAM y procesador de gama media-alta).
- **Periféricos:** Monitor externo para facilitar la lectura de código y documentación en paralelo, teclado ergonómico, ratón y equipo de audio/micrófono para las reuniones telemáticas.
- **Espacio de trabajo:** Un entorno físico acondicionado ergonómicamente, con acceso ininterrumpido a la red eléctrica y una conexión a internet de banda ancha estable.

Recursos de Software El ecosistema tecnológico del proyecto se apoya en diversas herramientas categorizadas según su propósito:

- **Entorno de Desarrollo:** Sistema operativo Windows con Windows Subsystem for Linux (WSL) habilitado, proporcionando un entorno Unix nativo indispensable para la ejecución de *scripts* en Bash y Python. El editor principal es Visual Studio Code (VSCode).
- **Servicios de Inteligencia Artificial:** Suscripciones activas o acceso a APIs de proveedores de modelos de lenguaje (LLMs) y agentes de código, como *Claude Code*, *OpenCode* o las APIs de Anthropic/OpenAI. Adicionalmente, **Azure Speech Service** para la transcripción de reuniones presenciales con diarización, y **Tavily** como motor de búsqueda semántica para el módulo de toma de requerimientos.
- **Análisis de código:** **radon**, herramienta Python de análisis estático para el cálculo de métricas de calidad del código generado.
- **Orquestación:** **LangChain** para la construcción del pipeline de procesamiento de requerimientos.
- **Control de Versiones y Repositorios:** *Git* como sistema de control de versiones local y *GitHub* como plataforma de alojamiento remoto e integración.
- **Gestión y Planificación:** *Odoo* para la gestión de tareas (seguimiento del *Product/Sprint Backlog*), *GanttProject* para la planificación temporal y *Draw.io* para el diseño de diagramas arquitectónicos.
- **Comunicación:** *Microsoft Teams* como canal principal para las reuniones de seguimiento y validación con el equipo supervisor.

Recursos Humanos Al tratarse de un Trabajo de Fin de Grado, el equipo humano se estructura de la siguiente manera:

- **Desarrollador / Investigador Principal (Autor):** Encargado de la totalidad de la carga de trabajo estimada de 540 horas. Al tratarse de un proyecto individual, el autor asume distintos roles a lo largo del desarrollo en función de la naturaleza de cada tarea:

- **Jefe del Proyecto (JP):** Responsable de la planificación, coordinación y seguimiento del progreso general. Incluye la gestión de reuniones y la toma de decisiones estratégicas.
- **Investigador / Documentador (ID):** Encargado del estudio del estado del arte, la redacción de documentación técnica y la investigación sobre metodologías de uso de IA.
- **Programador Junior (PJ):** Responsable de la implementación del código: desarrollo de los agentes, integración de herramientas MCP y sprints de desarrollo ágil.
- **Tester:** Encargado de la verificación y validación del sistema mediante pruebas unitarias de *Agent Skills* y pruebas del bucle completo (HITL).
- **Director del Proyecto:** Proporciona la visión técnica, valida las decisiones arquitectónicas del sistema y orienta sobre la viabilidad de la implementación.
- **Ponente y Tutor GEP:** Encargados del seguimiento metodológico de la gestión del proyecto, la revisión de la documentación y el asesoramiento académico continuo.

4.1.6. Tabla resumen de tareas

A continuación, se presenta una tabla resumen con la información clave de cada tarea del proyecto, detallando su código, descripción, tiempo estimado en horas, dependencias temporales y los recursos principales asignados para su ejecución. En la columna *Recursos específicos*, el término *Autor* se emplea de forma unificada dado que cada tarea implica una combinación variable de los roles definidos (JP, ID, PJ y Tester) en proporciones que dependen de la naturaleza y la fase concreta de la tarea. El desglose detallado de horas por rol se recoge en la subsección siguiente:

Cód.	Nombre de la tarea	Temps.	Depend.	Recursos específicos
GP1	Contexto y Alcance	20h	-	Autor, PC
GP2	Planificación temporal	15h	GP1	Autor, PC, GanttProject
GP3	Presupuesto y Sostenibilidad	15h	GP2	Autor, PC
GP4	Documento final	7h	GP3	Autor, PC
GP5	Reuniones de Seguimiento	20h	-	Autor, Director, Teams
GP6	Documentación	70h	-	Autor, PC, Overleaf/LaTeX
GP7	Defensa del proyecto	20h	GP6, DA, PV	Autor, Tribunal, PC
TP1	Estudio estado del arte	30h	-	Autor, PC, Internet
TP2	Planificación estructura y flujo	20h	TP1	Autor, PC, Draw.io
DA1	Sprint 1: MVP colbPowers	50h	TP2	Autor, PC, ClaudeCode, OpenCode
DA2	Sprint 2: Skills SDD y métricas	50h	DA1	Autor, PC, Python, radon
DA3	Sprint 3: Habilidades e Int. MCP	80h	DA1	Autor, PC, Git, Bash, Python
DA4	Sprint 4: Módulos compl. y refinamiento	80h	DA3	Autor, PC, Azure Speech, LangChain, Tavily
PV1	Pruebas unitarias Agent Skills	20h	DA3	Autor, PC, WSL (Bash/Python)
PV2	Pruebas bucle completo (HITL)	40h	DA1	Autor, Tutor, Compañero, PC

Tabla 3: Resumen de las tareas del proyecto con sus dependencias y recursos asignados.

Desglose de horas por rol Dado que el autor asume distintos roles a lo largo del proyecto (tal y como se detalla en la sección de Gestión Económica), la siguiente tabla resume cómo se distribuyen las aproximadamente 540 horas totales entre cada uno de ellos:

Rol	Horas estimadas	Tareas principales
Jefe del Proyecto	~77h	GP (25 %), TP2 (100 %), DA3 (19 %)
Investigador / Documen- tador	~190h	GP (75 %), TP1 (100 %), DA2 (30 %), DA4 (25 %)
Programador Junior	~210h	DA1 (100 %), DA2 (70 %), DA3 (81 %), DA4 (75 %)
Tester	~60h	PV1 (100 %), PV2 (100 %)
Total	~537h	

Tabla 4: Distribución estimada de horas del autor por rol a lo largo del proyecto.

4.1.7. Diagrama de Gantt

En la siguiente página se muestra el diagrama de gantt resultante de la planificación de las tareas anteriores.

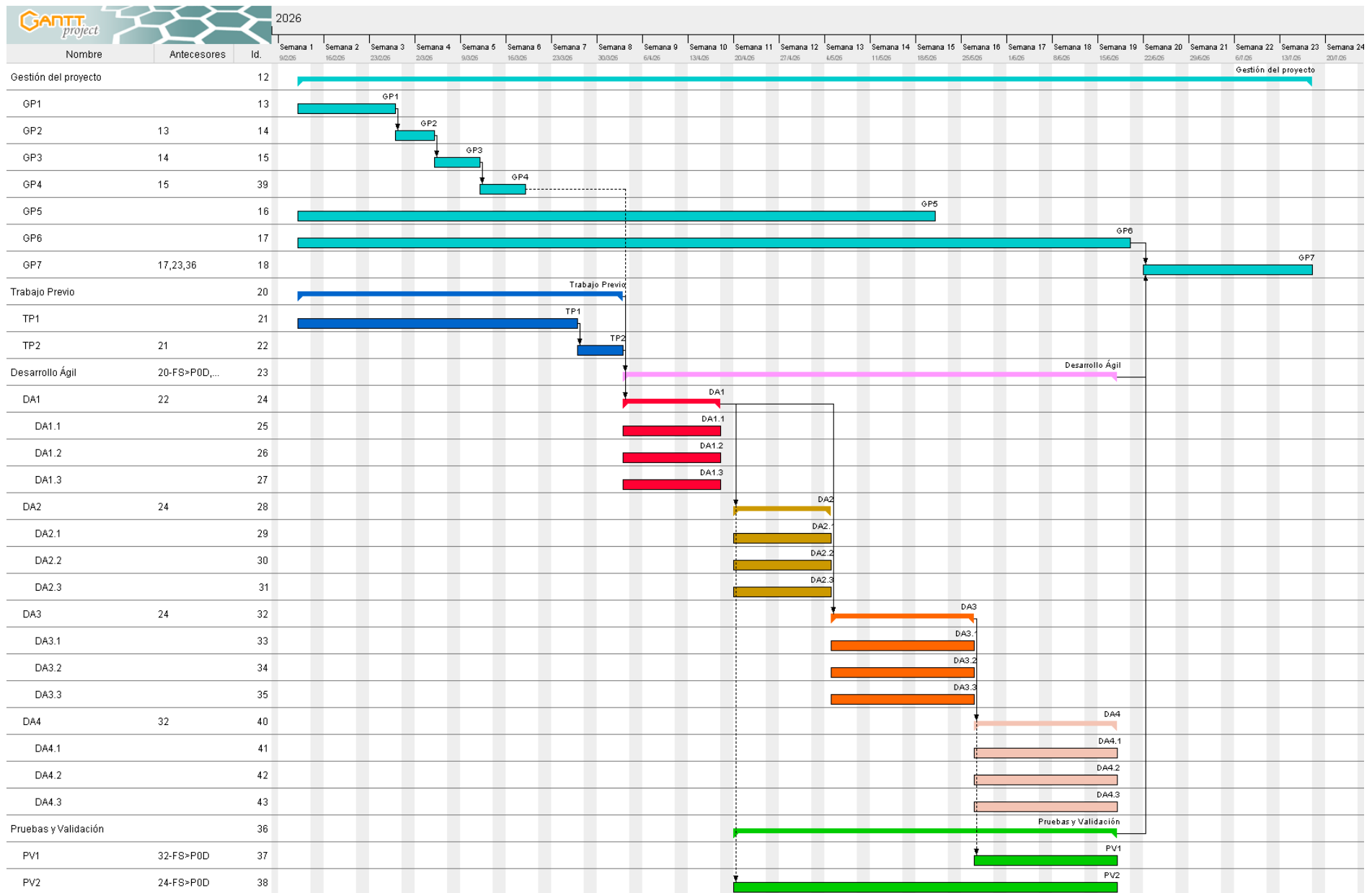


Figura 2: Diagrama de Gantt, creado con el software *GanttProject*.

4.1.8. Revisión de la planificación inicial

La planificación establecida en el GEP estructuraba el desarrollo en cuatro sprints centrados en la construcción de agentes desde cero. El cambio de alcance producido al finalizar TP1 implicó una redistribución del contenido de los sprints, manteniendo el número de fases y el total de horas estimado (540h) sin desviación significativa.

Sprint	Planificación GEP	Planificación final
DA1	Agentes Architect, Developer y Reviewer desde cero	Adopción de <i>superpowers</i> como base + capa <code>constitution.md/features.md</code> + inyección de <i>using-colbPowers</i> mediante hook <code>SessionStart</code>
DA2	Métricas de evaluación integradas en el agente Reviewer	Nuevas skills SDD (<i>defining-constitution, defining-features</i>) + adaptación de skills de <i>superpowers</i> + scripts de análisis estático
DA3	AgentSkills e integración MCP	Ampliación de MCPs y refinamiento del flujo colbPowers — sin cambio significativo
DA4	Information Retrieval y MCPs avanzados	Módulo de toma de requerimientos (Azure Speech + LangChain/Tavily) y módulo de seguimiento del cliente

Tabla 5: Comparativa entre la planificación del GEP y la planificación final ejecutada.

La justificación de estos cambios es consistente con lo descrito en la sección 2.6: la adopción de *superpowers* redujo el esfuerzo necesario en DA1 y DA2, liberando capacidad que se reinvertió en DA4 para los módulos complementarios. Las horas totales por fase se mantienen dentro del margen estimado; en la tabla 3 se recogen las horas estimadas, y en el apartado de gestión económica (4.3.6) se cuantifica el impacto económico real.

4.2. Gestión de riesgos: Planes alternativos y obstáculos

Durante la planificación y ejecución de este proyecto, se identificaron diversos obstáculos potenciales inherentes al desarrollo con IA. A continuación, se detallan estos riesgos junto con sus respectivos planes alternativos, tal y como dictan las buenas prácticas de gestión ágil de proyectos. En el apartado 4.3.4 se cuantifica el coste económico estimado de cada uno de estos imprevistos.

Estos riesgos se definieron al desarrollar la planificación inicial del proyecto, y se indicará cuales de ellos han ocurrido y las consecuencias de esto.

4.2.1. Riesgo 1: Cambios repentinos en los requerimientos por parte de la empresa

- **Probabilidad:** Baja / Media (40 %)
- **Tareas afectadas:** Principalmente las fases de Desarrollo Ágil (**DA1–DA4**), ya que un cambio de requerimientos puede invalidar trabajo ya implementado en cualquier *Sprint*. También se ve afectada la tarea **GP5** (Reuniones de Seguimiento), que deberá absorber sesiones extraordinarias de re-planificación, y potencialmente **PV** si los cambios implican redefinir los criterios de validación.
- **Tareas alternativas (Plan B):** Re-evaluación del *Sprint Backlog* actual, congelación del código afectado y re-planificación de los objetivos en una reunión extraordinaria para el siguiente *Sprint*.
- **Afectación a la duración total:** Retraso estimado de 15 horas sobre la planificación original, absorbible en la fase de pruebas (PV).
- **Recursos adicionales necesarios:** Horas extra del autor y disponibilidad del Director del proyecto para una sesión de rediseño.
- **Resultado:** Este riesgo se materializó durante la fase TP1. El mecanismo de revisión de sprint absorbió el cambio de alcance de forma controlada, reorientando el Sprint Backlog sin comprometer la planificación global ni el presupuesto estimado.

4.2.2. Riesgo 2: Bloqueo tecnológico (Vendor Lock-in) por cambio de precios o políticas en las APIs de los LLMs

- **Probabilidad:** Baja / Media (30 %) [41]
- **Tareas afectadas:** El impacto recae principalmente sobre **DA3** (Habilidades e Integración MCP), donde se construyen los módulos de conexión con las APIs, y sobre **DA4** (Recuperación de Información y Refinamiento), que depende de integraciones avanzadas con proveedores externos. En menor medida, también puede afectar a **DA1** si el agente base necesita ser reconectado a un proveedor distinto.
- **Tareas alternativas (Plan B):** Migración del motor cognitivo del sistema hacia modelos *Open Source* (ej. LLaMA o Mistral) ejecutados en local o a través de proveedores alternativos, modificando el módulo de conexión (*wrapper*).
- **Afectación a la duración total:** Aumento de 20 horas en la fase de Integración MCP (DA3).
- **Recursos adicionales necesarios:** Tiempo extra de desarrollo y posible necesidad de hardware local con mayor capacidad de procesamiento (GPU) temporal.

4.2.3. Riesgo 3: Documentación interna insuficiente para la generación autónoma de código estructurado

- **Probabilidad:** Media (50 %)
- **Tareas afectadas:** Afecta directamente a **DA1** (implementación del agente Developer, cuya capacidad de escritura se reduciría), **DA2** (las métricas de evaluación perderían parte de su utilidad si el agente no genera código autónomamente) y **PV2** (el bucle completo *Human in the Loop* requeriría una mayor intervención manual del autor, alterando la naturaleza de la validación).
- **Tareas alternativas (Plan B):** Reducción del alcance del sistema (*scope downgrade*). Se limitarán las capacidades de escritura del agente *Developer*. El desarrollador humano (usuario) asumirá la implementación directa del código base, mientras que los agentes (*Architect* y *Reviewer*) se reorientarán exclusivamente a tareas de lectura: documentar, auditar la estructura y comprobar la calidad del código manual contra la especificación original.
- **Afectación a la duración total:** Impacto temporal neutro. Las horas ahorradas en el desarrollo de *skills* complejas de escritura para los agentes se reinvertirán en la codificación manual por parte del autor durante el bucle de pruebas (PV).
- **Recursos adicionales necesarios:** Mayor carga de programación manual y esfuerzo cognitivo por parte del autor, al delegar menos carga de implementación en la inteligencia artificial.

4.2.4. Riesgo 4: Limitaciones de control sobre el flujo de orquestación en el ecosistema IDE

- **Probabilidad:** Media / Alta (65 %)
- **Tareas afectadas:** Afecta directamente a **DA1** (implementación del MVP sobre el entorno IDE) y a **DA3** (Habilidades e Integración MCP), donde se construyen los módulos de orquestación entre agentes. Si las restricciones del IDE resultan insalvables, estas tareas deberán ser rediseñadas.
- **Tareas alternativas (Plan B):** Rediseño de los *prompts* de cada agente para que sean más breves, concisos y con un alcance muy acotado, reduciendo así la tendencia del modelo a desviarse del comportamiento esperado. En este escenario, la coordinación entre agentes recae explícitamente sobre el desarrollador humano, que actúa como orquestador manual del flujo. El inconveniente de esta solución es que exige que el usuario conozca con precisión el proceso de trabajo del sistema para dirigir correctamente a cada agente, lo que aumenta la curva de aprendizaje de la herramienta.
- **Afectación a la duración total:** Aumento estimado de 20 horas en **DA3** para rediseñar y ajustar los *prompts* de los agentes afectados y documentar el flujo de coordinación manual para el usuario.
- **Recursos adicionales necesarios:** Tiempo de diseño de *prompts* por parte del autor y elaboración de documentación del flujo de uso dirigida al desarrollador.
- **Resultado:** Este riesgo se manifestó parcialmente. Los mecanismos nativos de extensibilidad de ClaudeCode y OpenCode (MCP, hooks, skills) resultaron suficientes para implementar el flujo de trabajo requerido.

4.2.5. Riesgo 5: Desbordamiento de la ventana de contexto en proyectos grandes

- **Probabilidad:** Baja / Media (35 %)
- **Tareas afectadas:** Impacta principalmente en **DA4** (Recuperación de Información y Refinamiento), cuyo objetivo específico es abordar este problema mediante técnicas de *Information Retrieval*. Si se manifiesta antes, también puede bloquear el avance de **DA1** y **DA3**, al impedir que los agentes mantengan suficiente contexto para operar correctamente.
- **Tareas alternativas (Plan B):** Aplicar dos estrategias complementarias de reducción de contexto. La primera consiste en implementar resúmenes automáticos o manuales de la conversación, descartando el historial detallado y reteniendo únicamente las decisiones clave ya tomadas. La segunda estrategia consiste en aislar al máximo el alcance de cada agente: en lugar de proporcionar contexto global del proyecto, cada agente opera exclusivamente sobre la *feature* o módulo que tiene asignado, desconociendo intencionadamente el resto de la implementación. Esto mantiene la ventana de contexto acotada y manejable incluso en proyectos grandes.
- **Afectación a la duración total:** Impacto absorbible dentro de las 80 horas ya asignadas a **DA4**, aunque podría suponer un retraso de hasta 15 horas adicionales si la solución de aislamiento requiere rediseñar la estructura de memoria del sistema.
- **Recursos adicionales necesarios:** Ninguno adicional si se resuelve dentro de **DA4**; posibles horas extra del autor si la estrategia de aislamiento debe adelantarse a *Sprints* anteriores.

4.2.6. Riesgo 6: Exceso de fricción en el bucle de desarrollo por auditorías demasiado estrictas

- **Probabilidad:** Baja (30 %)
- **Tareas afectadas:** Afecta a **DA3** (implementación del agente *Reviewer* y sus políticas de auditoría) y a **PV2** (pruebas del bucle completo *Human in the Loop*), donde se evalúa empíricamente si el sistema resulta usable o frustrante para el desarrollador.
- **Tareas alternativas (Plan B):** Rediseño del agente *Reviewer* para implementar niveles de severidad en las auditorías (errores críticos bloqueantes vs. advertencias no bloqueantes), permitiendo al usuario continuar el desarrollo ante desviaciones menores y reservando los bloqueos para incumplimientos graves de la especificación.
- **Afectación a la duración total:** Aumento de aproximadamente 10 horas en **DA3** para recalibrar y probar los umbrales de tolerancia del agente *Reviewer*.
- **Recursos adicionales necesarios:** Sesiones de prueba adicionales con el autor actuando como usuario final para validar que el nuevo umbral resulta equilibrado.

4.3. Gestión económica

En este apartado se presentan los distintos costes que aparecen a lo largo del desarrollo del proyecto.

4.3.1. Coste del personal

Se divide este apartado en distintos roles. Cada uno tiene un coste por hora determinado, y a partir del diagrama de Gantt y las tablas con las horas asignadas a cada tarea y sus recursos personales adjuntados, se calcula el total de cada uno de estos roles. Adicionalmente, se muestran tanto los salarios netos como los brutos - añadida la contribución de 30% a la Seguridad Social (SS) -, y cuando se hace la suma total de los costes, se utiliza obviamente el salario bruto.

Rol	Salario Neto	Salario Bruto (+ SS)
Jefe del Proyecto (Programador senior)	20,15€/h	26,20€/h
Investigador y documentador	16,04€/h	20,85€/h
Programador junior	11,48€/h	14,92€/h
Testers	14,72€/h	19,17€/h

Tabla 6: Costes de personal por hora según el rol asumido. Basada en información extraída de [26].

4.3.2. Coste por actividad

Se repasan las actividades presentadas en el entregable anterior, los recursos humanos invertidos en cada una, y se calcula su coste.

Gestión del proyecto Esta se ha estimado que durará unas 167 horas en total. Dando por hecho que se desarrolla en un 25% por el *Jefe del Proyecto* y el restante 75% por el *Investigador/Documentador*, el coste humano de esta tarea es de:

$$CosteGP = (0.25 * 26.2€/h + 0.75 * 20.85€/h) * 167h = 3705.32€ \quad (4)$$

Trabajo Previo Esta actividad se estima que es de unas 50 horas. Requiere un estudio del estado del arte, y planificación del flujo del funcionamiento del proyecto. Las 30 horas de investigación del estudio del arte podrían estar desarrolladas por el *Investigador/Desarrollador*, mientras que la planificación del proyecto se llevaría a cabo por el *Jefe del Proyecto*, por lo que 20 horas de su parte.

$$CosteTP = 20h * 26.2€/h + 30h * 20.85€/h = 1149.5€ \quad (5)$$

Desarrollo Ágil Como se hizo en la entrega anterior, el desarrollo ágil (DA) se dividirá en DA1, DA2, DA3 y DA4. Basándonos en la naturaleza de cada iteración, repartiremos la carga de trabajo entre los distintos roles definidos para estimar su coste:

- **DA1 (Sprint 1 - MVP colbPowers):** Consta de 50 horas en total. Al tratarse de la configuración del plugin e implementación del agente orquestador base, asignaremos la totalidad de estas horas al *Programador junior*.

$$CosteDA1 = 50h * 14.92€/h = 746€$$

- **DA2 (Sprint 2 - Subagentes y métricas):** Consta de 50 horas. Las 15 horas iniciales de investigación y configuración de radon son llevadas a cabo por el *Investigador y documentador*, mientras que las 35 horas restantes de implementación de subagentes y scripts recaen sobre el *Programador junior*.

$$CosteDA2 = 15h * 20.85€/h + 35h * 14.92€/h = 836€$$

- **DA3 (Sprint 3 - Habilidades e Integración MCP):** De las 80 horas estimadas, 65 horas se dedicarán al desarrollo de *Agent Skills* y protocolos MCP por parte del *Programador junior*. Las 15 horas restantes, dedicadas al análisis de errores complejos y mejoras de estabilidad, requerirán la experiencia y supervisión del *Jefe del Proyecto*.

$$CosteDA3 = 65h * 14.92€/h + 15h * 26.2€/h = 1362.8€$$

- **DA4 (Sprint 4 - Módulos complementarios y refinamiento):** Cuenta con 80 horas totales. Asignaremos 20 horas al *Investigador y documentador* para el diseño de los módulos de transcripción y seguimiento, y las 60 horas restantes al *Programador junior* para la implementación de los módulos y el refinamiento final del sistema.

$$CosteDA4 = 20h * 20.85€/h + 60h * 14.92€/h = 1312.2€$$

Sumando los costes de los cuatro *Sprints*, obtenemos el coste total de los recursos humanos para la fase de Desarrollo Ágil:

$$CosteDA = 746€ + 836€ + 1362.8€ + 1312.2€ = 4257€ \quad (6)$$

Pruebas y Validación Se estimó que en total esta tarea tomaría unas 60 horas a lo largo del proyecto. De estas 60 horas, todas ellas serían asignadas al Tester.

$$CostePV = 60h * 19.17€/h = 1150.2€ \quad (7)$$

4.3.3. Costes genéricos

En este apartado se referenciarán los costes que no se pueden asignar a una actividad específica, sino que están presentes a lo largo de todo el desarrollo. Estos incluyen los gastos de infraestructura, licencias de software y la amortización del equipo físico utilizado.

Costes estructurales Los costes estructurales contemplan los gastos derivados del uso de un espacio de trabajo físico, así como los suministros básicos necesarios para llevar a cabo el desarrollo. Asumiendo que el proyecto tiene una duración estimada de 4,5 meses (desde mediados de febrero hasta principios de julio), se han estimado los siguientes gastos mensuales:

- **Alquiler de la oficina:** Se asume el alquiler de un puesto en un espacio de *coworking* o la parte proporcional de una oficina compartida, estimado en 150€/mes. [7]
- **Electricidad:** Consumo derivado de los equipos informáticos, iluminación y climatización del espacio de trabajo, estimado en 50€/mes. [39]

- **Suministros adicionales:** Gastos de conectividad a Internet de alta velocidad y agua, calculados en 40€/mes.

En total, el coste estructural mensual asciende a 240€. Por lo tanto, el cálculo para los 4,5 meses de duración del proyecto es el siguiente:

$$CosteEstructural = 4.5 \text{ meses} * 240€/mes = 1080€ \quad (8)$$

Costes de *Software* El ecosistema tecnológico del proyecto se apoya en diversas herramientas. Afortunadamente, muchas de ellas cuentan con licencias gratuitas o versiones académicas sin coste (como Git, GitHub, VSCode, Odoo y Microsoft Teams). Sin embargo, el núcleo de este TFG requiere el uso de inteligencia artificial, lo que implica costes de suscripción y consumo de APIs (Claude Code, GitHub Copilot y servicios de Microsoft Azure / OpenAI).

Se ha estimado un presupuesto de consumo mensual para estas herramientas:

- **GitHub Copilot:** 10€/mes. [22]
- **Claude:** 20€/mes. [6]
- **Consumo de APIs LLM (Azure/OpenAI/Anthropic):** 30€/mes.

$$CosteSoftware = (10€/mes + 20€/mes + 30€/mes) * 4.5 \text{ meses} = 270€ \quad (9)$$

Costes de *Hardware* Para calcular el coste del hardware, no se imputa el precio total de compra de los equipos, ya que estos no agotan su vida útil exclusivamente con este proyecto. Se calcula su amortización.

La fórmula de amortización se define dividiendo el precio del equipo entre sus horas de vida útil, y multiplicando el resultado por las horas dedicadas al TFG (estimadas en 540 horas en total). Se asume una vida útil de 4 años para los equipos informáticos (aproximadamente 220 días laborables al año, a 8 horas diarias = 7040 horas).

Dispositivo	Precio Compra	Vida Útil (h)	Amortización (540h)
Estación de trabajo (PC)	1500€	7040h	115,06€
Monitor externo	200€	7040h	15,34€
Periféricos (Teclado, ratón, audio)	100€	7040h	7,67€
Total	1800€	—	138,07€

Tabla 7: Cálculo de la amortización de los recursos de *Hardware*.

$$CosteHardware = 115.06€ + 15.34€ + 7.67€ = 138.07€ \quad (10)$$

Sumando todos los componentes detallados en esta subsección, el total de costes genéricos asciende a:

$$TotalGenericos = 1080€ + 270€ + 138.07€ = 1488.07€ \quad (11)$$

4.3.4. Contingencias e imprevistos

En el desarrollo de cualquier proyecto de ingeniería informática, es fundamental disponer de un margen económico para hacer frente a posibles desviaciones temporales o problemas técnicos. Este margen se divide en dos partidas: las contingencias (para riesgos desconocidos) y los imprevistos (para riesgos ya identificados).

Contingencias La partida de contingencias actúa como un fondo de reserva general. Dado que el proyecto se basa en la integración de Inteligencia Artificial (LLMs) y herramientas en constante evolución, existe un grado de incertidumbre inherente. Aplicando las buenas prácticas de gestión de proyectos informáticos, se establece un margen de contingencia del 15% sobre la suma total de los costes de personal y los costes genéricos estimados hasta el momento.

$$CosteContingencias = (CostePersonal + CosteGenericos) * 0.15 \quad (12)$$

Imprevistos Por otro lado, la partida de imprevistos cuantifica económicamente los riesgos específicos que ya fueron identificados y analizados en el Entregable 2. El coste que se añade al presupuesto se calcula multiplicando el coste del plan de acción alternativo (Plan B) por la probabilidad estimada de que el riesgo ocurra.

- **Riesgo 1: Cambios repentinos en los requerimientos.** La probabilidad de ocurrencia se estimó como Baja/Media (40%). El plan de mitigación contempla un retraso de 15 horas, implicando re-planificación y desarrollo. Asignaremos 5 horas al *Jefe de Proyecto* (26,20€/h) y 10 horas al *Programador junior* (14,92€/h).

$$CosteR1 = ((5h * 26.20€/h) + (10h * 14.92€/h)) * 0.40 = 280.2€ * 0.40 = 112.08€$$

Impacto: Medio

- **Riesgo 2: Bloqueo tecnológico (Vendor Lock-in) en APIs LLM.** La probabilidad es Baja/Media (30%). El plan alternativo es la migración a modelos *Open Source*, con un impacto de 20 horas de desarrollo (*Programador junior*) y el alquiler puntual de hardware en la nube con GPU (estimado en 100€).

$$CosteR2 = ((20h * 14.92€/h) + 100€) * 0.30 = 398.4€ * 0.30 = 119.52€$$

Impacto: Medio

- **Riesgo 3: Documentación interna insuficiente.** Probabilidad Media (50%). El plan alternativo consiste en realizar un *scope downgrade* (reducir capacidades de escritura de los agentes). Al tener un impacto temporal neutro (las horas no invertidas en IA se invierten en programación manual), no genera un coste económico extra en el presupuesto.

$$CosteR3 = 0€ * 0.50 = 0€$$

Impacto: Bajo

- **Riesgo 4: Limitaciones de control en el ecosistema IDE.** Probabilidad Media/Alta (65%). El plan alternativo implica rediseñar los *prompts* y documentar el flujo manual, con un impacto estimado de 20 horas en DA3: 10 horas de *Programador junior* (14,92€/h) y 10 horas de *Investigador/Documentador* (20,85€/h).

$$CosteR4 = ((10h * 14.92€/h) + (10h * 20.85€/h)) * 0.65 = 357.7€ * 0.65 = 232.51€$$

Impacto: Medio

- **Riesgo 5: Desbordamiento de la ventana de contexto.** Probabilidad Baja/Media (35%). El plan alternativo puede requerir hasta 15 horas adicionales de *Programador junior* (14,92€/h) para rediseñar la estrategia de aislamiento de contexto.

$$CosteR5 = (15h * 14.92€/h) * 0.35 = 223.8€ * 0.35 = 78.33€$$

Impacto: Bajo

- **Riesgo 6: Exceso de fricción en el bucle de desarrollo.** Probabilidad Baja (30%). El plan alternativo requiere aproximadamente 10 horas adicionales de *Programador junior* (14,92€/h) en DA3 para recalibrar los umbrales del agente *Reviewer*.

$$CosteR6 = (10h * 14.92€/h) * 0.30 = 149.2€ * 0.30 = 44.76€$$

Impacto: Bajo

Sumando los valores obtenidos, calculamos el fondo total reservado para imprevistos:

$$TotalImprevistos = 112.08€ + 119.52€ + 0€ + 232.51€ + 78.33€ + 44.76€ = 587.20€ \quad (13)$$

4.3.5. Coste total

Una vez estimados todos los gastos asociados al desarrollo del proyecto, tanto directos (personal) como indirectos (genéricos), y habiendo reservado los fondos necesarios para contingencias e imprevistos, podemos definir el presupuesto final.

A continuación, se presenta una tabla resumen con la consolidación de todas las partidas económicas calculadas previamente:

Concepto	Coste Estimado
Coste de Personal (GP, TP, DA, PV)	10.262,02€
Costes Genéricos (Estructurales, Software, Hardware)	1.488,07€
Subtotal (Base del proyecto)	11.750,09€
Contingencias (15 % sobre el Subtotal)	1.762,52€
Imprevistos (Riesgos evaluados)	587,20€
PRESUPUESTO TOTAL	14.099,81€

Tabla 8: Presupuesto total estimado del proyecto.

4.3.6. Revisión de los costes iniciales

El presupuesto comprometido en el GEP ascendía a **14.099,81€**. El cambio de alcance producido durante TP1 tuvo un impacto económico prácticamente neutro: las horas previstas para construir agentes desde cero se redistribuyeron entre la investigación del ecosistema de extensibilidad (TP1 ampliado) y la implementación de los módulos complementarios (DA4 reorientado), sin incrementar el total de horas estimado. La redistribución de horas entre fases no alteró los costes de personal porque los roles implicados en las tareas nuevas son los mismos que en las tareas sustituidas. Los costes genéricos tampoco variaron de forma significativa: las herramientas nuevas incorporadas (Azure Speech Service, Tavily) están cubiertas por las suscripciones existentes de *Colb.AI* o tienen un coste marginal absorbible dentro de la partida de APIs LLM ya presupuestada. El presupuesto final se mantiene en **14.099,81€**.

4.3.7. Control de gestión

Para asegurar la viabilidad económica del Trabajo de Fin de Grado y evitar que el proyecto supere el presupuesto establecido, es estrictamente necesario aplicar un control de gestión continuo. Al utilizar una metodología ágil (SCRUM), este control se realiza al finalizar cada iteración o *Sprint*, así como en las reuniones de seguimiento estipuladas.

El control se basa en comparar los valores estimados en este documento con los gastos y horas reales consumidas. Para cada indicador se define su fórmula de cálculo y los criterios de actuación concretos en función de la magnitud de la desviación detectada:

- **Desviación de horas por tarea:** Mide si se ha invertido más o menos tiempo del planificado en una tarea concreta.

$$\text{Desviación Horas} = \text{Horas Estimadas} - \text{Horas Reales} \quad (14)$$

Criterio de actuación:

- Desviación entre 0 y -10 %: dentro del margen tolerable. Se registra como nota en la revisión del *Sprint* pero no requiere acción inmediata.
 - Desviación entre -10 % y -20 %: se analiza la causa en la reunión de seguimiento con el director y se ajusta la estimación de las tareas pendientes del mismo bloque.
 - Desviación superior al -20 %: se convoca una sesión extraordinaria de re-planificación, se re-prioriza el *Sprint Backlog* y se eliminan o posponen las subtarefas de menor valor del incremento actual.
- **Desviación de coste de personal:** Calcula el impacto económico de la desviación de horas, multiplicándolo por el salario bruto del rol implicado.

$$\text{Desviación Coste Personal} = (\text{Horas Estimadas} - \text{Horas Reales}) * \text{Coste Rol} \quad (15)$$

Criterio de actuación:

- Desviación acumulada inferior a -300€: absorbible con el fondo de contingencias (15 %), sin acción adicional.
 - Desviación acumulada entre -300€ y -800€: se activa el fondo de contingencias y se restringe el alcance de las tareas de menor prioridad del bloque en curso.
 - Desviación acumulada superior a -800€: se aplica un *scope downgrade* formal, eliminando funcionalidades secundarias para reconducir el presupuesto hacia los objetivos críticos del proyecto.
- **Desviación de costes genéricos:** Controla si herramientas de software, hardware o infraestructura han sufrido variaciones de precio (por ejemplo, subida de tarifas en las APIs de los LLMs).

$$\text{Desviación Genéricos} = \text{Coste Genérico Estimado} - \text{Coste Genérico Real} \quad (16)$$

Criterio de actuación:

- Desviación puntual inferior a -30€/mes: se registra y monitoriza. Si persiste más de dos meses consecutivos, se revisan alternativas.

- Desviación sostenida o superior a $-30\text{€}/\text{mes}$: se evalúa la sustitución de la herramienta afectada por una alternativa gratuita u *open source* equivalente (por ejemplo, migrar a un proveedor de LLM alternativo en caso de subida de tarifas de APIs).
- **Consumo del fondo de imprevistos:** Evalúa cuánto remanente queda del fondo de emergencias calculado. Si el resultado es negativo, significa que se ha agotado el presupuesto de seguridad.

$$\text{Estado Imprevistos} = \text{Total Imprevistos Estimado} - \text{Coste Imprevistos Consumido} \quad (17)$$

Criterio de actuación:

- Remanente superior al 50 % (más de 183€ disponibles): situación bajo control, sin cambios en el plan.
- Remanente entre el 0 % y el 50 %: se congela el uso del fondo para nuevos imprevistos menores y se activa el fondo de contingencias para cubrir desviaciones adicionales.
- Remanente negativo (fondo agotado): se aplica de inmediato un *scope downgrade* de las funcionalidades con mayor riesgo económico y se notifica al director del proyecto para valorar conjuntamente medidas extraordinarias.

4.4. Informe de sostenibilidad

4.4.1. Autoevaluación de la competencia

Tras realizar la encuesta de autoevaluación, identifiqué un perfil técnico sólido pero con carencias formativas en la gestión de la sostenibilidad. Uno de mis puntos fuertes es el compromiso con la eficiencia técnica y el orden del código (*Green Computing*), lo que garantiza soluciones robustas y mantenibles a largo plazo. No obstante, el cuestionario ha evidenciado que durante la carrera me han faltado asignaturas que profundicen en métricas de sostenibilidad medioambiental, económica y social.

Esta falta de base académica ha sido mi principal punto débil al afrontar la estimación de costes y riesgos de este proyecto. Sin embargo, mediante un proceso de aprendizaje autónomo durante la planificación, he comprendido la importancia crítica de estos conceptos para gestionar adecuadamente un proyecto de cierto calibre como puede ser un TFG. Esta reflexión me ha permitido adoptar una visión más empática de la ingeniería, utilizando la metodología SDD no solo para buscar la eficiencia, sino para prevenir el *burnout* y fomentar la sostenibilidad social del equipo.

Concretamente, en el bloque de métricas de impacto ambiental y social, comprendo de forma intuitiva conceptos básicos como la huella ambiental o el ciclo de vida de un producto, pero los marcos de medición más formalizados y los indicadores cuantitativos de impacto social eran prácticamente desconocidos para mí. Esta brecha fue evidente al tener que razonar sobre el impacto social de una herramienta de IA como la que propone este TFG.

El bloque donde partía con menor preparación era el de gestión económica: conceptos como las amortizaciones, el control de desviaciones presupuestarias o la estructura detallada de un presupuesto los he aprendido prácticamente desde cero durante la elaboración del proyecto, lo que ha supuesto el reto más importante a nivel de sostenibilidad de todo el GEP. En contraste, en lo relativo a los principios éticos y deontológicos del impacto de la IA, ya contaba con una conciencia bastante desarrollada antes de comenzar, al tratarse de un tema que me preocupa activamente como futuro ingeniero.

Nota: Los contenidos de la matriz de sostenibilidad (PPP, Vida Útil y Riesgos) para cada una de las tres dimensiones se desarrollan a continuación en un formato narrativo en lugar de tabular, para facilitar la exposición de los conceptos.

4.4.2. Dimensión económica

PPP: El coste total del proyecto se estima en 14.099,81€. Tras reflexionar sobre este presupuesto, considero que es realista y competitivo para un desarrollo de 540 horas que integra tecnologías punteras de IA. La optimización del gasto se logra mediante el uso de herramientas de código abierto y un modelo de pago por uso para las APIs, centrando la inversión en el valor del esfuerzo humano y la investigación.

Vida Útil: Actualmente, la industria aborda el desarrollo asistido por IA de forma impulsiva (Code-Driven), lo que reduce costes iniciales pero dispara los de mantenimiento debido a la deuda técnica [14]. Esta solución basada en SDD mejora económicamente este escenario al reducir dicha deuda desde la fase de planificación. Consideramos que la adopción del sistema desarrollado en este TFG puede llegar a decrementar las horas de mantenimiento y desarrollo a largo plazo, al garantizar que toda implementación esté respaldada por una especificación validada y coherente

con la arquitectura del proyecto. Idealmente, el ahorro en tiempo de comprensión y corrección de errores debería amortizar la inversión inicial del proyecto.

Riesgos: El principal riesgo económico durante la vida útil del sistema es la escalada de precios de las APIs de los LLMs. Una subida significativa de tarifas por parte de los proveedores (Anthropic, OpenAI) podría encarecer el coste de mantenimiento mensual estimado, comprometiendo la viabilidad económica de la herramienta a largo plazo. La mitigación consiste en que el diseño del sistema sea compatible con distintos proveedores, permitiendo migrar a distintos proveedores o alternativas más económicas.

Mantenimiento: El coste operativo tras la entrega es mínimo, estimado en aproximadamente 120€/mes. Este importe cubre los costes de suscripciones a las APIs de IA (GitHub, Anthropic, ...) y unas 4 horas mensuales de soporte técnico de un *programador junior* para actualizaciones menores y ajustes varios.

$$CosteMantenimiento = 60€/mes + 4h/mes * 14.92€/h = 119.68€/mes \quad (18)$$

4.4.3. Dimensión social

PPP: A nivel personal, este proyecto es un catalizador en mi formación. Me permite consolidar conocimientos técnicos en la orquestación de LLMs y, simultáneamente, adquirir habilidades críticas en gestión económica y metodologías ágiles (SCRUM), competencias que han demostrado ser tan vitales como la propia programación.

Vida Útil: Existe una necesidad real en el mercado debido al creciente estrés y frustración profesional que genera trabajar con bases de código incoherentes [57]. La metodología SDD mejora la calidad de vida de los programadores al obligar a establecer especificaciones claras, reduciendo el caos y fomentando un entorno laboral más estructurado y colaborativo.

Riesgos: El principal riesgo social es la resistencia por parte de los desarrolladores a adoptar una metodología más estructurada como SDD, especialmente en equipos con poca cultura de documentación previa. Si la herramienta se percibe como una carga burocrática en lugar de un asistente, podría ser abandonada o eludida, anulando su impacto positivo. La mitigación pasa por un diseño centrado en la usabilidad, que guíe al programador en su uso.

4.4.4. Dimensión ambiental

PPP: El impacto ambiental proviene del consumo eléctrico de los equipos durante las 540 horas de trabajo y el uso de servidores en la nube para las APIs. Para minimizar este impacto, se ha optado por reutilizar hardware personal y planificar un desarrollo local mediante WSL que limite las peticiones innecesarias a la red durante las fases iniciales de pruebas.

Vida Útil: La evaluación experimental confirma que colbPowers multiplica el consumo de tokens por un factor de entre $2,2\times$ y $4,5\times$ respecto a los asistentes sin flujo SDD, debido al overhead estructural del proceso (generación de especificaciones, delegación a subagentes y revisiones sistemáticas). El balance ambiental neto a largo plazo depende de si la mejora demostrada en la calidad del código se traduce en una reducción equivalente en sesiones de depuración, reescritura y mantenimiento futuro, compensando así el sobre coste por sesión de desarrollo. Esta hipótesis es plausible pero no ha podido verificarse en el alcance de este trabajo.

Riesgos: El principal riesgo ambiental, confirmado empíricamente, es que el overhead de tokens del flujo SDD no quede compensado por la reducción en ciclos de corrección posteriores. En ese escenario, el sistema tendría un impacto ambiental neto superior al de los asistentes sin flujo estructurado. Para evaluar este riesgo con rigor sería necesario un estudio longitudinal que mida el número total de sesiones de desarrollo hasta la entrega del producto, incluyendo las de mantenimiento.

4.5. Leyes y regulaciones

Reglamento europeo de inteligencia artificial (EU AI Act): El sistema no pertenece a ninguna de las categorías de alto riesgo definidas en el Anexo III del Reglamento (UE) 2024/1689 [45]: no toma decisiones autónomas con impacto significativo sobre personas físicas, y el diseño Human-in-the-Loop garantiza supervisión humana en todas las acciones con consecuencias reales.

Propiedad intelectual del código generado: La titularidad del software generado con asistencia de IA depende del grado de intervención humana en el proceso creativo. Un uso puramente autónomo de la IA —sin especificación previa ni revisión humana— no genera derechos de autor reconocibles. Sin embargo, cuando la IA actúa como herramienta asistida con intervención humana documentada, los derechos corresponden al autor humano, siempre que esa intervención quede acreditada. [35]

El flujo Spec-Driven Development implementado en este proyecto sitúa explícitamente la generación de código en la categoría de IA asistida: el desarrollador define la especificación, valida el plan de implementación antes de que comience la escritura de código, y revisa el resultado entregado por el subagente. Toda esta intervención queda registrada en el repositorio git del proyecto, proporcionando la documentación requerida para acreditar la autoría humana. Por otro lado, colbPowers es una obra derivada de *superpowers*, distribuido bajo licencia MIT, con contribuciones originales documentadas en el repositorio git del proyecto que constituyen la aportación diferencial del TFG.

Licencias de terceros y Convenio de Cooperación Educativa: Todas las herramientas de terceros utilizadas en el proyecto —OpenCode, code-review-graph, LangChain— disponen de licencias open source que permiten el uso comercial. Las APIs de Anthropic, Azure y Google Gemini no utilizan los datos de entrada para entrenar modelos bajo sus planes de pago. Por otro lado, Copilot sí que entrena con los datos adquiridos durante las interacciones con sus modelos -Des de el 24 de abril de 2026- de forma predeterminada, pero es una opción que se puede desactivar fácilmente [50]. El TFG se desarrolla en el marco de un Convenio de Cooperación Educativa entre la UPC y *Colb.AI*, que regula la propiedad intelectual del trabajo y la confidencialidad de la información de la empresa. Los proyectos reales utilizados en la validación experimental están sujetos a NDA; en la memoria únicamente se reportan métricas técnicas que no exponen información confidencial del cliente.

5. Módulos de Transcripción y Documentación

5.1. Toma de Requerimientos

La toma de requerimientos es la primera fase del ciclo de vida de un proyecto de software. En este contexto, el objetivo es transformar las necesidades expresadas por el cliente en documentos de especificación técnica utilizables por el equipo de desarrollo. El proceso se articula en dos módulos independientes y secuenciales: en primer lugar, el módulo de transcripción convierte las reuniones en texto estructurado; y en segundo lugar, el módulo de procesamiento analiza ese texto y genera el plan de implementación. Cada módulo dispone de dos implementaciones complementarias, con el propósito de no depender de un único proveedor, cubriendo así el riesgo de vendor lock-in mencionado anteriormente 4.2.2.

5.1.1. Módulo de transcripción

Para que el módulo de procesamiento pueda operar, es necesario disponer previamente de una transcripción de la reunión con el cliente. Dependiendo del tipo de reunión, la forma de obtenerla difiere.

Reuniones telemáticas Las principales plataformas de videoconferencia (Microsoft Teams, Google Meet, Zoom, entre otras) ofrecen transcripción automática integrada. Dado que cada participante habla desde su propio dispositivo y micrófono, la atribución de fragmentos de texto a cada hablante (diarización) es gestionada de forma nativa por la plataforma, sin necesidad de ningún procesamiento adicional. Para las reuniones telemáticas, se utiliza directamente la transcripción exportada por la plataforma correspondiente como entrada al módulo de procesamiento.

Reuniones presenciales Para cubrir este caso se decide implementar nuestra propia solución, se graba el audio de la reunión y se procesa a posteriori mediante Azure Speech Service, el servicio de transcripción de audio proporcionado por el proveedor de servicios cloud de la empresa *Colb.AI*.

La característica técnica diferencial en este contexto es la diarización (*speaker diarization*): se puede configurar el servicio para identificar a cada participante de forma independiente y etiquetar cada fragmento de texto con el hablante que lo pronunció. Esta atribución es especialmente relevante para la toma de requerimientos, ya que permite al módulo de procesamiento distinguir qué peticiones o restricciones provienen del cliente y cuáles son aclaraciones o comentarios del equipo técnico, mejorando la calidad del plan de implementación resultante.

Preprocesamiento del audio. Antes de enviar el audio al servicio de transcripción, se aplica un paso de compresión cuyo objetivo es doble: reducir el tamaño del fichero para minimizar costes de almacenamiento y transferencia, y garantizar que el proceso sea determinista. La función `downsample_audio_deterministic` utiliza `ffmpeg` para convertir el audio original a formato MP3 mono a 8 kHz, eliminando metadatos y aplicando flags de codificación bit-exacta (`-bitexact, -map_metadata -1, -id3v2_version 0`). Gracias a estas garantías, el mismo archivo de entrada siempre produce exactamente los mismos bytes de salida y, por tanto, el mismo hash SHA-256. Este hash actúa como identificador único del contenido del audio a lo largo de todo el sistema.

Gestión idempotente mediante máquina de estados. El componente `SpeechToText` implementa un mecanismo idempotente apoyado en Azure Blob Storage. Al recibir un fichero de audio, se calcula su hash determinista y se consulta el fichero `state.txt` asociado en el contenedor de almacenamiento. El ciclo de vida de una transcripción sigue la siguiente secuencia de estados:

1. **just_created:** el audio aún no ha sido procesado. Se sube el fichero comprimido al blob, se genera una URL de acceso temporal con firma de acceso compartido (SAS), y se lanza el trabajo de transcripción por lotes.
2. **transcribing:** la transcripción está en curso. Si se recibe una nueva solicitud para el mismo hash, el sistema devuelve inmediatamente un mensaje informativo sin lanzar un trabajo duplicado.
3. **transcribed:** la transcripción ha concluido con éxito. El texto resultante se almacena en el blob y se devuelve una URL SAS de acceso al fichero.

Este diseño garantiza que procesar dos veces el mismo audio no genere costes adicionales ni resultados duplicados.

Transcripción por lotes (*Batch API*). La transcripción se realiza a través de la API REST de Azure Speech Service en su versión 3.2, concretamente el endpoint de *batch transcription*. El componente `BatchTranscriptor` realiza tres pasos:

1. **Envío del trabajo:** se realiza una petición POST al endpoint de transcripciones con la URL SAS del audio comprimido. La configuración incluye el idioma (español por defecto), el modo de puntuación automática y, cuando la diarización está habilitada, los parámetros `diarizationEnabled: true` junto con el rango esperado de hablantes (mínimo 2, máximo 5).
2. **Sondeo del estado:** se consulta periódicamente el estado del trabajo (cada 5 segundos) hasta que la API reporta `Succeeded` o `Failed`.
3. **Descarga y formateo del resultado:** una vez completado el trabajo, se descarga el fichero de resultados y se procesan las frases reconocidas (`recognizedPhrases`). Cuando la diarización está activa, cada frase se ordena cronológicamente, y se formatea como `[Speaker N]: texto`, produciendo una transcripción estructurada por hablante.

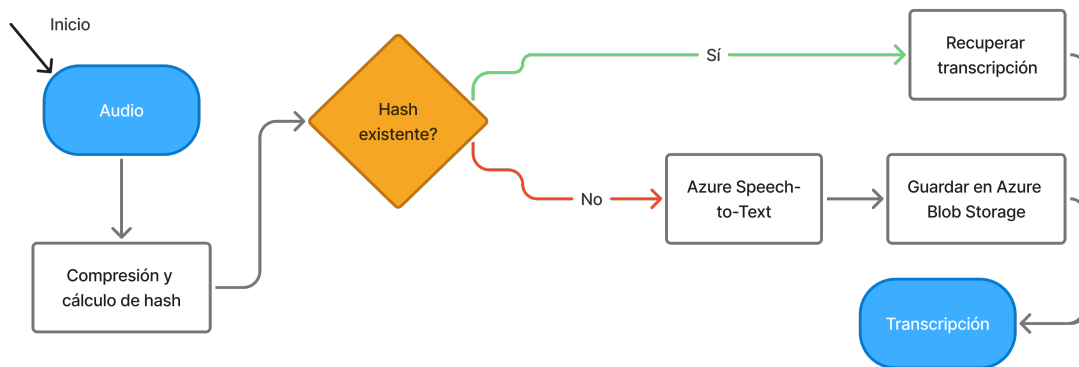


Figura 3: Diagrama de flujo de el proceso de transcripción.

5.1.2. Módulo de procesamiento de requerimientos

Una vez disponible la transcripción (producida por cualquiera de las vías descritas en la sección anterior), el módulo de procesamiento la analiza y genera un plan de implementación estructurado. Este módulo también dispone de dos implementaciones: un enfoque basado en la plataforma Claude Cowork y una implementación propia en Python.

Enfoque mediante Claude Cowork Claude Cowork es una herramienta de Anthropic orientada a la automatización de tareas en entorno de escritorio.

El modelo de uso se basa en el concepto de Skill: un workflow de automatización configurable que combina un prompt especializado con conexiones a herramientas, servicios externos y scripts personalizados.

La skill recibe una entrada (por ejemplo, una transcripción de texto), las herramientas que puede invocar durante su ejecución (búsqueda en internet, lectura y escritura de archivos, llamadas a APIs externas, ejecución de scripts) y el formato de la salida que produce. A diferencia de un prompt ordinario enviado en una conversación, una skill encapsula el flujo de trabajo completo, con scripts y ejecutables incluidos dentro de la propia skill, y puede reutilizarse de forma consistente en cualquier proyecto sin necesidad de redefinirlo.

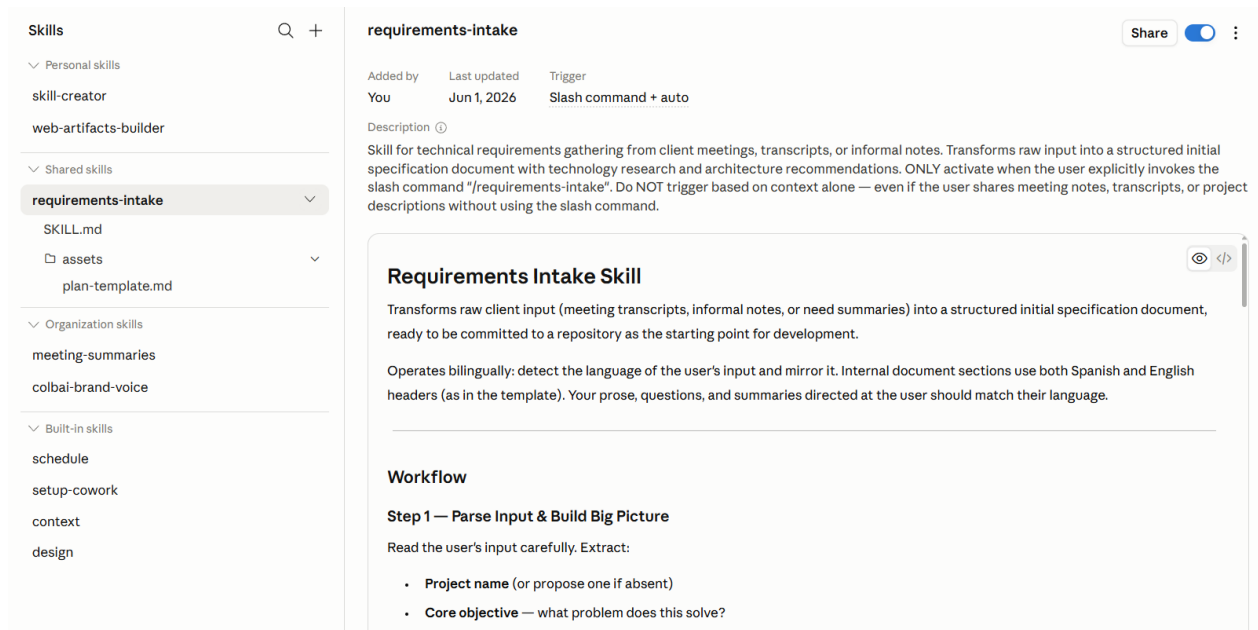


Figura 4: Skill de toma de requerimientos en Claude Cowork.

Se ha configurado una skill específica para la toma de requerimientos que implementa el siguiente flujo:

1. El usuario proporciona a la skill la transcripción de la reunión con el cliente, notas informales o un resumen de las necesidades expresadas.
2. La skill procesa el contexto y genera una visión global del proyecto (*big picture*), identificando los componentes principales del sistema, las posibles tecnologías a emplear y las áreas de incertidumbre que requieren aclaración.
3. Mediante las herramientas de búsqueda en internet integradas en Claude Cowork, el agente investiga las tecnologías propuestas y contrasta alternativas.
4. El agente presenta al usuario las opciones tecnológicas encontradas para que seleccione las preferidas. El resultado es un documento de especificación inicial estructurado, listo para incorporarse al repositorio como punto de partida del desarrollo.

Este enfoque es el preferido cuando se dispone de acceso a la plataforma Claude Cowork, dado que la integración con las herramientas de búsqueda y la calidad del razonamiento son superiores a los de la implementación propia.

Implementación propia Con el objetivo de disponer de una alternativa de coste reducido e independiente de suscripciones comerciales, se ha desarrollado una aplicación Python que replica el proceso anterior. La aplicación está estructurada en módulos desacoplados y orquestada mediante LangChain [10].

Arquitectura. El núcleo de la aplicación es el módulo `transcription_preprocessing`, que implementa la clase `TranscriptionProcessor` y coordina toda la lógica de negocio. El módulo recibe como entrada el texto de la transcripción y produce como salida un plan de implementación arquitectónico en formato Markdown.

El diseño emplea dos modelos de lenguaje con roles diferenciados: un modelo económico (Azure OpenAI) para las operaciones de alta frecuencia que no requieren razonamiento profundo, y un modelo de mayor capacidad (Google Gemini) para las tareas de síntesis y razonamiento arquitectónico. Esta estrategia de dos niveles optimiza el coste sin sacrificar la calidad del resultado final.

Pipeline de procesamiento. El flujo de procesamiento sigue cuatro fases secuenciales:

Fase 1: Chunking y filtrado: la transcripción se divide en fragmentos de hasta 25.000 caracteres con un solapamiento de 1.200 caracteres y separadores jerárquicos (párrafos, líneas, frases), lo que permite procesar reuniones de larga duración sin superar los límites de contexto de los modelos. El modelo económico procesa cada fragmento de forma independiente y extrae únicamente el contenido relevante según un criterio configurable (por defecto, «requerimientos funcionales y reglas de negocio»), descartando saludos, divagaciones y contenido irrelevante. Los fragmentos sin contenido útil se descartan con la marca `SIN_CONTENIDO_RELEVANTE`.

Fase 2: Generación del plan de investigación: el modelo de mayor capacidad analiza el texto filtrado e identifica las decisiones técnicas pendientes (elección de base de datos, framework, infraestructura). A partir de este análisis genera un plan de búsquedas estructurado con un máximo de tres *queries*, para no saturar el límite de créditos del servicio de búsqueda.

Fase 3: Investigación web mediante Tavily: Tavily opera como un motor de búsqueda semántica diseñado para agentes de IA, utilizando un *pipeline* de filtrado híbrido. Primero, realiza una recuperación léxica para identificar URLs candidatas; posteriormente, mediante un motor de *scraping* dinámico capaz de renderizar JavaScript, extrae el contenido relevante eliminando el ruido del HTML (anuncios, menús) y transformándolo en Markdown. El sistema procesa el texto mediante *embeddings* vectoriales, calculando la similitud de coseno entre la consulta y los fragmentos extraídos para realizar un *re-ranking* semántico en tiempo real. Finalmente, selecciona los segmentos con mayor densidad de información factual. Las *queries* se ejecutan con búsqueda profunda (`search_depth=advanced`), limitando los resultados a fuentes del último año y formateándolos como contexto estructurado.

Fase 4: Generación del plan arquitectónico: el modelo de mayor capacidad combina la transcripción original con los resultados de la investigación y produce un documento de arquitectura en formato Markdown. El documento incluye el resumen de objetivos, la propuesta de stack tecnológico justificada con la investigación, los componentes clave del sistema y la hoja de ruta de implementación.

Bucle de refinamiento con *Human in the Loop*. Una característica diferencial de la aplicación es la capacidad de refinamiento iterativo en cada fase. El usuario puede modificar el plan de investigación o el plan arquitectónico mediante instrucciones en lenguaje natural. El sistema mantiene un historial de conversación mediante la abstracción `MessagesPlaceholder` de LangChain [10], lo que permite que el modelo tome en cuenta el contexto de intercambios anteriores al procesar cada solicitud de refinamiento. Este mecanismo implementa el principio *Human in the Loop* definido en los objetivos del proyecto.

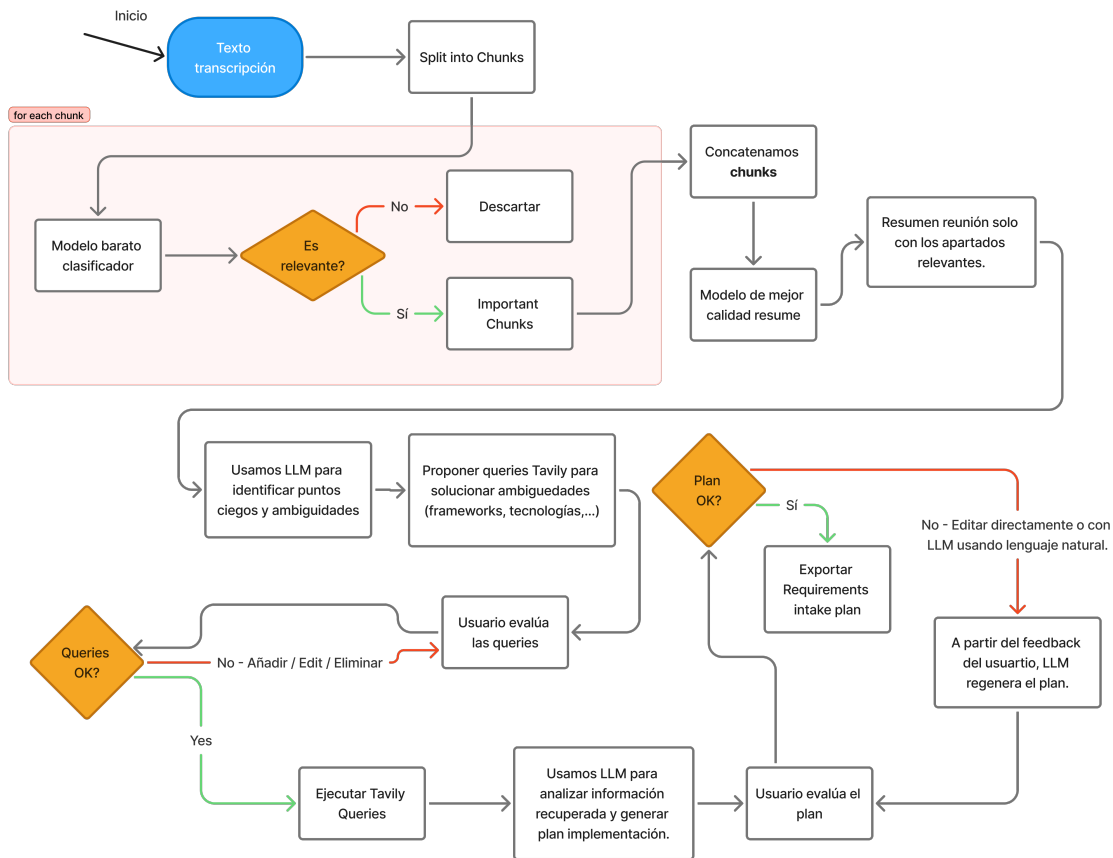


Figura 5: Diagrama de flujo de el proceso generación de plan en la toma de requerimientos.

5.2. Seguimiento del Cliente

La tercera dimensión del proyecto aborda la automatización del seguimiento posterior a las reuniones periódicas con el cliente. El objetivo es minimizar el tiempo dedicado a la redacción de documentación de seguimiento y garantizar que los acuerdos alcanzados en cada reunión queden recogidos de forma estructurada y accesible para ambas partes.

5.2.1. Generación de documentación con Claude Cowork

El flujo implementado mediante Claude Cowork se integra directamente con el proceso de reunión. Al finalizar cada reunión de seguimiento con el cliente, la transcripción o el resumen de lo acordado se proporciona como entrada a una skill de Claude Cowork desarrollada específicamente para este propósito. La skill procesa la información y genera de forma automática un documento Word estructurado que incluye los siguientes apartados:

- Resumen de los temas tratados en la reunión.
- Decisiones tomadas y acuerdos alcanzados.
- Tareas asignadas al equipo de *Colb.AI* con sus plazos correspondientes.
- Tareas o acciones que corresponden al cliente.

Este documento se genera y está disponible para ser enviado al cliente en el mismo momento en que finaliza la reunión, eliminando la latencia habitual en la producción de este tipo de documentación.

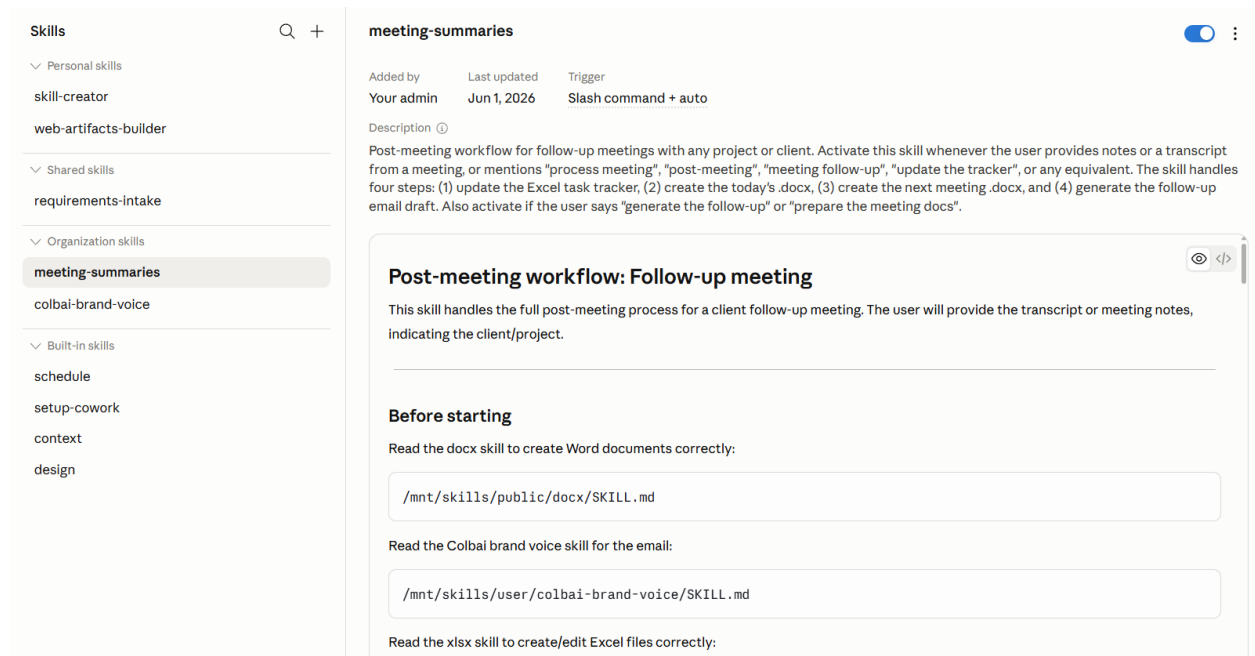


Figura 6: Skill de seguimiento del cliente en Claude Cowork.

La skill aprovecha las capacidades nativas de Claude Cowork para acceder a herramientas de gestión de archivos en el entorno de escritorio, lo que le permite escribir directamente el documento en el directorio de trabajo del proyecto, integrándose de forma transparente en el flujo de gestión documental de la empresa.

5.2.2. Alternativa de código abierto

De forma análoga al enfoque adoptado en la toma de requerimientos, se dispone de una alternativa de código abierto e independiente para los casos en que no se desee depender de la plataforma Claude Cowork. Esta alternativa se basa en una aplicación que combina un modelo de lenguaje de gran tamaño, accedido mediante API, con la biblioteca `python-docx` para la generación de documentos Word con formato estructurado. El flujo de la aplicación es equivalente al de la skill: a partir de la transcripción de la reunión, el modelo genera los distintos apartados del documento de seguimiento con un formato definido, que son entonces volcados al documento Word mediante la biblioteca de generación. Aunque la integración es menos fluida que la ofrecida por Claude Cowork, esta alternativa es igualmente válida para los casos de uso habituales y no incurre en costes de suscripción adicionales más allá del consumo de la API del modelo.

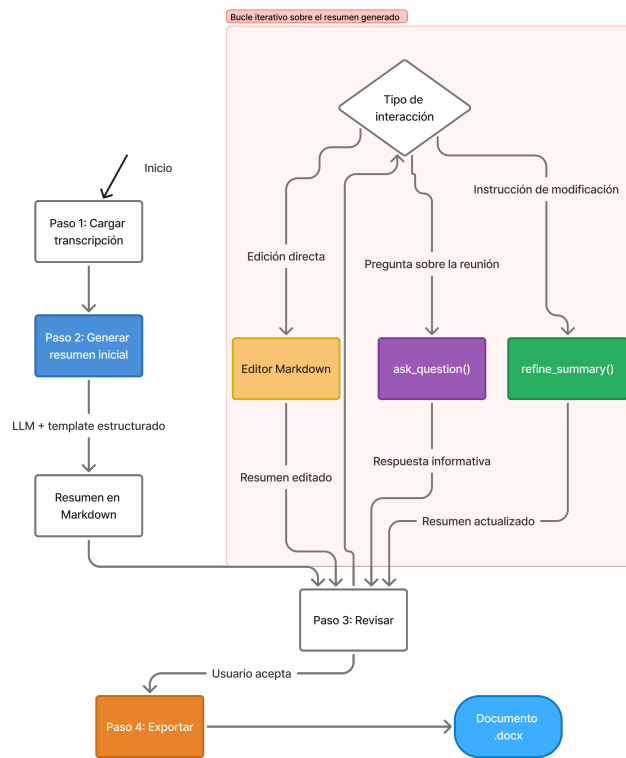


Figura 7: Flujo de la implementación propia del módulo de seguimiento del cliente: generación del resumen inicial y bucle iterativo de refinamiento con *Human in the Loop*.

6. Spec Driven Development: Implementación

Este capítulo describe la arquitectura de herramientas y asistentes que se ha configurado para implementar la metodología SDD en el día a día del equipo de desarrollo.

6.1. Asistentes de Programación

Una de las decisiones de diseño fundamentales del sistema es el rechazo a la dependencia exclusiva de un único proveedor de asistencia al código, fenómeno conocido como *vendor lock-in*. Para mitigar este riesgo, identificado ya en la planificación inicial (Riesgo 4.2.2), se ha optado por la integración de dos asistentes de programación de tipo CLI (*Command-Line Interface*) que operan directamente desde la terminal:

6.1.1. ClaudeCode

ClaudeCode es la herramienta de asistencia a la programación desarrollada por Anthropic [2]. Se trata de un agente CLI que opera directamente en la terminal del desarrollador, con acceso nativo al sistema de archivos del proyecto. ClaudeCode ha establecido el estándar actual en el ámbito de los agentes de programación, introduciendo o popularizando conceptos clave como los agentes especializados, los hooks de ciclo de vida, las skills, los plugins y el protocolo MCP (Model Context Protocol). Al tratarse de un servicio de pago, su uso está sujeto a las condiciones y tarifas de Anthropic.

6.1.2. OpenCode

OpenCode [42] es un asistente de programación de código abierto compatible con modelos de lenguaje locales y remotos. Su arquitectura es funcionalmente equivalente a la de ClaudeCode: opera desde la terminal, tiene acceso al sistema de archivos y es compatible con el protocolo MCP, lo que lo hace interoperable con la mayoría de las herramientas y plugins desarrollados para el ecosistema. Su naturaleza open source y su compatibilidad con modelos locales lo convierten en la alternativa directa a ClaudeCode en escenarios donde se requiere control total sobre los costes o la privacidad de los datos.

6.1.3. Justificación de la doble integración

Al configurar el sistema de modo que ambos asistentes sean capaces de operar con la misma configuración de proyecto (archivos de instrucciones, plugins y MCPs), se elimina la dependencia de cualquiera de los dos proveedores, ucbriendo el riesgo 4.2.2 de *vendor lock-in*. Si las condiciones de uno de ellos cambian de forma desfavorable, el equipo puede migrar su flujo de trabajo sin necesidad de reconfigurarlo desde cero.

Adicionalmente, los asistentes de tipo CLI resuelven de forma más limpia los problemas de integración con el entorno de desarrollo que afectan a los asistentes integrados en el IDE, ya que disponen de herramientas de edición, lectura y búsqueda implementadas directamente sobre el sistema de archivos del sistema operativo.

6.1.4. Arquitectura interna: el patrón ReAct y su implementación en TypeScript

Para comprender plenamente cómo se pueden extender y configurar estas herramientas, es conveniente conocer la arquitectura interna sobre la que están construidas. Tanto ClaudeCode como OpenCode son aplicaciones de TypeScript que implementan un bucle de agente basado en el patrón ReAct (Reasoning + Acting) [61] como paradigma estándar para la construcción de agentes basados en modelos de lenguaje.

El patrón ReAct estructura el comportamiento del agente en un ciclo iterativo de tres fases:

Razonamiento (Thought): El modelo recibe el estado actual del contexto y genera un razonamiento interno en el que planifica el siguiente paso. Este razonamiento es el mecanismo por el que el agente decide qué acción tomar.

Acción (Action): El modelo emite una llamada a herramienta estructurada (*tool call*) especificando qué herramienta invocar y con qué parámetros. Las herramientas disponibles incluyen habitualmente la lectura de archivos, la escritura de código y la ejecución de comandos en terminal, aunque por supuesto dependen del sistema que se utilice.

Observación (Observation): Se ejecuta la acción sobre el sistema de archivos de la terminal y devuelve el resultado al modelo como nuevo mensaje en el historial. Este resultado se convierte en la entrada del siguiente ciclo.

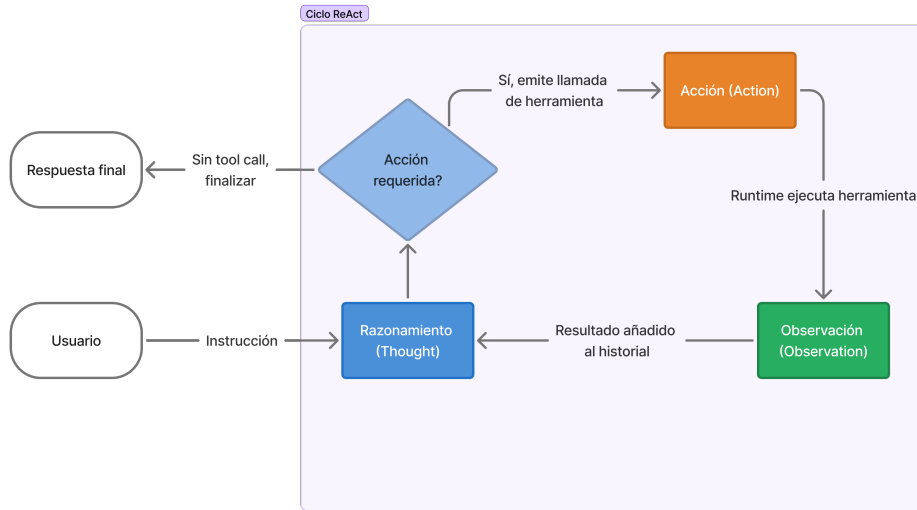


Figura 8: Ciclo iterativo ReAct: el agente alterna entre Razonamiento, Acción y Observación hasta que no hay más herramientas que invocar.

El flujo de ejecución que orquesta este bucle puede describirse de forma simplificada en tres pasos:

1. Al iniciarse una sesión, el runtime construye el system prompt combinando las instrucciones globales del asistente con el contenido del archivo de configuración del proyecto (CLAUDE.md, AGENT.md o equivalente), además de definir el conjunto de herramientas disponibles median-

te un esquema JSON (habitualmente).

2. En cada iteración del bucle, el runtime envía el historial de conversación actualizado a la API del modelo. Si la respuesta contiene llamadas a herramienta, las ejecuta secuencialmente, añade los resultados al historial y lanza una nueva iteración.
3. Si la respuesta no contiene llamadas a herramienta, el runtime la presenta al usuario como respuesta final y espera una nueva instrucción, reiniciando el ciclo con el historial acumulado.

Este diseño es la base sobre la que se apoyan todos los mecanismos de extensibilidad descritos en las próximas secciones. Es posible interceptar el bucle en distintos puntos mediante hooks, inyectar herramientas adicionales mediante plugins, y conectar servicios externos mediante el protocolo MCP.

6.2. Mecanismos de extensibilidad

El valor diferencial de los asistentes CLI frente a otras soluciones reside en su arquitectura de extensibilidad. Antes de describir la configuración concreta del sistema, es necesario definir con precisión los conceptos que la componen, ya que estos términos (plugin, MCP, hook, skill, agente) se utilizan con frecuencia de forma en la literatura del sector.

6.2.1. Conceptos clave

Concepto	Qué es	Cuándo actúa
Hook	Callback registrado en el runtime que se ejecuta automáticamente ante eventos del ciclo de vida del agente (SessionStart, PreToolUse, PostToolUse, etc.). Permite inyectar comportamiento sin intervención explícita del usuario.	Al inicio de sesión, antes o después de editar archivos, o al ejecutar comandos de terminal.
Skill	Documento Markdown con instrucciones procedimentales especializadas para el agente, análogo a las skills de Claude Cowork (véase Fig. 6). Puede incluir scripts, templates y referencias a herramientas.	Cuando el agente detecta que una tarea requiere un procedimiento específico; el contenido se carga bajo demanda para no consumir tokens innecesariamente.
Agente (subagente)	Instancia del LLM creada dinámicamente por el orquestador para ejecutar una tarea en aislamiento, sin acceso al historial de conversación del orquestador.	Para tareas que requieren un contexto acotado e independiente, eliminando interferencias con el hilo principal.
Plugin	Paquete instalable (análogo a un paquete npm) que agrupa y distribuye skills, hooks, comandos y agentes. Es la unidad de distribución del ecosistema. Su estructura incluye carpetas <code>skills/</code> , <code>agents/</code> y un fichero <code>hooks.json</code> .	Al inicializar el proyecto; instalable a nivel de usuario o de proyecto con un único comando.
MCP (Model Context Protocol)[3]	Protocolo estándar abierto que define cómo un proceso externo expone herramientas invocables al agente. El servidor MCP es un proceso independiente (en cualquier lenguaje) que ejecuta las operaciones y devuelve los resultados al runtime.	Cuando el agente necesita capacidades externas: consultar un grafo de código, buscar en la web, leer una base de datos, etc.

Tabla 9: Conceptos clave del sistema de extensibilidad de los asistentes CLI.

Distinción entre MCP y Skill. Aunque ambos mecanismos amplían las capacidades del agente, su naturaleza es diferente. Una skill es esencialmente un prompt procedimental: incluye instrucciones detalladas sobre cómo abordar una tarea y puede invocar herramientas y scripts locales durante su ejecución. Un servidor MCP, en cambio, expone herramientas atómicas acompañadas únicamente de una descripción de su interfaz, sin lógica de orquestación; la ejecución se produce en el proceso del servidor MCP, no en el runtime local del asistente. En la práctica, las skills definen *cómo* debe razonar el agente ante una clase de tarea, mientras que los MCPs definen *qué* operaciones externas puede invocar.

6.2.2. El ciclo de vida completo de una sesión en OpenCode

Para extender y configurar OpenCode de forma eficaz, es necesario comprender con exactitud en qué momento del ciclo se carga cada componente (plugins, skills, herramientas y agentes) y cómo interactúan durante una conversación. Aprovechando que opencode es una herramienta de **código abierto**, a continuación vamos a describir el ciclo de funcionamiento de forma detallada, desde la inicialización del proceso hasta la entrega de la respuesta al usuario.

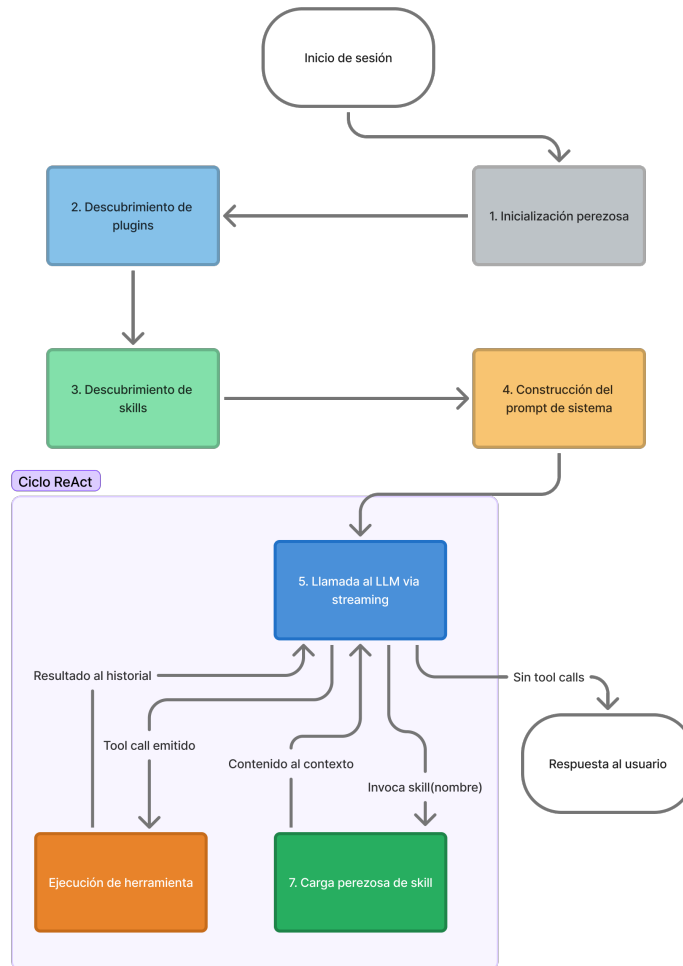


Figura 9: Ciclo de vida completo de una sesión en OpenCode: desde la inicialización perezosa hasta la entrega de la respuesta, con el ciclo ReAct en el núcleo del bucle conversacional.

Inicialización perezosa del espacio de trabajo. OpenCode implementa el patrón de inicialización perezosa (*lazy initialization*): cuando el usuario abre un proyecto por primera vez en la sesión, el sistema no instancia de inmediato todos los servicios. Los plugins, el registro de herra-

mientas, el catálogo de skills y los clientes MCP se crean la primera vez que son necesarios y se mantienen en caché para el resto de la sesión. Esto permite que el proceso arranque rápidamente con independencia del tamaño del proyecto.

Descubrimiento de plugins y herramientas. Cuando el registro de herramientas se inicializa por primera vez, OpenCode ejecuta el proceso de descubrimiento de plugins. Los plugins se cargan desde dos fuentes: (1) los plugins internos del propio asistente, integrados directamente en el binario (autenticación con Codex, GitHub Copilot, GitLab, Cloudflare y otros), y (2) los plugins externos declarados explícitamente en el archivo de configuración de OpenCode, que pueden ser paquetes npm instalados bajo demanda o rutas de archivo locales (por ejemplo, `./mi-plugin.ts`). OpenCode no escanea automáticamente los directorios del proyecto en busca de plugins; la declaración explícita en la configuración es requisito indispensable para cargar un plugin externo. Cada plugin puede registrar herramientas adicionales mediante la interfaz de hooks que OpenCode expone, concretamente el campo `Hooks.tool`. Todas las herramientas —las nativas del asistente más las aportadas por plugins y por los servidores MCP configurados— se registran en el registro global y se describen al modelo mediante un esquema JSON.

Descubrimiento de skills. Paralelamente al descubrimiento de plugins, el sistema escanea el sistema de archivos para localizar las skills disponibles. La búsqueda se realiza en un conjunto predefinido de rutas: `~/.claude/skills/`, `~/.agents/skills/`, los directorios `.claude/` y `.agents/` de la raíz del proyecto, los directorios de configuración global de OpenCode, y las rutas adicionales declaradas en el campo `skills.paths` del archivo `opencode.json`. Para cada skill encontrada, el sistema registra su nombre, descripción y ubicación. Sin embargo, el contenido completo de la skill *no* se carga en este momento: el descubrimiento construye únicamente el catálogo, sin consumir tokens del contexto.

Construcción del system prompt. Al recibir un mensaje del usuario, el sistema construye el system prompt que se enviará al LLM combinando cuatro componentes:

1. **Prompt base del proveedor:** instrucciones genéricas del asistente, adaptadas al modelo concreto que esté activo (Anthropic Claude, Google Gemini u otros). Si el agente activo define un campo `prompt` propio en su configuración, este sustituye al prompt base del proveedor.
2. **Entorno y contexto:** directorio de trabajo actual, fecha, plataforma del sistema operativo y agente activo en la sesión.
3. **Catálogo de skills:** una sección del system prompt lista los nombres y descripciones breves de todas las skills descubiertas. Esta lista es lo único que el LLM recibe sobre las skills en el prompt inicial; el contenido detallado de cada skill se carga bajo demanda más adelante.
4. **Archivo de instrucciones del proyecto:** el contenido de un archivo de instrucciones del repositorio con el nombre `AGENTS.md` o `CLAUDE.md` (para asegurar compatibilidad con proyectos desarrollados usando ClaudeCode), que típicamente incluye la descripción del proyecto, el stack tecnológico, los estándares de código del equipo y las restricciones que el agente debe respetar. Además, los plugins pueden modificar el system prompt resultante mediante el hook `experimental.chat.system.transform`, inyectando secciones adicionales antes de que el prompt se envíe al modelo.

La llamada al LLM mediante streaming. Una vez construido el system prompt y serializado el historial de conversación, OpenCode realiza la llamada al modelo a través de la función `streamText()` del SDK de Vercel AI. Esta función gestiona la transmisión de tokens en tiempo real (*streaming*), de modo que el agente puede reaccionar a cada fragmento de texto generado por el modelo sin esperar a que la respuesta esté completa. Antes de la llamada, los hooks `chat.params` y `chat.headers` permiten a los plugins inyectar parámetros adicionales (temperatura, cabeceras HTTP, etc.).

La respuesta del modelo llega como un flujo asíncrono de eventos que el runtime consume uno a uno. Los tipos de eventos posibles incluyen: inicio de paso, fragmentos de texto, razonamiento interno del modelo, llamadas a herramienta, resultados de herramienta y señales de finalización. A medida que se reciben fragmentos de texto, estos se propagan a la interfaz de usuario en tiempo real, lo que permite al desarrollador seguir el razonamiento del agente sin esperar a que la respuesta esté completa.

Ejecución de herramientas. Cuando el modelo decide invocar una herramienta, el runtime recibe el evento correspondiente con el nombre de la herramienta y sus parámetros. Antes de ejecutar la herramienta, se disparan los hooks `tool.execute.before` de todos los plugins registrados; tras la ejecución, se disparan los hooks `tool.execute.after`. El resultado se añade al historial de conversación como un nuevo mensaje de tipo *tool result* y se inicia un nuevo ciclo del bucle ReAct, reenviando el historial actualizado al modelo.

Carga perezosa de skills bajo demanda. La herramienta `skill` ilustra con claridad el principio de carga perezosa. Cuando el modelo, al ver el catálogo de skills en el system prompt, decide que necesita las instrucciones de una skill concreta para ejecutar una tarea, invoca la herramienta `skill` con el nombre de la skill como parámetro. Es en ese momento —y solo en ese momento— cuando el sistema lee el archivo `SKILL.md` correspondiente del disco, enumera los ficheros asociados a la skill (hasta un máximo de diez), y devuelve el contenido completo al modelo envuelto en una etiqueta XML estructurada. De esta forma, el contenido detallado de las skills no ocupa tokens del contexto inicial: solo las descripciones breves del catálogo están siempre presentes; los contenidos completos se cargan únicamente cuando el modelo los solicita. La skill *using-colbPowers* de `colbPowers` es una excepción deliberada a este principio: al inyectarse mediante el hook `SessionStart` antes de que comience la conversación, su contenido completo llega al modelo directamente en el system prompt, sin pasar por el mecanismo de carga perezosa. Esto garantiza que las reglas de orquestación estén activas desde el primer mensaje, sin depender de que el modelo recuerde invocar la herramienta `skill`.

6.2.3. Similitud con ClaudeCode

ClaudeCode implementa un pipeline estructuralmente análogo en lo que respecta al ciclo de vida de la conversación. Aunque las implementaciones concretas difieren —ClaudeCode no es software libre—, el flujo conceptual comparte los mismos patrones: inicialización perezosa de servicios al inicio de la sesión, construcción del system prompt a partir de archivos de instrucciones y catálogos de skills, bucle ReAct con llamadas a herramienta y streaming de tokens, carga perezosa de skills bajo demanda, y mecanismos de hooks para interceptar el ciclo de vida.

Sin embargo, existen diferencias arquitectónicas significativas que conviene señalar:

Dimensión	ClaudeCode	OpenCode
Proveedores LLM	Exclusivo de modelos compatibles con la API de Anthropic (modelos Claude principalmente)	Más de 25 proveedores a través del SDK de Vercel AI (OpenAI, Google, Azure, Bedrock, Mistral, Groq, GitHub Copilot, entre otros)
Modelo de interfaz	Exclusivamente CLI; interfaz renderizada en terminal mediante React e Ink	Servidor HTTP local (Hono) con aplicación web SolidJS; soporta además aplicaciones de escritorio nativas (Tauri y Electron)
Persistencia	Estado en memoria y ficheros JSON locales	Sesiones persistidas en SQLite con instantáneas del sistema de ficheros antes de cada invocación de herramienta, lo que permite revertir cambios
Framework técnico	TypeScript estándar para la composición de servicios	Framework Effect para gestión de dependencias, errores y concurrencia

Tabla 10: Comparativa arquitectónica entre ClaudeCode y OpenCode.

Esta equivalencia a nivel de flujo de conversación —y no de arquitectura completa— es la que hace posible que un plugin como colbPowers, que se apoya en skills y hooks genéricos, funcione de forma comparable en ambos asistentes, si bien los adaptadores de hooks concretos difieren entre plataformas.

6.2.4. Hooks de ciclo de vida

ClaudeCode expone un sistema de hooks configurables que permite registrar callbacks para distintos eventos del ciclo de vida del agente. Estos hooks pueden definirse de dos formas: directamente por el desarrollador en el archivo `settings.json` del proyecto o del usuario, o bien declararse dentro de un plugin mediante un archivo `hooks.json` incluido en el paquete del plugin —que es el mecanismo que utiliza colbPowers. Los hooks más relevantes para el flujo SDD son **SessionStart** (inyección de contexto al inicio de sesión) y **PreToolUse** (intercepción antes de ejecutar una herramienta concreta, útil para forzar consultas al grafo de código antes de cualquier modificación). En OpenCode, estos hooks se implementan mediante callbacks definidos en el plugin JavaScript correspondiente, con semántica equivalente. En ambas plataformas, plugins y servidores MCP se instalan a nivel de proyecto o de usuario, lo que garantiza que el conjunto de herramientas y comportamientos esté disponible de forma consistente en todas las sesiones del equipo.

6.2.5. Skills y agentes en colbPowers

colbPowers hace un uso central de ambos mecanismos. Las skills definen el comportamiento del orquestador en cada fase del ciclo SDD: el skill *using-colbpowers* (inyectado en SessionStart) contiene las reglas de orquestación y detección de estado del proyecto; *brainstorming* y *writing-plans* instruyen al agente sobre cómo producir los documentos de diseño y planificación; *requesting-code-review* define el protocolo de auditoría del código generado. Los subagentes se crean dinámicamente cuando una tarea requiere ejecución aislada: el orquestador genera un prompt específico para esa

tarea (incluyendo el `design.md` y el `tasks.md` relevantes, y excluyendo el resto del historial) y lanza un subagente que opera exclusivamente sobre ese contexto acotado.

6.3. Herramientas de Apoyo al Flujo SDD

La efectividad del flujo SDD depende en gran medida de la calidad del contexto que el agente tiene disponible en cada momento. A continuación se describen las herramientas principales integradas en el sistema.

6.3.1. `code-review-graph`

`code-review-graph` [12] es una herramienta de código abierto que utiliza el parser de sintaxis Tree-Sitter [56] para analizar el repositorio y construir un grafo de dependencias entre las distintas entidades de código (funciones, clases, módulos). Este grafo se almacena en una base de datos SQLite local y se expone al agente a través de un servidor MCP, proporcionando un mecanismo eficiente para navegar la estructura del código sin necesidad de leer todos los archivos del proyecto.

La integración de esta herramienta aborda directamente el problema de gestión de la ventana de contexto identificado en el apartado de riesgos (sección 4.2.5). Según los *benchmarks* oficiales publicados en el repositorio de la herramienta [12], evaluados sobre 6 repositorios reales de código abierto (Flask, FastAPI, Express, Gin, httpx y el propio `code-review-graph`), el contexto del grafo reduce el consumo de tokens respecto a proporcionar el repositorio completo en un rango de **38× a 528×**, con una mediana de $\sim 82\times$: una consulta típica requiere entre 1.986 y 3.465 tokens extraídos del grafo, frente a los 89.492–951.071 que supondría leer la totalidad de los archivos fuente del repositorio correspondiente. Las capacidades que expone al agente incluyen:

- Análisis de diffs de git para identificar cambios de mayor riesgo.
- Cálculo del radio de impacto (*blast radius*) de cualquier modificación.
- Identificación de los flujos de ejecución afectados por un cambio.
- Búsqueda semántica de entidades de código (funciones, clases, módulos).
- Obtención de una visión de alto nivel de la arquitectura del sistema.

La integración de *code-review-graph* con ClaudeCode se realiza de forma nativa mediante hooks de ciclo de vida. Sin embargo, dado que los sistemas de hooks de ClaudeCode y OpenCode difieren arquitectónicamente, fue necesario desarrollar adaptadores en JavaScript para que OpenCode pudiera invocar el servidor MCP con el mismo comportamiento.

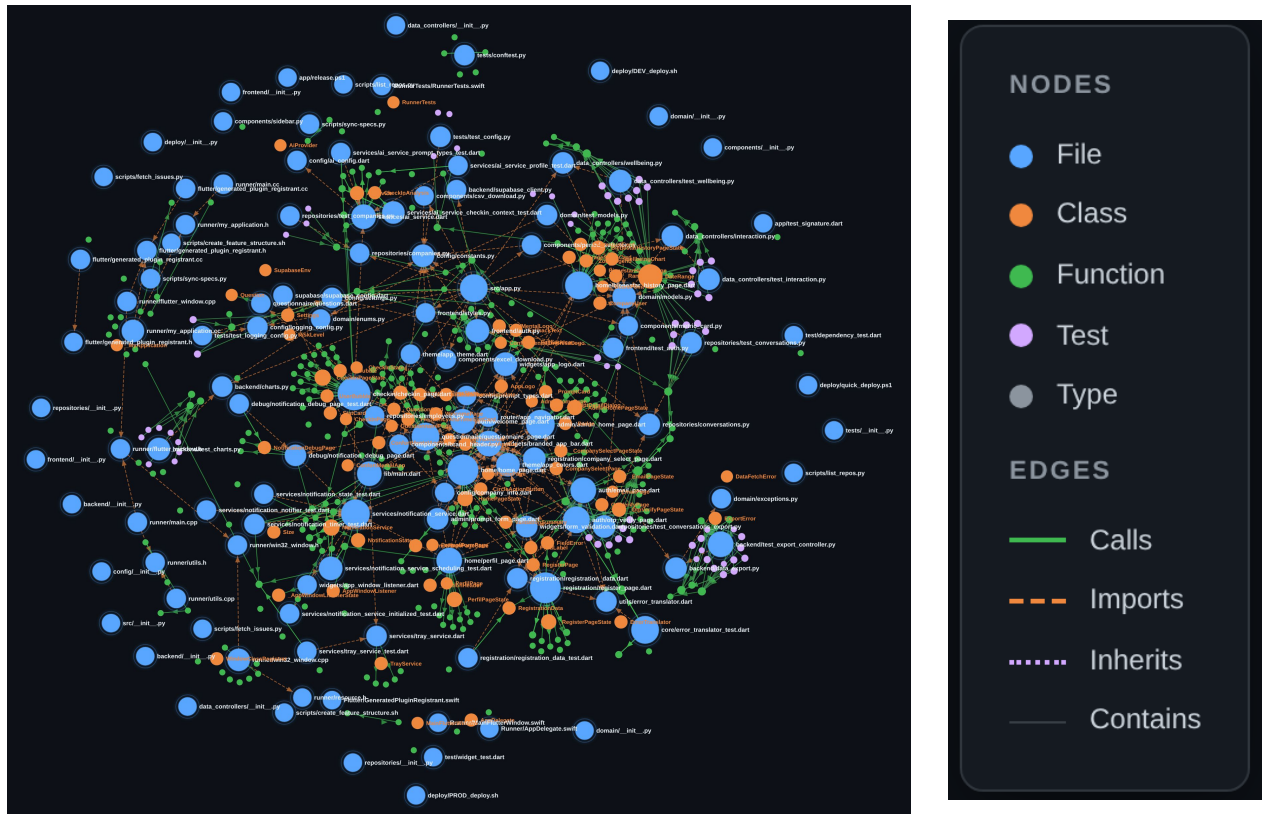


Figura 10: Gráfico estructural del código generado por code-review-graph sobre el repositorio del proyecto (izquierda) y leyenda de tipos de nodo y arista (derecha). Generado sobre un repositorio ya existente de *Colb.AI*.

6.3.2. Definición del flujo de trabajo, colbPowers

colbPowers [9] es un plugin desarrollado para este proyecto a partir de la base proporcionada por el plugin "superpowers" [53], *colbPowers* es instalable en cualquier proyecto y implementa el flujo de trabajo Spec-First de forma autónoma. Una vez instalado, no requiere que el desarrollador recuerde invocar comandos ni seguir un protocolo manual, gracias a una combinación de hooks que inyectan contexto y skills, el agente es capaz de entender la etapa del desarrollo en la que se encuentra en todo momento.

Distribuir el sistema como un plugin —en lugar de como un conjunto de archivos de skills copiados manualmente en el proyecto— aporta dos garantías que los archivos por sí solos no pueden ofrecer: el hook `SessionStart`, que inyecta la skill de forma incondicional en cada sesión sin depender de que el desarrollador la invoque, y una instalación cómoda y reproducible en cualquier proyecto, ya que los plugins se referencian directamente desde su repositorio en GitHub y ambos asistentes los descargan e instalan automáticamente.

El diseño de *colbPowers* no es arbitrario: cada mecanismo de extensibilidad descrito en las secciones anteriores cumple un papel arquitectónico específico dentro del flujo SDD. El hook `SessionStart` convierte el flujo de trabajo de un protocolo opcional en un comportamiento por defecto: al inyectar las reglas de orquestación antes de que el modelo reciba el primer mensaje, garantiza que el agente no pueda operar sin tenerlas presentes, independientemente de si el desarrollador las invoca o no. Las skills codifican cada fase del ciclo SDD como un procedimiento especializado e independiente:

al cargarse únicamente cuando son necesarias, la especificación de brainstorming no ocupa contexto durante la revisión de código, y viceversa, manteniendo el consumo de tokens acotado a la fase activa en cada momento. Los subagentes aislados protegen la integridad de la especificación frente a la deriva conversacional: al arrancar sin acceso al historial de la sesión principal, el subagente implementador solo puede basar sus decisiones en el `design.md` que el orquestador le proporciona explícitamente, eliminando el riesgo de que cambios de alcance discutidos informalmente en el chat contaminen la implementación. Finalmente, el propio skill *using-colbPowers*, inyectado en cada arranque, instruye al agente a verificar que `constitution.md` y `features.md` existen antes de cualquier otra acción, y a crearlos interactivamente si no es así: la especificación no es un documento opcional que el desarrollador puede consultar, sino una precondition estructural al inicio de cada sesión.

Estructura del proyecto esperada Para funcionar, colbPowers asume que el proyecto dispone de una plantilla de repositorio con una estructura específica bajo el directorio `.specs/memory/`. Esta plantilla incluye los documentos base del proyecto y puede contener adicionalmente indicaciones de estilo, planificación, o apuntes y referencias de proyectos anteriores que sirven de contexto al agente:

constitution.md: Documento de alcance global. Define la visión del proyecto, el stack tecnológico, los estándares de código no negociables y las reglas de gobernanza. Es la fuente de verdad sobre cómo se trabaja en este proyecto.

features.md: Roadmap del producto. Lista todas las features aprobadas con su descripción, requisitos funcionales, restricciones técnicas y dependencias. Cualquier nueva funcionalidad debe estar aquí definida antes de poder planificarse.

En caso de que una de las anteriores no exista, al agente se le indica que ejecute las skills de "defining-constitution" y "defining-features", para popular estos archivos a través de un proceso interactivo de preguntas al usuario.

docs/specs/: Documentos de diseño por feature: descripción del comportamiento esperado, decisiones de arquitectura y justificación técnica de cada implementación.

docs/plans/: Planes de implementación granulares por feature: listas de tareas ordenadas con criterios de aceptación.

Flujo de trabajo autónomo El flujo de trabajo completo se desarrolla de la siguiente forma cuando el usuario solicita cualquier cambio o nueva funcionalidad:

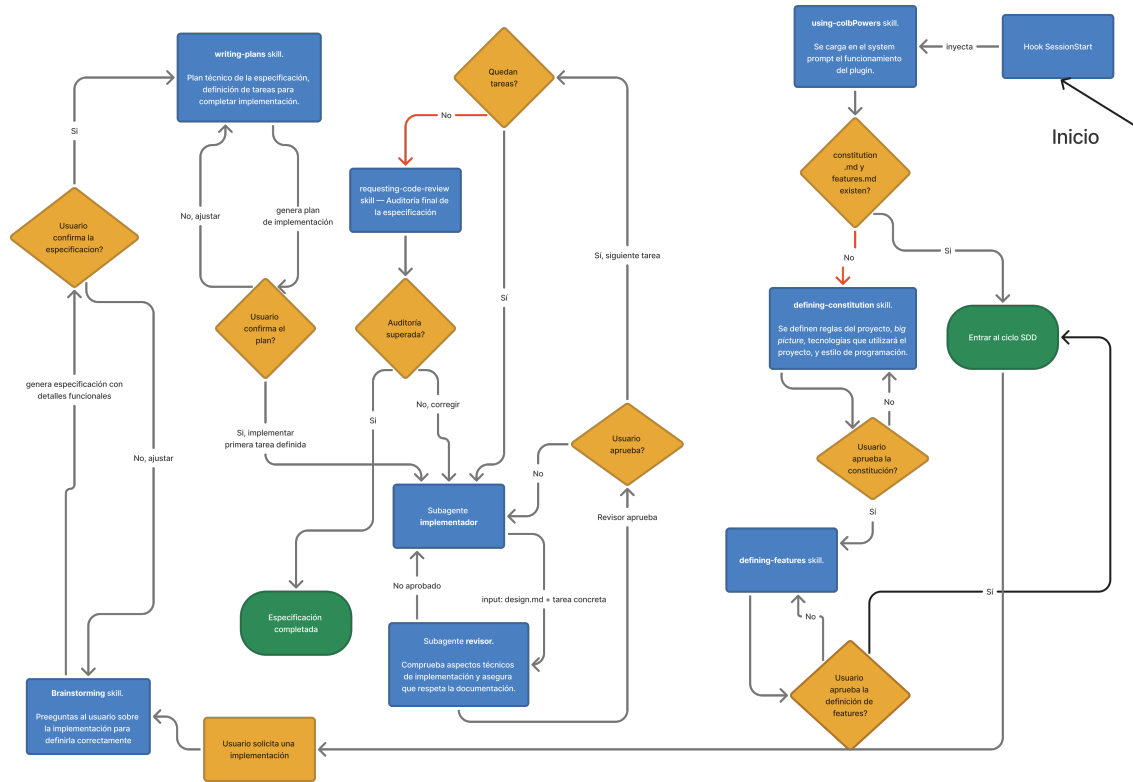


Figura 11: Diagrama de flujo de colbPowers.

1. **Verificación del estado del proyecto:** al iniciar la sesión, el agente lee `.specs/memory/` y comprueba si existen la constitución y el roadmap de features. Si alguno de los dos está ausente, el agente inicia de forma proactiva una conversación guiada con el usuario para definirlos, formulando preguntas sobre la visión del proyecto, el stack tecnológico, los estándares de código y las funcionalidades previstas. Este paso se ejecuta una única vez por proyecto.
2. **Definición interactiva de la especificación:** una vez que los documentos base existen, el agente lleva al usuario por un proceso de preguntas estructuradas para definir con precisión la nueva implementación. Se determina qué debe hacer, a cuál de las features del roadmap pertenece, cuáles son los comportamientos esperados, qué restricciones técnicas aplican y qué criterios de aceptación debe cumplir. El objetivo es eliminar toda ambigüedad antes de escribir una sola línea de código.
3. **Generación de la especificación:** cuando el usuario confirma que la definición es correcta, el agente redacta el documento de especificación (`design.md`) y lo guarda en `docs/specs/`. Este documento es la fuente de verdad de la implementación y es consultado en cada paso posterior.
4. **Generación del plan de implementación:** a partir de la especificación, el agente elabora un plan de tareas (`plan.md`) en `docs/plans/` que desglosa la implementación en pasos concretos y ordenados, cada uno con su criterio de aceptación explícito.

5. **Ejecución iterativa con implementador y revisor:** el agente utiliza la herramienta de lista de tareas (**ToDoWrite**) integrada en los asistentes CLI para ejecutar el plan de forma iterativa. El agente orquestador da la opción al desarrollador de usar **subagent-driven-development** o **inline-execution**: (1) La primera opción para cada tarea actúa un subagente implementador que escribe el código, y a continuación un subagente revisor que verifica que el código se ajusta a la especificación, sigue el plan y pasa los tests definidos. (2) La segunda opción utiliza al propio agente orquestador como implementador, y revisor, pero como al ser el mismo agente el que lo hace todo, estará mas sesgado por el contexto anterior, adicionalmente, mediante se llene su contexto, será mas probable que se salte alguno de los pasos del flujo de implementación.

Apunte sobre ToDoWrite: Esta herramienta mantiene el plan de implementación como una lista de tareas con estados explícitos (*pendiente, en progreso, completada*), visible en todo momento en la interfaz del asistente. Esta visibilidad actúa como ancla de atención: al inicio de cada iteración, el agente consulta la lista para saber exactamente qué tarea tiene en curso y cuál es la siguiente, lo que reduce significativamente la tendencia de los LLMs a desviarse del plan o a saltarse pasos. Cabe mencionar que el runtime no bloquea al agente si este no respeta el orden establecido; la efectividad de **ToDoWrite** como mecanismo de seguimiento proviene de la combinación de la visibilidad explícita del estado y de las instrucciones del sistema que describen su protocolo de uso.

6.3.3. La plantilla de repositorio

La plantilla de repositorio que colbPowers espera encontrar está diseñada para mantenerse ligera y no imponer burocracia innecesaria. Los únicos documentos imprescindibles son `constitution.md` y `features.md`, que el propio agente puede generar desde cero en la primera sesión si no existen. El resto de la estructura (`docs/specs/`, `docs/plans/`) se va poblando de forma orgánica a medida que el equipo desarrolla features, convirtiéndose en un registro vivo y trazable de todas las decisiones de diseño del proyecto.

Esta aproximación resuelve uno de los problemas identificados en el análisis del estado del arte: a diferencia de herramientas como Spec Kit, que exigen documentación exhaustiva incluso para cambios pequeños, colbPowers adapta el nivel de formalismo a la magnitud de la tarea. Una corrección menor puede implementarse directamente con el agente; una nueva feature importante pasa por el ciclo completo de especificación, planificación y revisión. El agente decide el nivel de rigor apropiado en cada caso, basándose en el contexto del proyecto y la magnitud del cambio solicitado.

7. Experimentos y evaluación

Con el objetivo de evaluar empíricamente el impacto de las herramientas desarrolladas, se ha ejecutado un experimento controlado que compara dos condiciones, **baseline** (herramientas nativas sin plugins) frente a **colbPowers** (colbPowers + code-review-graph), sobre dos casos de uso distintos, con dos asistentes de programación y tres repeticiones por condición ($n=3$).

7.1. Diseño experimental

Las dos condiciones experimentales son:

Condición	Herramientas	acti-vas	Descripción
baseline	Herramientas CLI	nativas	Solo Read, Write, Edit, Bash, Grep. Sin plugins ni servidores MCP adicionales. Actúa como línea base.
colbPowers	Nativas + colbPowers + code-review-graph		El agente sigue el flujo SDD de colbPowers con acceso al servidor MCP de code-review-graph. La tarea sigue el ciclo: constitution → features → spec → plan → execute → review.

Tabla 11: Condiciones experimentales.

Cada condición se ha ejecutado con dos asistentes: **OpenCode con DeepSeek V4-Flash**[17] y **ClaudeCode con Haiku 4.5**[5]. Las métricas recogidas son: tokens totales consumidos (incluyendo caché), número de tool calls del orquestador, y número de llamadas a herramientas de code-review-graph. En ambos casos se recogen adicionalmente métricas de calidad del código generado (complejidad ciclomática e índice de mantenibilidad) mediante análisis estático: **radon** [33] para el Caso 2 (Python) y **DCM — Dart Code Metrics** [16] para el Caso 1 (Flutter/Dart).

Limitaciones del diseño experimental. Deben señalarse dos limitaciones estructurales.

1. El tamaño muestral de $n = 3$ repeticiones por condición limita la potencia estadística de los resultados e impide verificar los supuestos de normalidad del test t de Welch; los resultados deben interpretarse como evidencia sugerente.
2. Tanto el diseño de la intervención (colbPowers) como la ejecución y evaluación de los experimentos son obra del mismo autor, en el contexto de la empresa para la que se desarrolla el sistema (*Colb.AI*). Esta ausencia de evaluación ciega o independiente es una limitación metodológica reconocida, y los resultados deben interpretarse teniendo en cuenta un potencial sesgo de confirmación.

Metodología: Para reducir al máximo los sesgos de confirmación y manipulación involuntaria por parte del experimentador, se aplica un protocolo de intervención mínima con las siguientes reglas, ordenadas por prioridad:

- **(P1) Seguridad ante todo:** El experimentador puede interrumpir al agente en cualquier momento si detecta que está a punto de ejecutar una acción destructiva o peligrosa (eliminación de archivos, sobrescritura de datos críticos, etc.).
- **(P2) Solo responder lo que el agente solicite:** El experimentador únicamente responde preguntas directas del agente o acepta/deniega las solicitudes de permiso que este realice explícitamente. No se ofrece información ni correcciones de forma proactiva.
- **(P3) Feedback sobre pruebas manuales:** Si el agente solicita validación o el experimentador detecta un fallo durante las pruebas manuales de la implementación, puede comunicar el resultado observado (por ejemplo, que la funcionalidad no se comporta como se espera), sin indicar la causa ni cómo corregirlo.
- **(P4) Pistas como último recurso:** Solo en caso de bloqueo prolongado sin progreso, el experimentador puede proporcionar una pista mínima que desbloquee al agente, dejando constancia de la intervención en el registro del experimento.

Metodología estadística. Se utiliza el test t de Welch [60] para comparar las medias de cada métrica entre condiciones, dado que no puede asumirse igualdad de varianzas entre grupos. Como estimador del tamaño del efecto se emplea la g de Hedges [24], que corrige el sesgo de la d de Cohen en muestras pequeñas. Se emplea el umbral estándar de $p < 0,05$ como indicativo de significación estadística; la g cuantifica la magnitud solo cuando el p -value permite rechazar la hipótesis nula. Los umbrales de interpretación de g son: trivial ($g < 0,2$), pequeño ($0,2 \leq g < 0,5$), mediano ($0,5 \leq g < 0,8$) y grande ($g \geq 0,8$) [13]. Dado el tamaño muestral ($n = 3$), todos los p -values deben considerarse indicativos, no confirmatorios.

7.2. Caso 1: Aplicación Flutter

El primer experimento se llevó a cabo sobre un repositorio real de *Colb.AI*. Con tal de respetar las políticas de confidencialidad, la descripción del proyecto es lo suficientemente detallada como para entender el contexto, omitiendo ciertos detalles para anonimizarlo.

El proyecto consiste en una aplicación de escritorio desarrollada con el SDK *Flutter*. La tarea asignada al asistente fue la **eliminación de tres funcionalidades** de la aplicación. Estas funcionalidades eran de naturaleza técnica compleja: una de ellas coordinaba un modelo de IA mediante *LangGraph*, y otras requerían interacción con una base de datos. Su eliminación implicaba identificar y desacoplar código repartido a lo largo de múltiples archivos del repositorio, sin comprometer el resto de la funcionalidad existente.

La hipótesis de partida era que colbPowers obtendría mejores resultados que el baseline en esta tarea por dos razones complementarias. En primer lugar, el servidor MCP *code-review-graph* permitiría al agente explorar el grafo de dependencias del repositorio para identificar con precisión qué entidades de código dependían de las funcionalidades a eliminar, reduciendo el riesgo de dejar referencias huérfanas o eliminar código del que otras partes dependen. En segundo lugar, el flujo de planificación previa de colbPowers (especificación del alcance de la eliminación y plan de tareas antes de modificar código) debería producir una intervención más ordenada y con menor complejidad residual que una eliminación directa.

Condición	Ejecución	Total (M)	Input (M)	Output (M)	Caché (M)	Tool calls	CRG	Comp. cic.	Mant. Index
baseline	Ejec. 1	3,67	1,15	0,02	2,49	74	0	2,67	64,85
	Ejec. 2	2,76	0,89	0,02	1,85	68	0	2,84	68,66
	Ejec. 3	3,37	0,91	0,02	2,43	66	0	2,50	69,27
	Media	3,26	0,98	0,02	2,26	69	0	2,67	67,59
colbPowers	Ejec. 1	10,64	5,40	0,07	5,15	108	0	3,05	69,55
	Ejec. 2	10,55	4,18	0,07	6,27	71	2	3,53	65,11
	Ejec. 3	6,95	2,61	0,05	4,27	98	0	3,53	65,11
	Media	9,38	4,07	0,06	5,23	92	0,7	3,37	66,59

Tabla 12: Caso 1 — OpenCode con DeepSeek V4-Flash. Caché = tokens leídos de caché. Métricas de calidad calculadas con DCM.

Condición	Ejecución	Total (M)	Input (M)	Output (M)	Caché (M)	Tool calls	CRG	Comp. cic.	Mant. Index
baseline	Ejec. 1	6,64	0,01	0,04	6,50	76	0	2,76	63,63
	Ejec. 2	2,24	0,01	0,03	2,15	44	0	3,19	70,51
	Ejec. 3	2,33	0,00	0,03	2,25	38	0	2,98	64,02
	Media	3,74	0,00	0,03	3,63	53	0	2,98	66,05
colbPowers	Ejec. 1	12,23	0,00	0,11	11,66	56	0	2,97	67,03
	Ejec. 2	14,91	0,00	0,10	14,28	56	0	3,53	69,03
	Ejec. 3	15,85	0,00	0,11	15,15	76	3	3,94	63,51
	Media	14,33	0,00	0,11	13,70	63	1,0	3,48	66,52

Tabla 13: Caso 1 — ClaudeCode con Haiku 4.5. Caché = tokens leídos de caché. Métricas de calidad calculadas con DCM.

En ambos asistentes el paso de baseline a colbPowers multiplica el consumo de tokens ($\times 2,9$ en OpenCode, $\times 3,8$ en ClaudeCode). El número de tool calls crece moderadamente en OpenCode ($69 \rightarrow 92$) y se mantiene similar en ClaudeCode ($53 \rightarrow 63$). Las llamadas a *code-review-graph* son esporádicas: la Ejecución 2 de OpenCode colbPowers registró 2 llamadas y la Ejecución 3 de ClaudeCode colbPowers registró 3; el resto, cero.

El desglose de tokens revela una diferencia estructural entre asistentes. En OpenCode los tokens de entrada no cacheados son significativos (media 0,98 M en baseline, 4,07 M en colbPowers), lo que refleja que cada subagente inicia un contexto propio sin aprovechar caché de sesiones anteriores. En ClaudeCode, en cambio, los tokens de entrada son prácticamente nulos (aprox. 0,00 M) y casi la totalidad del consumo corresponde a tokens de caché (3,63 M en baseline, 13,70 M en colbPowers).

En cuanto a la calidad del código, colbPowers produce mayor complejidad ciclomática que baseline en ambos asistentes (OpenCode: 3,37 frente a 2,67, +26 %; ClaudeCode: 3,48 frente a 2,98, +17 %), sin variación apreciable en el índice de mantenibilidad (OpenCode: 66,59 frente a 67,59; ClaudeCode: 66,52 frente a 66,05). El análisis estadístico y la interpretación de estos resultados se presentan en la Sección 7.4.

7.3. Caso 2: Visualizador de clasificadores en Python

El segundo caso aplica el experimento sobre una tarea de nueva implementación desde cero: el desarrollo en Python de un visualizador interactivo que muestra, iteración a iteración, cómo dos clasificadores —*K-Nearest Neighbors* (KNN) y *Logistic Regression*— aprenden a separar un conjunto de clases en un espacio bidimensional. A diferencia del caso anterior (modificación de una base de código existente), este caso evalúa el comportamiento de las herramientas en la creación de funcionalidad nueva.

Este caso es especialmente relevante para evaluar colbPowers, ya que puede ejecutar el ciclo completo de la metodología SDD: primero se generan de forma interactiva con el usuario los documentos de constitución y *features* del proyecto, y a continuación se genera la especificación técnica de la funcionalidad a implementar. En todo momento el proceso sigue el principio de *Human-in-the-Loop*: el usuario participa en cada etapa y puede controlar el flujo.

Condición	Ejecución	Total (M)	Input (M)	Output (M)	Caché (M)	Tool calls	CRG	Comp. ciclom.	Mant. Index
baseline	Ejec. 1	1,59	0,97	0,01	0,59	36	0	4,67	44,09
	Ejec. 2	2,52	1,41	0,02	1,08	70	0	4,00	39,24
	Ejec. 3	1,53	0,38	0,02	1,13	49	0	4,25	41,30
	Media	1,88	0,92	0,02	0,93	52	0	4,31	41,54
colbPowers	Ejec. 1	6,53	2,69	0,06	3,77	80	1	3,27	47,20
	Ejec. 2	10,22	6,28	0,07	3,84	86	1	2,97	48,66
	Ejec. 3	8,47	5,12	0,06	3,25	89	0	3,16	47,47
	Media	8,41	4,70	0,06	3,62	85	0,7	3,13	47,78

Tabla 14: Caso 2 — OpenCode con DeepSeek V4-Flash. Caché = tokens leídos de caché.

Condición	Ejecución	Total (M)	Input (M)	Output (M)	Caché (M)	Tool calls	CRG	Comp. ciclom.	Mant. Index
baseline	Ejec. 1	6,12	0,00	0,05	6,00	90	0	2,50	49,92
	Ejec. 2	2,75	0,00	0,02	2,68	54	0	3,23	62,24
	Ejec. 3	2,37	0,00	0,04	2,27	44	0	2,83	58,76
	Media	3,74	0,00	0,04	3,65	63	0	2,85	56,97
colbPowers	Ejec. 1	7,30	0,00	0,11	6,82	46	0	2,77	50,56
	Ejec. 2	6,28	0,00	0,09	5,93	44	1	3,65	67,18
	Ejec. 3	10,97	0,00	0,11	10,11	55	0	2,90	59,28
	Media	8,19	0,00	0,10	7,62	48	0,3	3,11	59,01

Tabla 15: Caso 2 — ClaudeCode con Haiku 4.5. Caché = tokens leídos de caché.

El desglose de tokens del Caso 2 confirma el mismo patrón estructural del Caso 1: en ClaudeCode el consumo se concentra en caché (3,65 M en baseline, 7,62 M en colbPowers) con tokens de entrada prácticamente nulos, mientras que en OpenCode los tokens de entrada son significativos (0,92 M en baseline, 4,70 M en colbPowers). A diferencia del Caso 1, en OpenCode la distribución entre input y caché es más equilibrada en la condición baseline (0,92 M frente a 0,93 M), consistente con que el proyecto parte de un repositorio vacío. El sobrecoste de colbPowers es $\times 4,5$ en OpenCode y $\times 2,2$ en ClaudeCode. En ClaudeCode el orquestador realiza menos tool calls bajo colbPowers (48) que bajo baseline (63), reflejo de su mayor delegación a subagentes.

En cuanto a la calidad del código, en OpenCode colbPowers genera código de menor complejidad ciclomática (3,13 frente a 4,31, -27%) y mayor índice de mantenibilidad (47,78 frente a 41,54, $+15\%$). En ClaudeCode los cambios son menores: complejidad $2,85 \rightarrow 3,11$ y mantenibilidad $56,97 \rightarrow 59,01$ ($+3,6\%$). Destaca que en la condición baseline ClaudeCode produce código con un índice de mantenibilidad notablemente mayor que OpenCode (56,97 frente a 41,54). El análisis estadístico y la interpretación de estos resultados se presentan en la Sección 7.4.

7.4. Discusión de los resultados

7.4.1. Análisis de consumo

El sobrecoste de tokens de colbPowers es el hallazgo más robusto del experimento: presente en todas las condiciones evaluadas, con tamaño de efecto grande en todas ellas. En el Caso 1, el factor respecto al baseline es $\times 2,9$ para OpenCode ($p = 0,032$, $g = +3,21$) y $\times 3,8$ para ClaudeCode ($p = 0,005$, $g = +3,81$); en el Caso 2, $\times 4,5$ para OpenCode ($p = 0,019$, $g = +3,84$) y $\times 2,2$ para ClaudeCode ($p = 0,077$, $g = +1,56$). La única excepción es el Caso 2 con ClaudeCode, donde la diferencia no alcanza significación estadística con $n = 3$, aunque el tamaño de efecto es igualmente grande. Agrupando ambos casos, el sobrecoste es altamente significativo tanto para OpenCode ($p < 0,001$, $g = +4,03$) como para ClaudeCode ($p = 0,004$, $g = +2,23$). Este coste refleja el overhead estructural del flujo SDD: generación de documentación (constitution, features, spec, plan), delegación a una media de 9 a 14 subagentes por sesión y ejecución de revisiones sistemáticas. La Figura 12 muestra el consumo por experimento, asistente y condición.

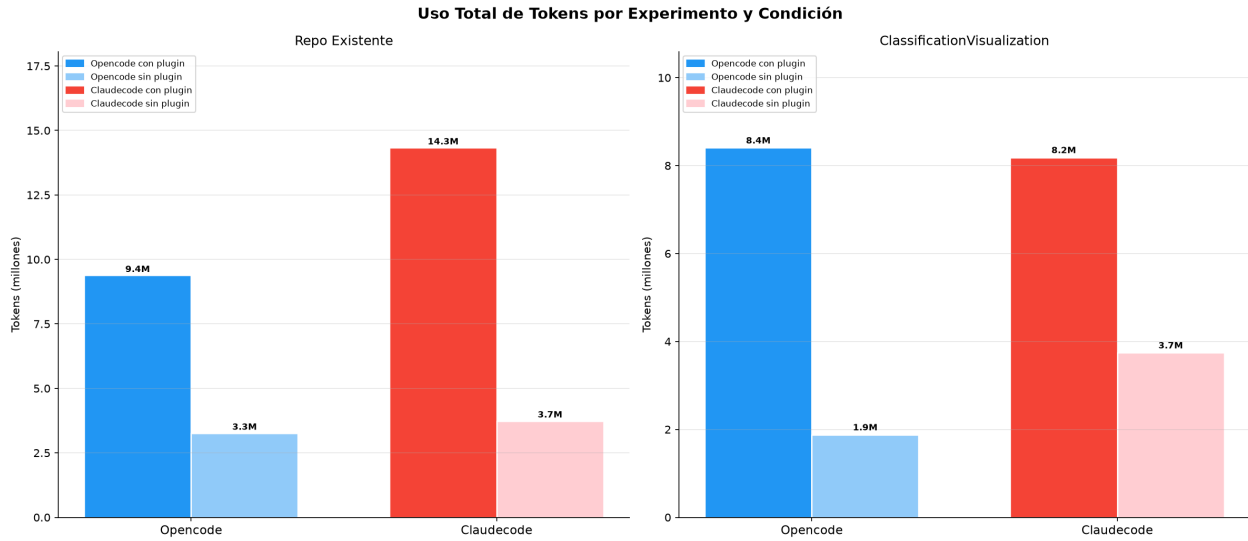


Figura 12: Tokens totales por experimento y condición, separados por asistente. La comparación entre columnas de distinto asistente no es directa por diferencias de tokenizador y caché.

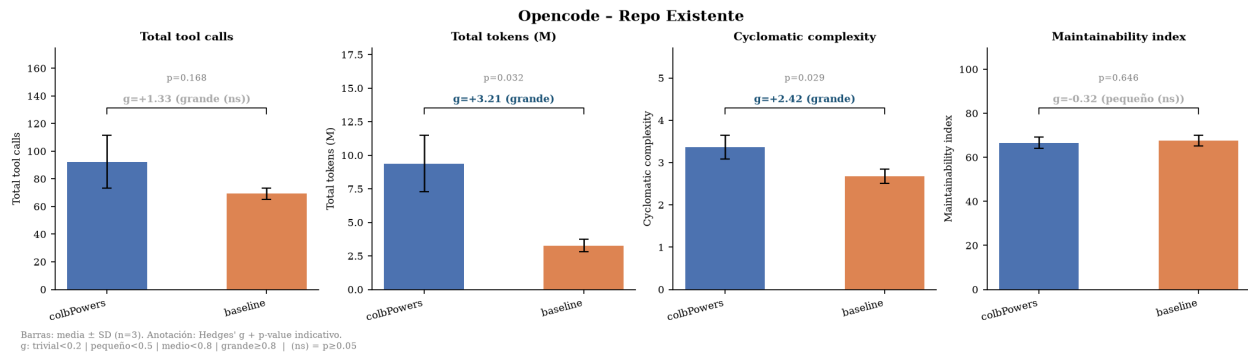


Figura 13: Test t de Welch — colbPowers vs. baseline, OpenCode, Caso 1.

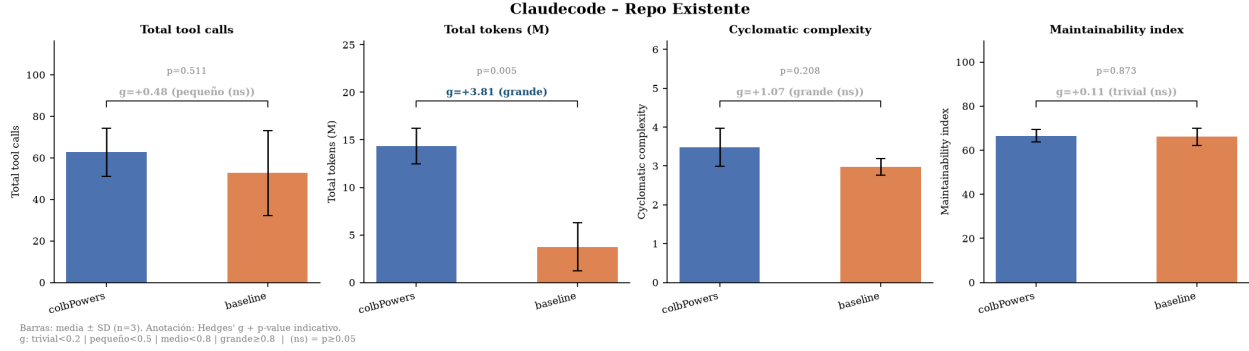


Figura 14: Test t de Welch — colbPowers vs. baseline, ClaudeCode, Caso 1.

El desglose por tipo de token revela una diferencia estructural entre los dos asistentes. En OpenCode los tokens de entrada no cacheados son significativos (media 0,98 M en baseline y 4,07 M en colbPowers para el Caso 1), lo que indica que cada subagente inicia un contexto propio sin aprovechar la caché de sesiones anteriores. En ClaudeCode, en cambio, los tokens de entrada son prácticamente nulos y casi la totalidad del consumo corresponde a tokens de caché (3,63 M en baseline y 13,70 M en colbPowers para el Caso 1); la causa de esta diferencia de comportamiento no se ha podido determinar en el alcance de este trabajo. En cualquier caso, esta asimetría hace que la comparación del total de tokens entre asistentes no sea directa: ClaudeCode acumula más tokens totales que OpenCode bajo colbPowers en el Caso 1 (14,33 M frente a 9,38 M, $p = 0,039$, $g = -1,98$), pero esto refleja principalmente mayor reutilización de caché, no mayor trabajo efectivo. En baseline, en cambio, los totales de ambos asistentes son comparables (3,26 M frente a 3,74 M, diferencia no significativa).

Este sobrecoste no debe interpretarse en aislamiento. colbPowers genera como subproducto documentación estructurada (constitución del proyecto, especificación de cambios, plan de implementación) que persiste en el repositorio, y su énfasis en criterios de aceptación explícitos propicia la generación de tests como parte natural del flujo. En un proyecto con horizonte de vida largo, estos beneficios trascienden el consumo inmediato de tokens.

7.4.2. Métricas de calidad

Las métricas de calidad —complejidad ciclomática e índice de mantenibilidad— se calcularon para ambos casos con herramientas de análisis estático adaptadas al lenguaje: DCM para Flutter/Dart (Caso 1) y radon para Python (Caso 2). Se presentan primero los resultados del Caso 2 y a continuación los del Caso 1, dado que el patrón difiere cualitativamente entre ellos.

Caso 2 (ClassificationVisualization) — Baseline vs. colbPowers. La Figura 15 muestra las métricas medias por asistente y condición. En OpenCode, colbPowers produce una reducción de la complejidad ciclomática (de 4,31 a 3,13, -27% ; $p = 0,014$, efecto grande) y un aumento del índice de mantenibilidad (de 41,54 a 47,78, $+15\%$; $p = 0,037$, efecto grande). En ClaudeCode las diferencias no alcanzan significación estadística: la complejidad ciclomática pasa de 2,85 a 3,11 ($p = 0,508$) y el índice de mantenibilidad de 56,97 a 59,01 ($p = 0,754$), por lo que este test no ha podido detectar un efecto estadísticamente significativo; con $n = 3$ el efecto podría existir sin ser detectable.

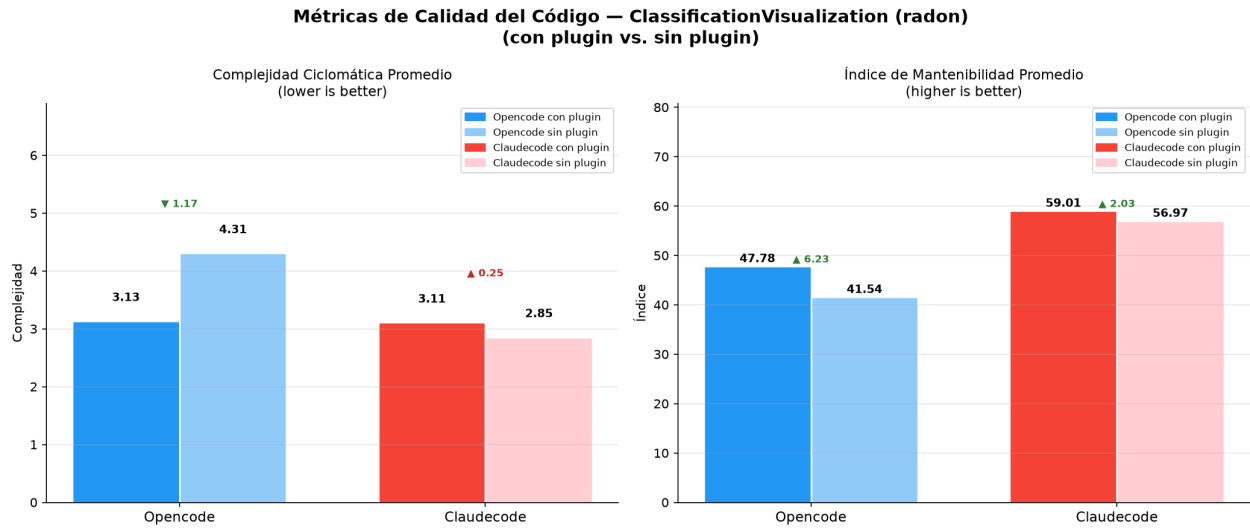


Figura 15: Métricas de calidad del código — Caso 2 (ClassificationVisualization, radon/Python). Medias por asistente y condición.

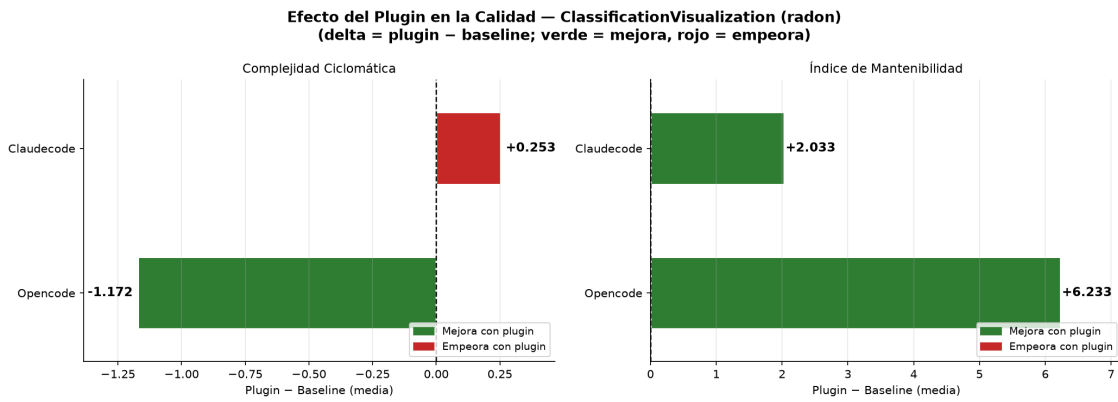


Figura 16: Efecto de colbPowers sobre la calidad del código en el Caso 2 (delta = colbPowers – baseline; verde: mejora, rojo: empeora).

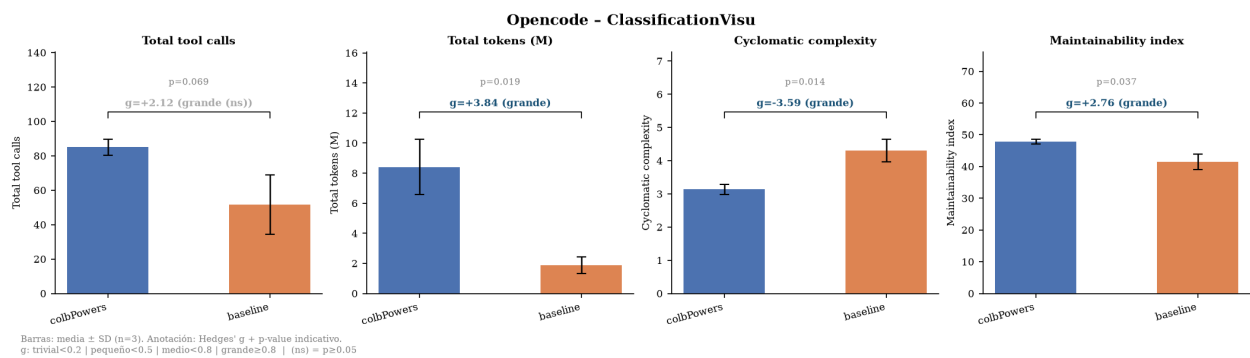


Figura 17: colbPowers vs. baseline, OpenCode, Caso 2. Barras: media ± SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).

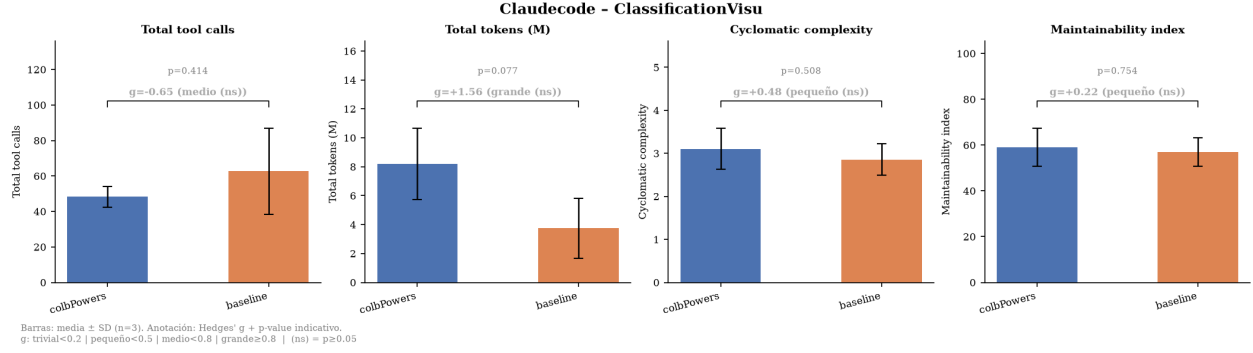


Figura 18: colbPowers vs. baseline, ClaudeCode, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).

Caso 2 — OpenCode vs. ClaudeCode. En baseline, ClaudeCode genera código con menor complejidad ciclomática (2,85 vs. 4,31; $p = 0,007$, efecto grande) y mayor índice de mantenibilidad (56,97 vs. 41,54; $p = 0,039$, efecto grande). Con colbPowers activo, las diferencias se reducen considerablemente —la complejidad converge a valores casi idénticos (3,13 OpenCode, 3,11 ClaudeCode)— y los p -values no alcanzan significación estadística, excepto en las tool calls del orquestador, donde ClaudeCode realiza menos llamadas que OpenCode ($p = 0,001$, $g = +5,58$), reflejo de su mayor delegación a subagentes.

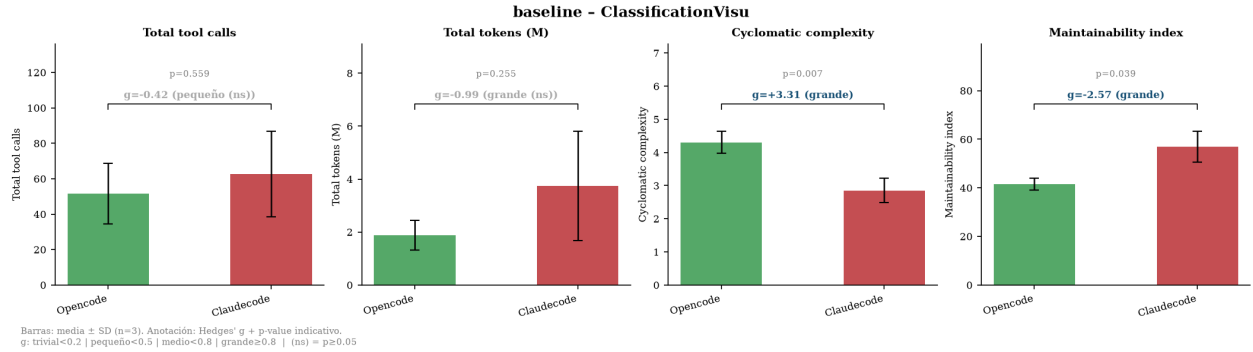


Figura 19: OpenCode vs. ClaudeCode, baseline, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).

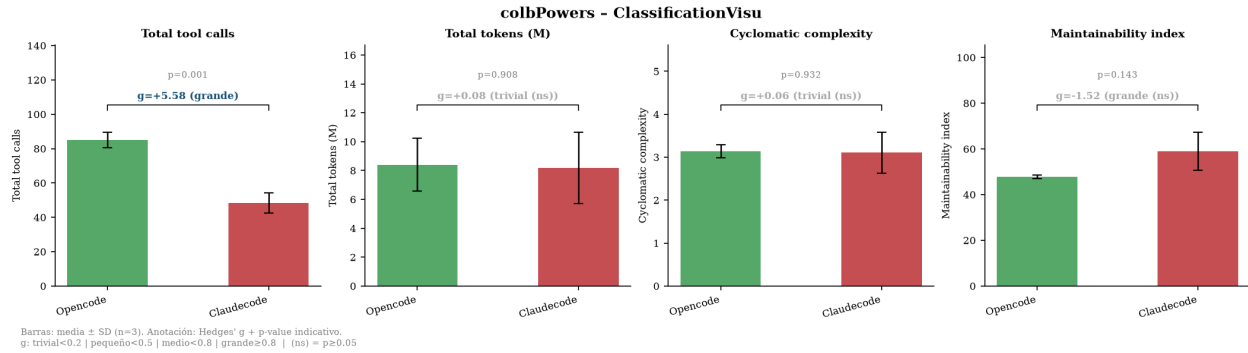


Figura 20: OpenCode vs. ClaudeCode, colbPowers, Caso 2. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo. (ns: no significativo, $p > 0,05$).

Caso 1 (Repo Existente) — Baseline vs. colbPowers. En el Caso 1 el patrón se invierte respecto al Caso 2: colbPowers produce mayor complejidad ciclomática que baseline tanto en OpenCode (3,37 vs. 2,67, +26 %; $p = 0,029$, $g = +2,42$) —diferencia estadísticamente significativa— como en ClaudeCode (3,48 vs. 2,98, +17 %; $p = 0,208$, $g = +1,07$), aunque en este último sin soporte estadístico. El índice de mantenibilidad no muestra diferencia apreciable en ningún asistente (OpenCode: $p = 0,646$, $g = -0,32$; ClaudeCode: $p = 0,873$, $g = +0,11$). Este resultado no confirma la hipótesis de que *code-review-graph* proporcionaría un contexto de dependencias superior: las llamadas al servidor MCP fueron prácticamente nulas en todas las ejecuciones, lo que privó al flujo SDD de la ventaja de exploración que habría justificado su overhead.

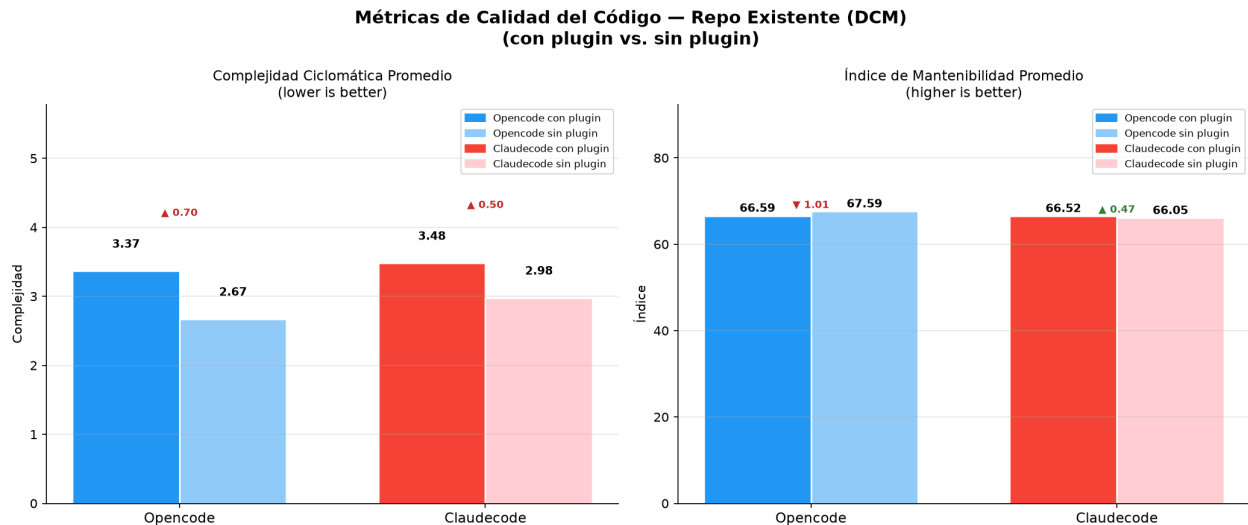


Figura 21: Métricas de calidad del código — Caso 1 (Repo Existente, DCM/Flutter). Medias por asistente y condición.

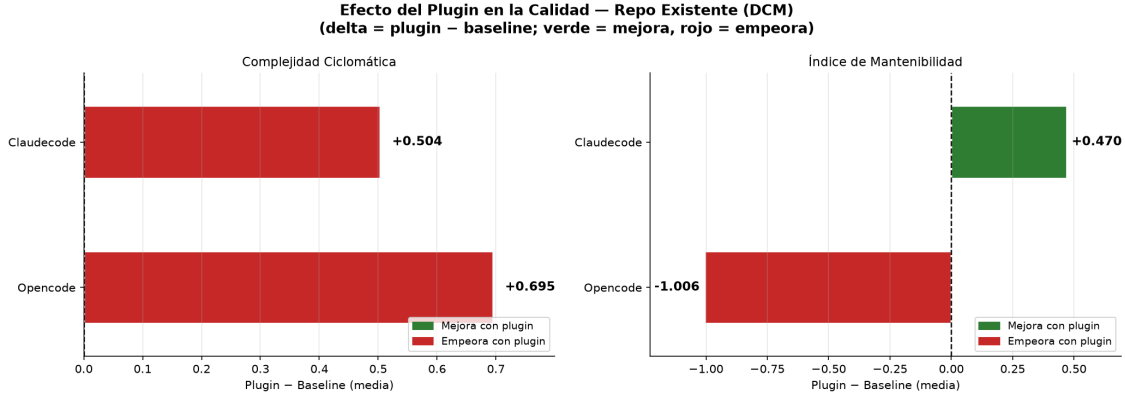


Figura 22: Efecto de colbPowers sobre la calidad del código en el Caso 1 (delta = colbPowers – baseline; verde: mejora, rojo: empeora).

Caso 1 — OpenCode vs. ClaudeCode. En el Caso 1 las diferencias de calidad entre asistentes son menos pronunciadas que en el Caso 2 y ninguna alcanza significación estadística. En baseline, la complejidad ciclomática es similar entre asistentes ($p = 0,129$, $g = -1,26$) y el índice de mantenibilidad también ($p = 0,596$, $g = +0,38$). Con colbPowers la brecha es igualmente irrelevante estadísticamente (complejidad: $p = 0,746$, $g = -0,23$; mantenibilidad: $p = 0,979$, $g = +0,02$), lo que sugiere que en tareas de modificación incremental la elección del asistente tiene menor impacto en la calidad estructural del código producido.

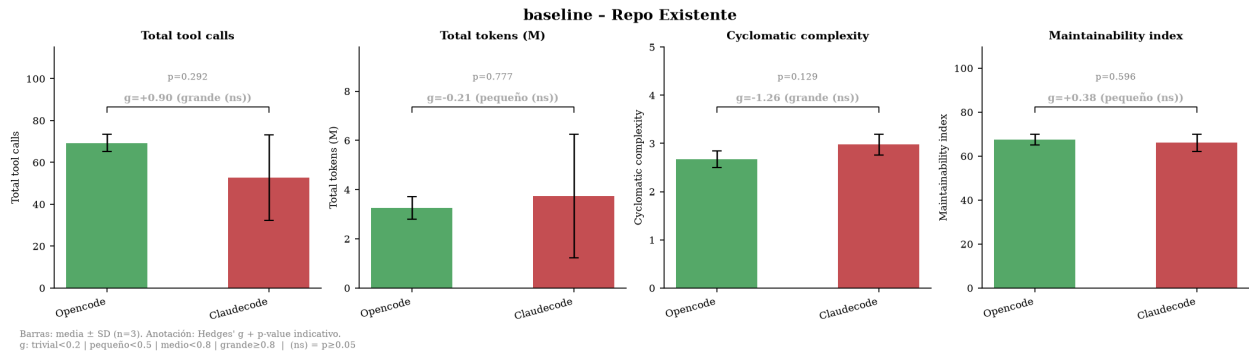


Figura 23: OpenCode vs. ClaudeCode, baseline, Caso 1. Barras: media ± SD. Anotación: g de Hedges y p -value indicativo.

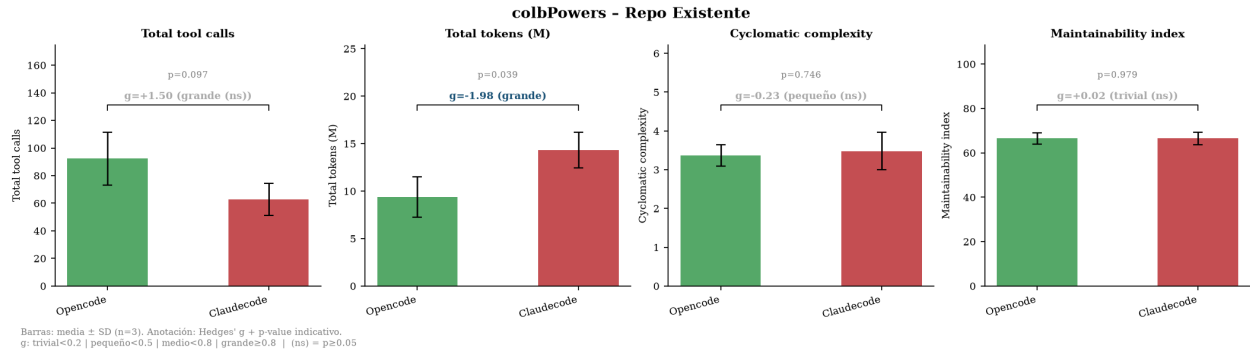


Figura 24: OpenCode vs. ClaudeCode, colbPowers, Caso 1. Barras: media $\pm$ SD. Anotación: g de Hedges y p -value indicativo.

Las tablas completas del análisis estadístico se recogen en el Anexo A.

8. Conclusiones

8.1. Conclusiones generales

Este trabajo ha abordado un problema relevante en el desarrollo de software asistido por inteligencia artificial: la tendencia del paradigma *Code-Driven* a generar código de baja calidad estructural cuando no existe una fase previa de especificación y planificación. La propuesta central del proyecto, el plugin colbPowers y el flujo *Spec-Driven Development* (SDD), ha demostrado ser técnicamente viable y evaluable dentro del entorno de trabajo de *Colb.AI*, donde actualmente se utiliza este sistema.

Cumplimiento de requisitos

Se van a mostrar los requisitos funcionales y no funcionales definidos anteriormente en 2.2.1 y 2.2.2, se iterará sobre cada uno de estos justificando si se han conseguido cubrir adecuadamente.

Requisitos funcionales

El sistema debe permitir la ingesta y gestión de reglas fundamentales del proyecto (pila tecnológica, estándares de código) y mantener un registro actualizado de las características a desarrollar. Este requisito queda cubierto mediante los archivos `constitution.md` y `features.md`, que actúan como fuente de verdad del proyecto. `constitution.md` centraliza la pila tecnológica y los estándares de código del proyecto; `features.md` mantiene el registro actualizado de funcionalidades pendientes, en progreso y completadas. Adicionalmente, los documentos `plan.md` y `spec.md` generados durante el flujo SDD permiten consultar qué apartados han sido ya implementados y de qué forma concreta se llevó a cabo su implementación. Todos estos archivos se actualizan de forma iterativa durante el flujo SDD y persisten versionados en el repositorio Git del proyecto.

El sistema debe ser capaz de interpretar funcionalidades de alto nivel y desglosarlas en un plan de tareas granulares y estructuradas. Este requisito se implementa en la fase de planificación del flujo SDD, donde el agente orquestador genera un plan de tareas granulares (`spec.md` y `plan.md`) a partir de una funcionalidad de alto nivel definida en por el usuario, antes de delegar la implementación a los subagentes especializados. Este mecanismo ha sido validado en los dos casos experimentales del Capítulo 7.

El sistema debe incluir un mecanismo de auditoría autónoma que contraste el código fuente generado con las especificaciones documentadas, para ver si existe algún conflicto con las documentación y el código generado. Este requisito queda cubierto por el subagente revisor de coherencia con especificaciones, que contrasta el código generado con los documentos de contexto (`constitution.md`, `features.md`) e identifica posibles conflictos o desviaciones. Como se discute en las limitaciones, este mecanismo es no determinista, puesto que está basado en *prompts*, y en uno de los experimentos el orquestador omitió su invocación, lo que representa la principal limitación en el cumplimiento de este requisito.

El sistema debe incluir un módulo de procesamiento de transcripciones capaz de generar planes de implementación estructurados a partir de reuniones con el cliente, explorando automáticamente los *frameworks* y tecnologías relevantes mediante búsqueda semántica en la web. Este requisito queda cubierto por el módulo de Toma de Requerimientos (Sección 5.1), que procesa la transcripción de una reunión inicial con el cliente y genera un plan de implementación estructurado. El módulo incluye búsqueda web automática para explorar *frameworks* y tecnologías relevantes según el contexto del proyecto descrito en la reunión.

El sistema debe incluir un módulo de generación de documentación de seguimiento post-reunión con el formato específico de *Colb.AI*, a partir de la transcripción de reuniones periódicas con el cliente. Este requisito queda cubierto por el módulo de Seguimiento del Cliente (Sección 5.2), que genera documentación post-reunión en el formato específico de *Colb.AI* a partir de la transcripción de reuniones periódicas. El módulo produce los artefactos de seguimiento utilizados actualmente en los proyectos de la empresa.

Requisitos no funcionales

Transparencia y Trazabilidad: Todo el estado, la memoria y las normativas manejadas por los agentes deben persistir en formatos de texto en un sistema de control de versiones. Todos los artefactos del sistema —`constitution.md`, `features.md`, los documentos generados por los módulos de transcripción y los propios *prompts* y *skills*— son archivos de texto plano versionados en Git. Esto garantiza que cualquier cambio en el estado del sistema sea auditable y reversible, cumpliendo plenamente este requisito.

Integración con asistentes CLI: El plugin debe ser compatible con ClaudeCode y OpenCode, utilizando sus mecanismos nativos de extensibilidad (MCP, hooks, skills) para actuar directamente sobre los archivos locales del proyecto. El plugin colbPowers ha sido validado experimentalmente sobre los dos asistentes CLI objetivo: ClaudeCode y OpenCode. Los experimentos del Capítulo 7 confirman la compatibilidad efectiva con ambos asistentes.

Modularidad: El diseño de los roles (*prompts* y *skills*) debe ser lo suficientemente modular para permitir futuras expansiones. El sistema está estructurado en componentes independientes: subagentes especializados (implementador, revisor de código, revisor de coherencia), módulos de transcripción y documentación, y herramientas de apoyo como *code-review-graph*. Esta separación de responsabilidades permite añadir, modificar o sustituir componentes sin afectar al resto del sistema. Las líneas de trabajo futuro descritas en la sección siguiente ilustran la viabilidad de extensiones concretas sobre esta arquitectura modular.

Reducción del *vendor lock-in*. Aunque este aspecto no fue formulado explícitamente como requisito no funcional, sino como un riesgo identificado en la Sección 2.4, merece destacarse que ha sido abordado de forma muy satisfactoria a lo largo del proyecto. Para cada módulo o componente con dependencia de una plataforma comercial se ha desarrollado una alternativa propia o de código abierto: los módulos de transcripción disponen de implementaciones independientes de Claude Cowork; el flujo SDD puede ejecutarse tanto sobre ClaudeCode como sobre OpenCode; y

las herramientas de análisis estático empleadas (radon, DCM) son de código abierto. Esta cobertura garantiza que el sistema pueda desplegarse y mantenerse sin quedar a la merced de un único proveedor externo.

Resultados experimentales

La evaluación experimental produce resultados mixtos. Dado el tamaño muestral ($n = 3$ por condición), se emplea la g de Hedges como estimador principal del tamaño del efecto; el p -value del test t de Welch indica si puede rechazarse la hipótesis nula de ausencia de efecto, y la g cuantifica su magnitud solo cuando el p -value permite ese rechazo. El hallazgo principal es la mejora estadísticamente significativa y de efecto grande en las métricas de calidad del código en el Caso 2 (proyecto generado desde cero): colbPowers reduce la complejidad ciclomática un 27 % y mejora el índice de mantenibilidad un 15 % en OpenCode ($p = 0,014$ y $p = 0,037$ respectivamente). Con ClaudeCode, que de base produce código de mayor calidad, los cambios fueron mucho menos notables —un aumento del 9 % en complejidad ciclomática y una variación del 3 % en mantenibilidad— y no alcanzan significación estadística ($p = 0,508$ y $p = 0,754$), por lo que este test no ha podido rechazar la hipótesis nula; el efecto podría existir sin ser detectable con $n = 3$. En el Caso 1 (eliminación de funcionalidades complejas sobre un repositorio existente), el patrón se invierte: colbPowers produce código de mayor complejidad ciclomática (+26 % en OpenCode, $p = 0,029$, $g = +2,42$; +17 % en ClaudeCode, $p = 0,208$, $g = +1,07$), con efecto estadísticamente significativo solo en OpenCode, sin mejora neta en mantenibilidad en ningún asistente (OpenCode: $p = 0,646$; ClaudeCode: $p = 0,873$). Uno de los facores destacables es que parece ser que los agentes no invocaron *code-review-graph* de forma efectiva, privando al flujo SDD del contexto de dependencias.

Un resultado secundario con implicaciones prácticas directas es la convergencia de calidad entre asistentes: con colbPowers activo, las diferencias de calidad entre OpenCode y ClaudeCode no alcanzan significación estadística, y la g de Hedges cae a valores próximos a cero en ambas métricas, lo que indica que el flujo SDD homogeneiza la calidad del código con independencia del asistente y permite a un equipo combinar herramientas distintas sin comprometer la consistencia del producto.

El coste del sistema es real y cuantificado: colbPowers multiplica el consumo de tokens por un factor de entre 2,2x y 4,5x respecto al baseline, con tamaños de efecto grandes en todas las combinaciones evaluadas. Este sobrecoste es constante e independiente del resultado: se produce en todos los experimentos, pero la mejora de calidad que lo justificaría solo ha podido confirmarse estadísticamente en el Caso 2 con OpenCode. En el Caso 1, el overhead no se tradujo en mejoras de calidad, lo que indica que la relación entre coste computacional y beneficio estructural depende del tipo de tarea abordada.

Limitaciones

El diseño experimental con $n = 3$ repeticiones por condición tiene potencia estadística limitada e impide verificar los supuestos de normalidad requeridos por el test t de Welch. Los resultados deben considerarse evidencia sugerente, no concluyente, y deben ser confirmados con tamaños muestrales mayores ($n \geq 5$) en trabajos futuros. Adicionalmente, las herramientas de análisis estático empleadas difieren entre casos (radon para Python y DCM para Flutter/Dart), lo que implica que las métricas de calidad no son directamente comparables entre experimentos, aunque sí lo son dentro de cada caso entre condiciones. Por último, el mecanismo de garantía del flujo SDD es no determinista, en un experimento el agente orquestador omitió la invocación del subagente revisor.

Valoración global

Los resultados respecto a la hipótesis principal —que estructurar el desarrollo mediante un flujo SDD mejora la calidad del código generado— son mixtos: el flujo mostró mejoras estadísticamente significativas únicamente en el Caso 2 con OpenCode, mientras que en el Caso 1 produjo un aumento de la complejidad ciclomática y no aportó mejoras en mantenibilidad en ningún asistente. Estos resultados, aun sin confirmar la hipótesis de forma general, justifican continuar investigando el enfoque, ampliar el tamaño muestral y extender las métricas de evaluación a otros lenguajes y tipos de proyecto. La contribución técnica del TFG (un plugin funcional, dos módulos de apoyo y un conjunto de datos experimentales cuantificables sobre el efecto del flujo SDD) proporciona una base concreta para este trabajo futuro.

Cabe señalar, no obstante, que los resultados experimentales mixtos contrastan con la experiencia de uso real del sistema en *Colb.AI*, donde la valoración ha sido positiva desde su adopción. Esta divergencia puede explicarse por dos factores no controlados en el experimento: por un lado, los modelos empleados en producción son en general más capaces que los utilizados en las ejecuciones experimentales; por otro, es posible que el entorno de experimentación no haya replicado con suficiente fidelidad las condiciones reales de uso, en particular la calidad y exhaustividad de los documentos de contexto proporcionados. Ambas hipótesis presentan líneas de análisis relevantes para trabajos futuros.

8.2. Trabajo futuro

A continuación se describen las líneas de mejora y expansión identificadas durante el desarrollo del proyecto, organizadas por componente del sistema.

8.2.1. Plugin colbPowers y flujo SDD

Refuerzo determinista del flujo mediante hooks. El mecanismo actual para asegurar que el agente orquestador siga el flujo SDD se basa exclusivamente en el prompt de instrucciones, lo que es inherentemente no determinista: en los experimentos se observaron casos en los que el orquestador omitió la llamada al subagente revisor tras implementar código. Una mejora natural sería reemplazar parte de esta responsabilidad por hooks de ciclo de vida que comprueben el estado del flujo en puntos clave y bloqueen el avance si no se han completado los pasos requeridos. Esto transformaría una restricción de comportamiento en una restricción de ejecución, más robusta y reproducible.

Mayor presencia de code-review-graph en los archivos de contexto. Los experimentos mostraron que el servidor MCP *code-review-graph* fue prácticamente ignorado por los agentes: las llamadas registradas fueron esporádicas y en su mayoría no iniciadas por el orquestador. Una causa probable es que el orquestador no transmitía a los subagentes información sobre la disponibilidad y el propósito del servidor. Añadir instrucciones explícitas en los archivos de contexto del agente — indicando cuándo y cómo utilizar el grafo de dependencias— podría incrementar significativamente su adopción y, con ello, la calidad del análisis de impacto previo a cada cambio.

Experimentos adicionales. Los experimentos realizados requieren supervisión manual continua, lo que limitó el tamaño muestral a $n = 3$ por condición. Con más recursos, se proponen las siguientes comparaciones:

- **colbPowers con y sin code-review-graph**, para aislar el efecto del servidor MCP de forma independiente al resto del flujo. Aun que se ha observado que el servidor fue poco utilizado, un experimento controlado podría cuantificar su impacto en la calidad del código y en el consumo de tokens.
- **colbPowers vs a superpowers**, su plugin predecesor, para cuantificar el valor añadido de las extensiones introducidas en este proyecto.
- **Ejecución inline vs a subagent-driven**, para evaluar si el overhead de tokens de la ejecución por subagentes se traduce en mejoras de calidad consistentes respecto a que el orquestador resuelva las tareas directamente.
- **Efecto del modelo subyacente sobre los resultados**, para contrastar si la divergencia entre los resultados experimentales y la experiencia de uso real en *Colb.AI* se debe a la mayor capacidad de los modelos empleados en producción. Esto requeriría repetir los experimentos con los mismos modelos utilizados en el entorno real y comparar los efectos obtenidos.
- **Validación del entorno experimental**, para evaluar en qué medida las condiciones del experimento replican fielmente el uso real del sistema. En particular, convendría analizar si la calidad y exhaustividad de los documentos de contexto (*constitution.md*, *features.md*) proporcionados en los experimentos es comparable a la de los proyectos reales de la empresa, dado que la efectividad del flujo SDD depende directamente de la calidad de dichos documentos.

En todos los casos se recomendaría aumentar el número de iteraciones a $n \geq 5$ para poder aplicar tests no paramétricos (que con $n = 3$ no pueden alcanzar $p < 0,05$) y obtener estimaciones más robustas del tamaño de efecto.

8.2.2. Módulos de transcripción y documentación

Memoria acumulativa entre reuniones. Actualmente, tanto el módulo de toma de requerimientos como el de seguimiento del cliente procesan cada reunión de forma completamente independiente, sin acceso al historial de sesiones anteriores. Esto obliga al equipo a proporcionar manualmente el contexto acumulado en cada nueva ejecución. Una mejora sustancial consistiría en incorporar un almacén persistente —por ejemplo, una base de datos vectorial— que indexe los documentos generados en reuniones previas y permita al modelo recuperar contexto relevante mediante búsqueda semántica. Esto haría los documentos generados más coherentes con la evolución del proyecto a lo largo del tiempo.

Base de datos RAG para estimaciones de presupuesto y horas. El módulo de toma de requerimientos genera estimaciones de tiempo y coste por funcionalidad basándose únicamente en el razonamiento del modelo, sin datos históricos de referencia. Incorporar una base de datos de recuperación aumentada (*Retrieval-Augmented Generation*) con ejemplos reales de proyectos anteriores —incluyendo desglose de horas por tipo de tarea y presupuestos asociados— permitiría al módulo fundamentar sus estimaciones en precedentes concretos, aumentando su fiabilidad y utilidad práctica para la planificación de proyectos.

Integración con herramientas de gestión de proyectos. El documento de seguimiento generado actualmente es un artefacto estático en formato Word. Un paso natural sería integrarlo con plataformas de gestión de proyectos como Linear o Jira, de modo que las tareas identificadas en la reunión se conviertan automáticamente en tickets asignados, con descripción, estimación y responsable. Esto cerraría el ciclo entre la reunión con el cliente y el flujo de trabajo del equipo de desarrollo sin intervención manual.

Reducción de la dependencia de Claude Cowork. Ambos módulos disponen de una implementación propia de código abierto como alternativa a la skill de Claude Cowork, pero esta alternativa no cubre todos los casos de uso de la plataforma comercial. Consolidar y extender las implementaciones propias reduciría la dependencia de una suscripción externa y facilitaría el despliegue del sistema en entornos con requisitos de privacidad o control de costes estrictos.

Referencias

- [1] AgentSkills. Agentskills documentation. <https://agentskills.io/home>, 2026. Accedido el 25/02/2026.
- [2] Anthropic. Claude code: Agentic coding in your terminal. <https://docs.anthropic.com/en/docs/claude-code>, 2024. Accedido el 29/04/2026.
- [3] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Accedido el 20/06/2026.
- [4] Anthropic. Claude cowork. <https://claude.com/product/cowork>, 2025. Accedido el 02/06/2026.
- [5] Anthropic. Claude haiku 4.5. <https://www.anthropic.com/claude/haiku>, 2025. Accedido el 18/06/2026.
- [6] Anthropic. Choosing a claude plan: Free, pro, and team. <https://support.claude.com/en/articles/11049762-choosing-a-claude-plan>, 2026. Accedido el 09/03/2026.
- [7] Barcelona Life. Guide to coworking spaces in barcelona: Shared offices. <https://www.barcelona-life.com/barcelona-coworking-spaces>, 2026. Accedido el 10/03/2026.
- [8] Builder Methods. buildermethods/agent-os. <https://github.com/buildermethods/agent-os>, 2025. Accedido el 25/02/2026.
- [9] Adrià Cebrián Ruiz. colbpowers: Spec-first plugin for Claude Code and OpenCode. <https://github.com/Adria-Colbai/colbPowers>, 2026. Accedido el 14/05/2026.
- [10] Harrison Chase. LangChain. <https://github.com/langchain-ai/langchain>, 2022. Accedido el 20/06/2026.
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code, 2021. arXiv:2107.03374, doi:10.48550/arXiv.2107.03374.
- [12] code-review-graph. Repositorio code-review-graph. <https://github.com/nicholasgriffintn/llm-code-review-graph>, 2025. Accedido el 29/04/2026.
- [13] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988. URL: <https://utstat.toronto.edu/~brunner/oldclass/378f16/readings/CohenPower.pdf>.
- [14] Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications*, OOPSLA '92. ACM, 1992. doi:10.1145/157709.157715.
- [15] Cursor. Cursor: The best way to code with ai. <https://cursor.com>, 2026. Accedido el 25/02/2026.
- [16] DCM Team. Dcm: Dart code metrics — static analysis tool for dart and flutter. <https://dcm.dev>, 2024. Accedido el 18/06/2026.
- [17] DeepSeek. Deepseek v4-flash. <https://api-docs.deepseek.com/news/news260424>, 2025. Accedido el 18/06/2026.
- [18] Departament d'Organització d'Empreses (FIB). Mòdul 2.1.b – gestión ágil de proyectos. Universitat Politècnica de Catalunya. Accedido el 24/02/2026.

- [19] Fireflies.ai. Fireflies.ai – AI notetaker for meetings. <https://fireflies.ai>, 2026. Accedido el 05/05/2026.
- [20] GitHub. github/spec-kit repository. <https://github.com/github/spec-kit>, 2025. Accedido el 24/02/2026.
- [21] GitHub. Github copilot documentation. <https://docs.github.com/en/copilot>, 2026. Accedido el 25/02/2026.
- [22] GitHub. Github pricing plans: Personal and business. <https://github.com/pricing>, 2026. Accedido el 09/03/2026.
- [23] Maurice H. Halstead. *Elements of Software Science*. Elsevier North-Holland, 1977. URL: <https://api.semanticscholar.org/CorpusID:61365155>.
- [24] Larry V. Hedges. Distribution theory for glass’s estimator of effect size and related estimators. *Journal of Educational Statistics*, 6(2):107–128, 1981. URL: <http://www.jstor.org/stable/1164588>.
- [25] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL: <https://arxiv.org/abs/2308.00352>.
- [26] Indeed. Guía de salarios en españa: Explorar sueldos por cargo y ubicación. <https://es.indeed.com/career/salaries>, 2026. Accedido el 08/03/2026.
- [27] Michael Inzlicht, Amitai Shenhav, and Christopher Y. Olivola. The effort paradox: Effort is both costly and valued. *Trends in Cognitive Sciences*, 22(4):337–349, 2018. doi:10.1016/j.tics.2018.01.007.
- [28] Jama Software. What is requirements gathering in software engineering? <https://www.jamasoftware.com/requirements-management-guide/requirements-gathering-and-management-processes/what-is-requirements-gathering/>, 2026. Accedido el 25/02/2026.
- [29] JetBrains. Developer ecosystem survey 2025: Artificial intelligence. <https://devecosystem-2025.jetbrains.com/artificial-intelligence>, 2025. Accedido el 20/06/2026.
- [30] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology*, 33(5), 2024. URL: <https://arxiv.org/abs/2303.06689>, doi: 10.1145/3672459.
- [31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL: <https://arxiv.org/abs/2310.06770>.
- [32] Wouter Kool, Joseph T. McGuire, Zev B. Rosen, and Matthew M. Botvinick. Decision making and the avoidance of cognitive demand. *Journal of Experimental Psychology: General*, 139(4):665–682, 2010. doi:10.1037/a0020198.
- [33] Michele Lacchia. radon: Various code metrics for python code. URL: <https://github.com/rubik/radon>.

- [34] T.J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976. doi:10.1109/TSE.1976.233837.
- [35] Raúl Medina Tamayo. Propiedad intelectual del contenido generado por IA: guía práctica para emprendedores, freelance y empresas innovadoras. UCAEmprende – Servicio de Apoyo al Emprendimiento, Universidad de Cádiz, 2025. Accedido el 20/05/2026. URL: <https://emprende.uca.es/propiedad-intelectual-del-contenido-generado-por-ia-guia-practica-para-emprendedores-freeel>
- [36] Microsoft. Azure AI speech service. <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/>, 2024. Accedido el 05/05/2026.
- [37] Microsoft. Code metrics values: Maintainability index. <https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-values?view=visualstudio>, 2024. Accedido el 17/03/2026.
- [38] Microsoft. Using agents in visual studio code. <https://code.visualstudio.com/docs/copilot/agents/overview>, 2026. Accedido el 25/02/2026.
- [39] Naturgy. Precio de la luz y tarifas eléctricas para el hogar. https://www.naturgy.es/hogar/luz/precio_de_la_luz, 2026. Accedido el 10/03/2026.
- [40] P. Oman and J. Hagemester. Metrics for assessing a software system’s maintainability. In *Proceedings Conference on Software Maintenance*, 1992. doi:10.1109/ICSM.1992.242525.
- [41] Justice Opara-Martins, Reza Sahandi, and Feng Tian. Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective. *Journal of Cloud Computing: Advances, Systems and Applications*, 5(1):1–23, 2016. doi:10.1186/s13677-016-0054-z.
- [42] OpenCode. Opencode: Open source coding assistant. <https://github.com/sst/opencode>, 2025. Accedido el 29/04/2026.
- [43] J. Ostroff, D. Makalsky, and R. Paige. Agile specification-driven development. In *Extreme Programming and Agile Methods*. Springer, 2004. doi:10.1007/978-3-540-24853-8_12.
- [44] Otter.ai. Otter.ai – AI meeting assistant. <https://otter.ai>, 2026. Accedido el 05/05/2026.
- [45] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689 del parlamento europeo y del consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). Diario Oficial de la Unión Europea, L 2024/1689, julio 2024. Accedido el 20/05/2026. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>.
- [46] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768, 2022. URL: <https://arxiv.org/abs/2108.09293>, doi:10.1109/SP46214.2022.9833571.
- [47] Sida Peng, Eirini Kalliamvakou, et al. The impact of ai on developer productivity: Evidence from github copilot, 2023. arXiv:2302.06590, doi:10.48550/arXiv.2302.06590.
- [48] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, et al. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, 2024. URL: <https://arxiv.org/abs/2307.07924>.

- [49] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavy, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022. URL: <https://arxiv.org/abs/2212.04356>.
- [50] Mario Rodriguez. Updates to GitHub Copilot interaction data usage policy. The GitHub Blog, GitHub Inc., marzo 2026. Accedido el 20/05/2026. URL: <https://github.blog/news-insights/company-news/updates-to-github-copilot-interaction-data-usage-policy/>.
- [51] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum: The rules of the game. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>, 2020. Accedido el 17/03/2026.
- [52] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015. ISBN: 978-0133943030.
- [53] Superpowers. Superpowers plugin for Claude Code. <https://github.com/superpowers-ai/superpowers>, 2025. Accedido el 29/04/2026.
- [54] Tavily. Tavily AI search API. <https://tavily.com>, 2025. Accedido el 29/04/2026.
- [55] The Standish Group. The CHAOS report. Technical report, The Standish Group International, 1994. URL: https://personal.utdallas.edu/~chung/SYSM6309/chaos_report.pdf.
- [56] Tree-sitter Contributors. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>, 2018. Accedido el 20/06/2026.
- [57] Tien Rahayu Tulili, Andrea Capiluppi, and Ayushi Rastogi. Burnout in software engineering: A systematic mapping study. *Information and Software Technology*, 155:107116, 2023. doi: 10.1016/j.infsof.2022.107116.
- [58] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, 2022. doi:10.1145/3491101.3519665.
- [59] Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, 2024. doi:10.48550/arXiv.2308.11432.
- [60] Bernard Lewis Welch. The generalization of “Student’s” problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947. doi:10.1093/biomet/34.1–2.28.
- [61] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*, 2023. doi: 10.48550/arXiv.2210.03629.

A. Tablas completas análisis estadístico

colbPowers vs baseline — summary table

Comparison	Metric	Mean colbPowers	Mean baseline	t	p	Hedges' g	Efecto
Claudecode - ClassificationVisu	Total tool calls	48.33	62.67	-0.997	0.4141	-0.65	medio (ns)
Claudecode - ClassificationVisu	Total tokens (M)	8.19M	3.74M	2.390	0.0772	+1.56	grande (ns)
Claudecode - ClassificationVisu	Cyclomatic complexity	3.11	2.85	0.732	0.5075	+0.48	pequeño (ns)
Claudecode - ClassificationVisu	Maintainability index	59.01	56.97	0.337	0.7544	+0.22	pequeño (ns)
Claudecode - Repo Existente	Total tool calls	62.67	52.67	0.738	0.5115	+0.48	pequeño (ns)
Claudecode - Repo Existente	Total tokens (M)	14.33M	3.74M	5.839	0.0054	+3.81	grande
Claudecode - Repo Existente	Cyclomatic complexity	3.48	2.98	1.638	0.2080	+1.07	grande (ns)
Claudecode - Repo Existente	Maintainability index	66.52	66.05	0.171	0.8734	+0.11	trivial (ns)
Opencode - ClassificationVisu	Total tool calls	85.00	51.67	3.251	0.0694	+2.12	grande (ns)
Opencode - ClassificationVisu	Total tokens (M)	8.41M	1.88M	5.871	0.0187	+3.84	grande
Opencode - ClassificationVisu	Cyclomatic complexity	3.13	4.31	-5.496	0.0145	-3.59	grande
Opencode - ClassificationVisu	Maintainability index	47.78	41.54	4.225	0.0371	+2.76	grande
Opencode - Repo Existente	Total tool calls	92.33	69.33	2.034	0.1679	+1.33	grande (ns)
Opencode - Repo Existente	Total tokens (M)	9.38M	3.26M	4.918	0.0323	+3.21	grande
Opencode - Repo Existente	Cyclomatic complexity	3.37	2.67	3.704	0.0289	+2.42	grande
Opencode - Repo Existente	Maintainability index	66.59	67.59	-0.497	0.6457	-0.32	pequeño (ns)

n=3 por grupo. Los p-values son indicativos (Welch). Hedges's g es la métrica principal de tamaño de efecto.
g: trivial<0.2 | pequeño<0.5 | medio<0.8 | grande≥0.8

Figura 25: Análisis estadístico: colbPowers vs. baseline. Un $p < 0,05$ permite rechazar la hipótesis nula; un p alto no descarta el efecto, solo que el test no lo ha detectado. La g de Hedges cuantifica el tamaño del efecto.

Opencode vs Claudecode — summary table

Comparison	Metric	Mean Opencode	Mean Claudecode	t	p	Hedges' g	Efecto
baseline - ClassificationVisu	Total tool calls	51.67	62.67	-0.642	0.5592	-0.42	pequeño (ns)
baseline - ClassificationVisu	Total tokens (M)	1.88M	3.74M	-1.510	0.2549	-0.99	grande (ns)
baseline - ClassificationVisu	Cyclomatic complexity	4.31	2.85	5.071	0.0072	+3.31	grande
baseline - ClassificationVisu	Maintainability index	41.54	56.97	-3.929	0.0386	-2.57	grande
baseline - Repo Existente	Total tool calls	69.33	52.67	1.385	0.2915	+0.90	grande (ns)
baseline - Repo Existente	Total tokens (M)	3.26M	3.74M	-0.321	0.7767	-0.21	pequeño (ns)
baseline - Repo Existente	Cyclomatic complexity	2.67	2.98	-1.934	0.1291	-1.26	grande (ns)
baseline - Repo Existente	Maintainability index	67.59	66.05	0.585	0.5955	+0.38	pequeño (ns)
colbPowers - ClassificationVisu	Total tool calls	85.00	48.33	8.538	0.0013	+5.58	grande
colbPowers - ClassificationVisu	Total tokens (M)	8.41M	8.19M	0.124	0.9081	+0.08	trivial (ns)
colbPowers - ClassificationVisu	Cyclomatic complexity	3.13	3.11	0.094	0.9320	+0.06	trivial (ns)
colbPowers - ClassificationVisu	Maintainability index	47.78	59.01	-2.330	0.1430	-1.52	grande (ns)
colbPowers - Repo Existente	Total tool calls	92.33	62.67	2.299	0.0975	+1.50	grande (ns)
colbPowers - Repo Existente	Total tokens (M)	9.38M	14.33M	-3.038	0.0391	-1.98	grande
colbPowers - Repo Existente	Cyclomatic complexity	3.37	3.48	-0.353	0.7463	-0.23	pequeño (ns)
colbPowers - Repo Existente	Maintainability index	66.59	66.52	0.028	0.9793	+0.02	trivial (ns)

n=3 por grupo. Los p-values son indicativos (Welch). Hedges's g es la métrica principal de tamaño de efecto.
g: trivial<0.2 | pequeño<0.5 | medio<0.8 | grande≥0.8

Figura 26: Análisis estadístico: OpenCode vs. ClaudeCode. Un $p < 0,05$ permite rechazar la hipótesis nula; un p alto no descarta el efecto, solo que el test no lo ha detectado. La g de Hedges cuantifica el tamaño del efecto.

B. Gráficas adicionales del experimento

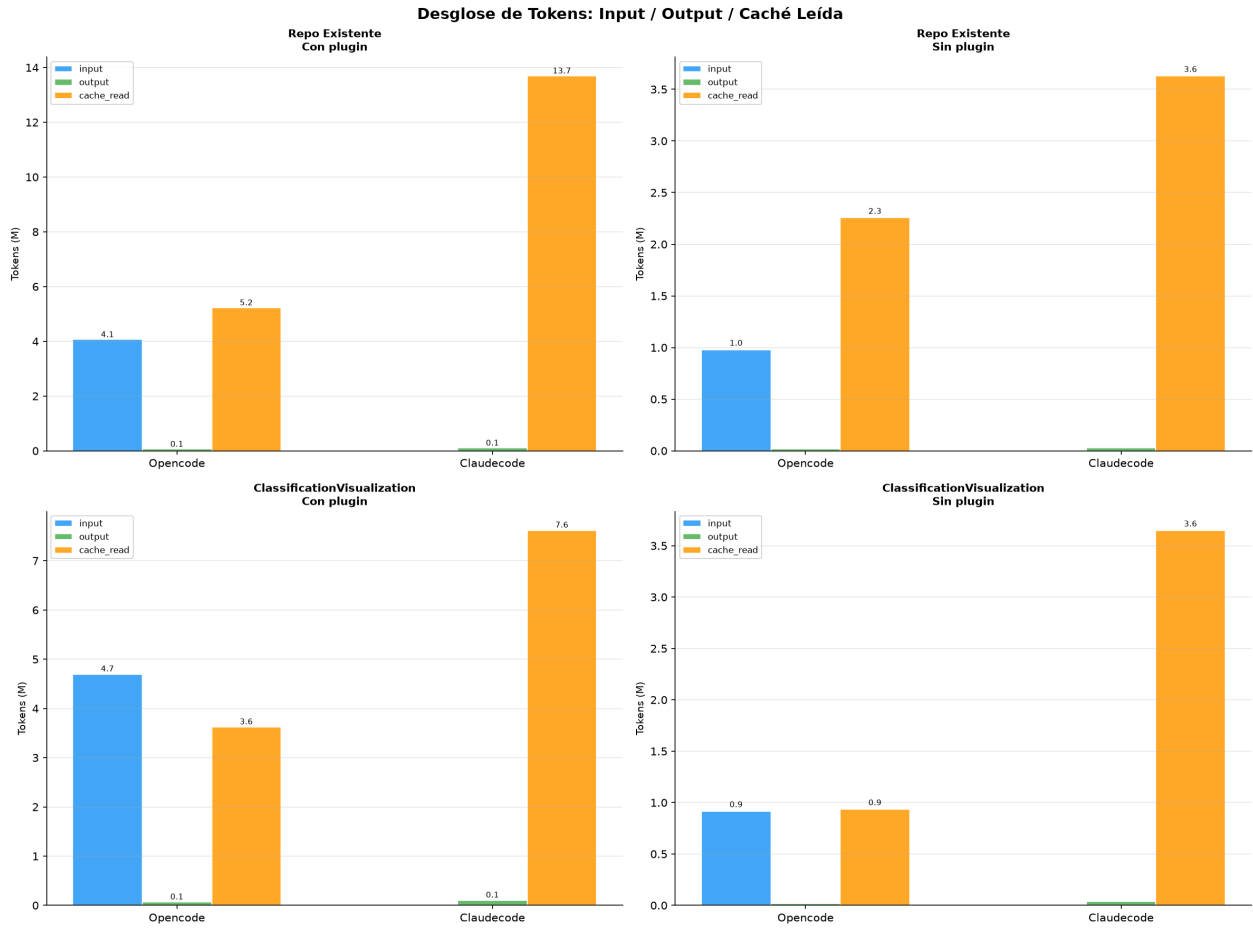


Figura 27: Desglose de tokens por tipo (input, output, caché leída) para cada experimento y condición.

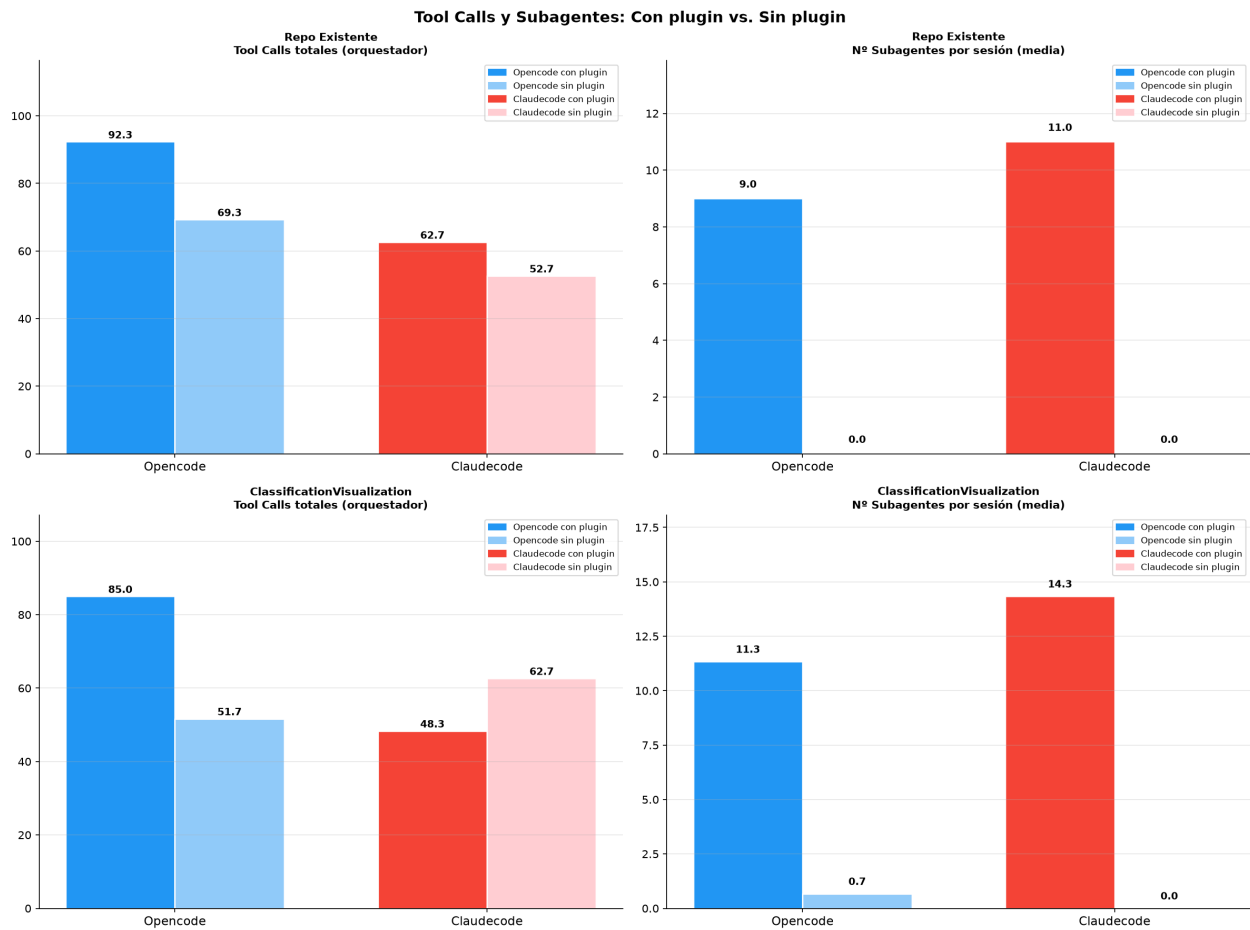


Figura 28: Tool calls totales del orquestador y número de subagentes por sesión, por condición y experimento.

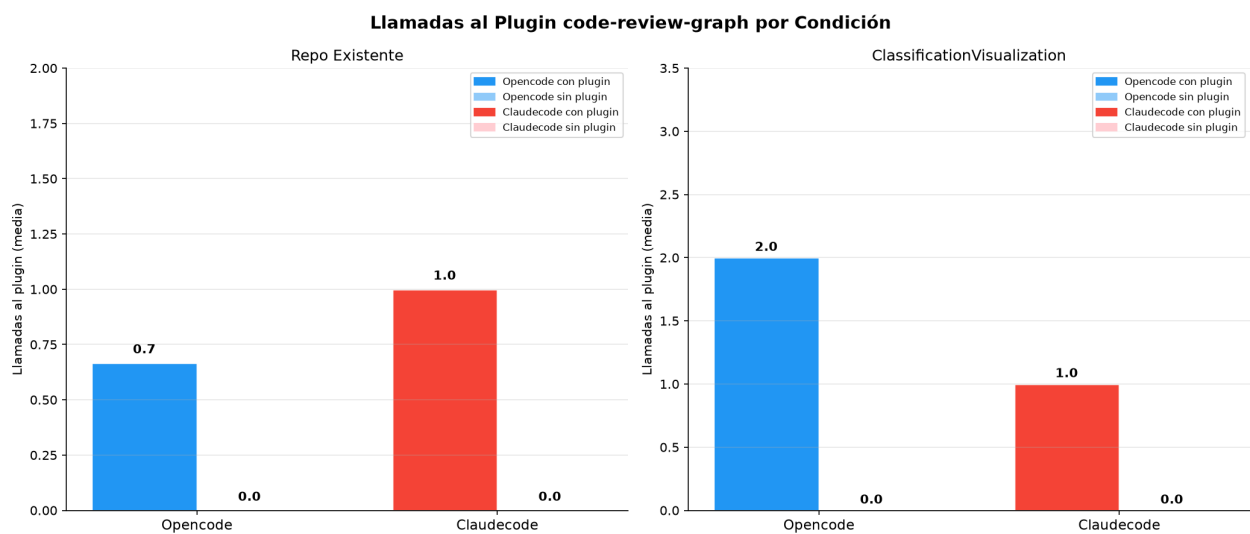


Figura 29: Llamadas al plugin code-review-graph por condición y experimento.

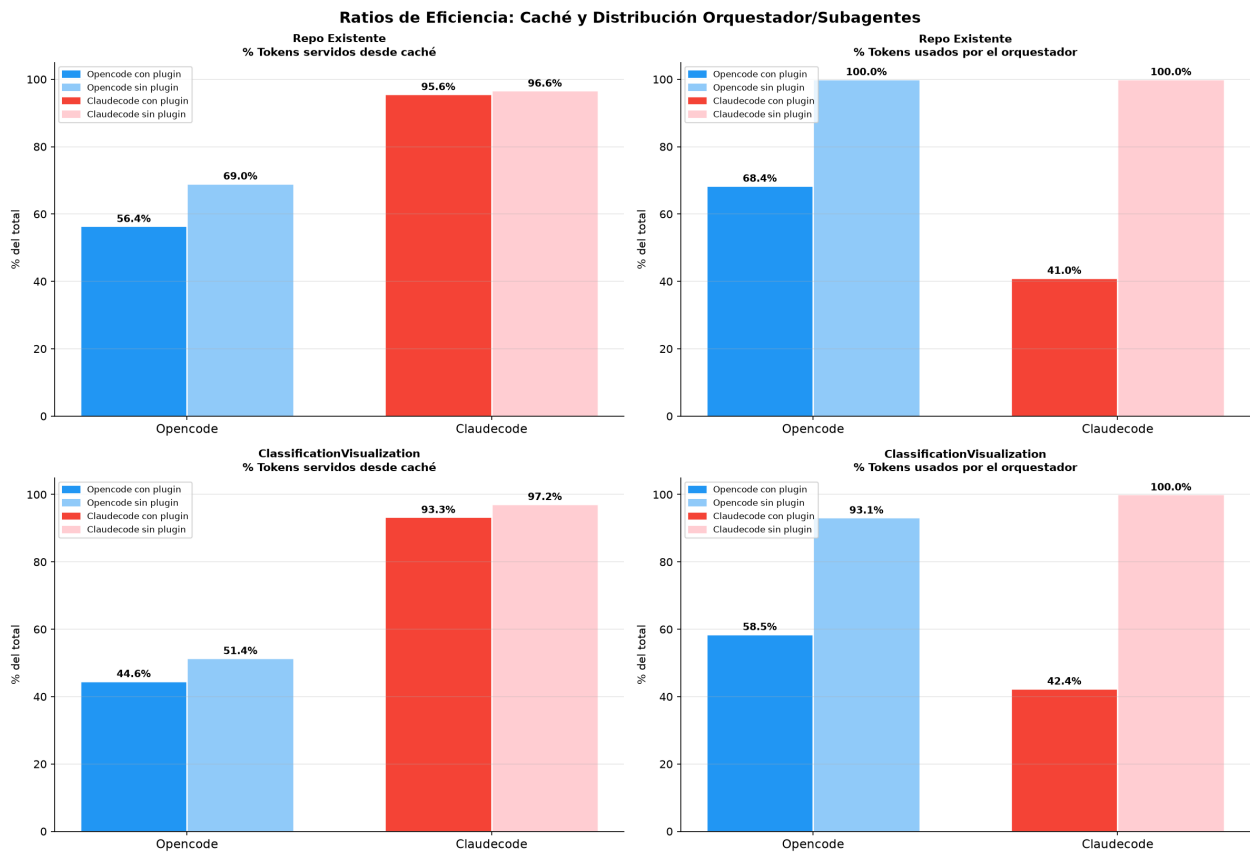


Figura 30: Ratios de eficiencia: porcentaje de tokens servidos desde caché y porcentaje de tokens consumidos por el orquestador.

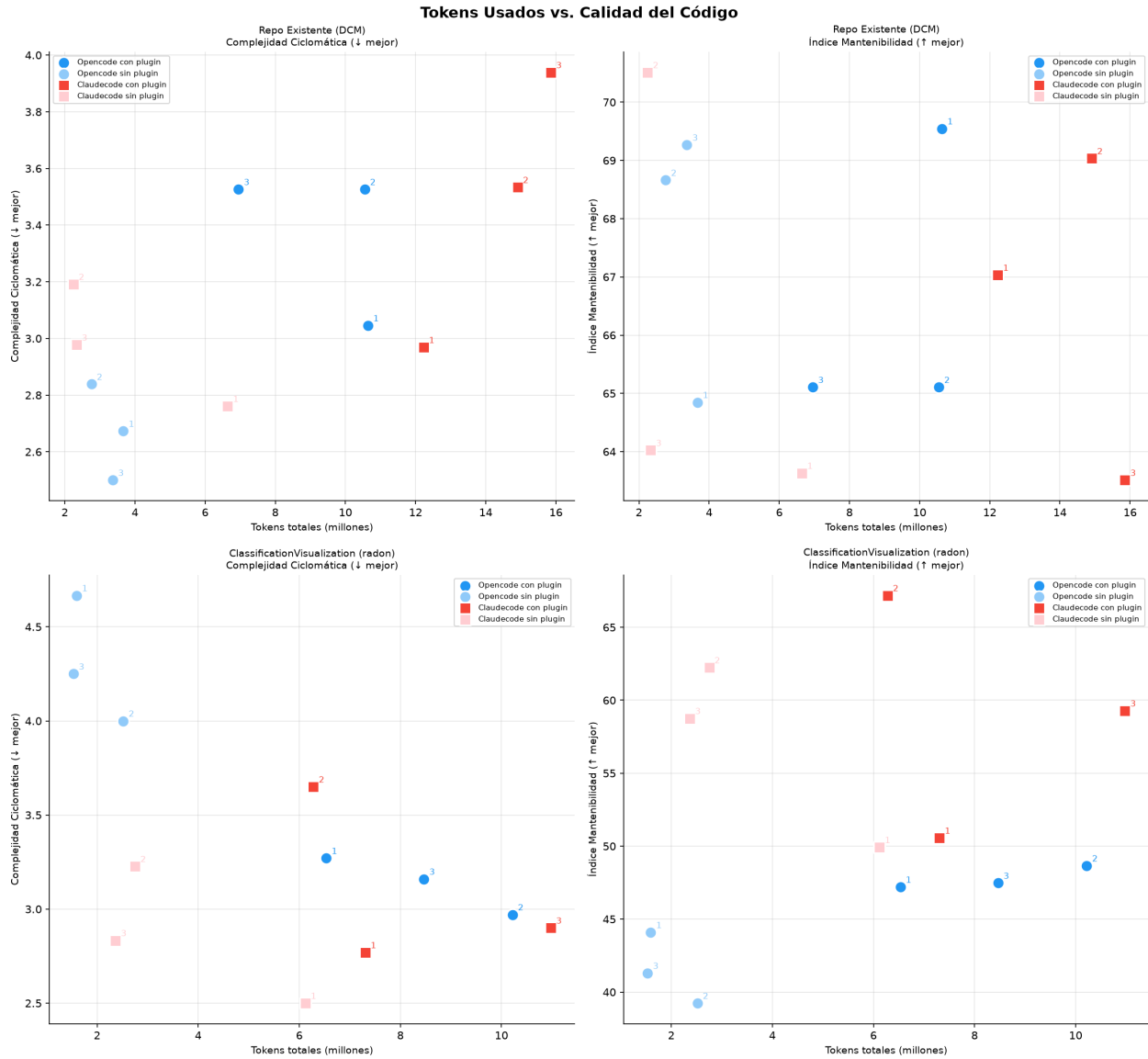


Figura 31: Dispersión: tokens totales vs. métricas de calidad del código en ambos experimentos. Fila superior: Repo Existente (DCM); fila inferior: ClassificationVisualization (radon). Cada punto representa una sesión individual.

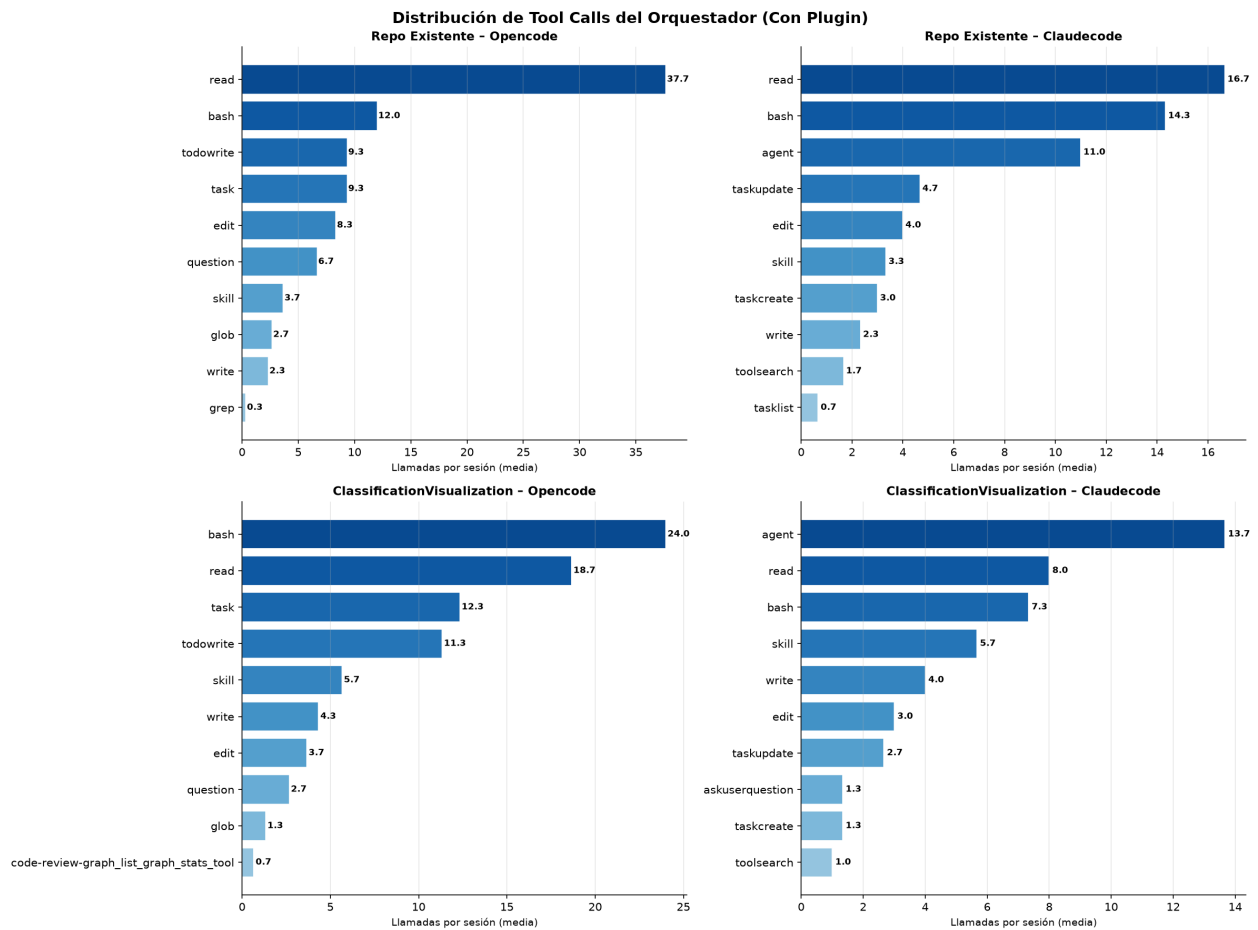


Figura 32: Distribución de tool calls del orquestador por tipo (top 10 herramientas, sesiones con colbPowers).